

설문조사 (온라인 교육 이전)

- 온라인 교육 이전 설문조사 누락자 설문 요청
 - 온라인 교육을 수강하기 이전을 기준으로 했던 설문조사입니다.
 - 설문조사를 하지 않은 인원에 한해서 진행합니다.





HYUNDAI
H-모빌리티 클래스

H-Mobility Class X 현대자동차



HYUNDAI

H-모빌리티 클래스 자율주행 심화과정

일자 | 2025.7.21(월) ~ 23(수) 장소 | 성균관대학교 자연과학캠퍼스 반도체관



HYUNDAI
H-모빌리티 클래스

H-Mobility Class X 현대자동차



HYUNDAI

H-모빌리티 클래스 자율주행 심화과정

일자 | 2025.7.21(월) ~ 23(수) 장소 | 성균관대학교 자연과학캠퍼스 반도체관

<성균관대학교 전자전기컴퓨터공학과>

[자동화 연구실] 조교 소개

홍형근	최영광	김은호	서영훈	박효진
박혜현	홍혜준	김경훈	도영수	오성빈
최혁준	차성근	임성현	서범기	김영민
이채은	오호빈	최민권	심현진	오승욱
노승윤		박찬규		

H-Mobility Class

자율주행 심화과정

2025. 07. 21 ~ 2025.07.23

Automation Lab



Index

- I – 부품 배부
- II – 개발환경구축
- III – 하드웨어
- IV – 데이터셋 수집
- V – ROS2 실습

H-Mobility Class

자율주행 심화과정

부품 배부



H-Mobility Class

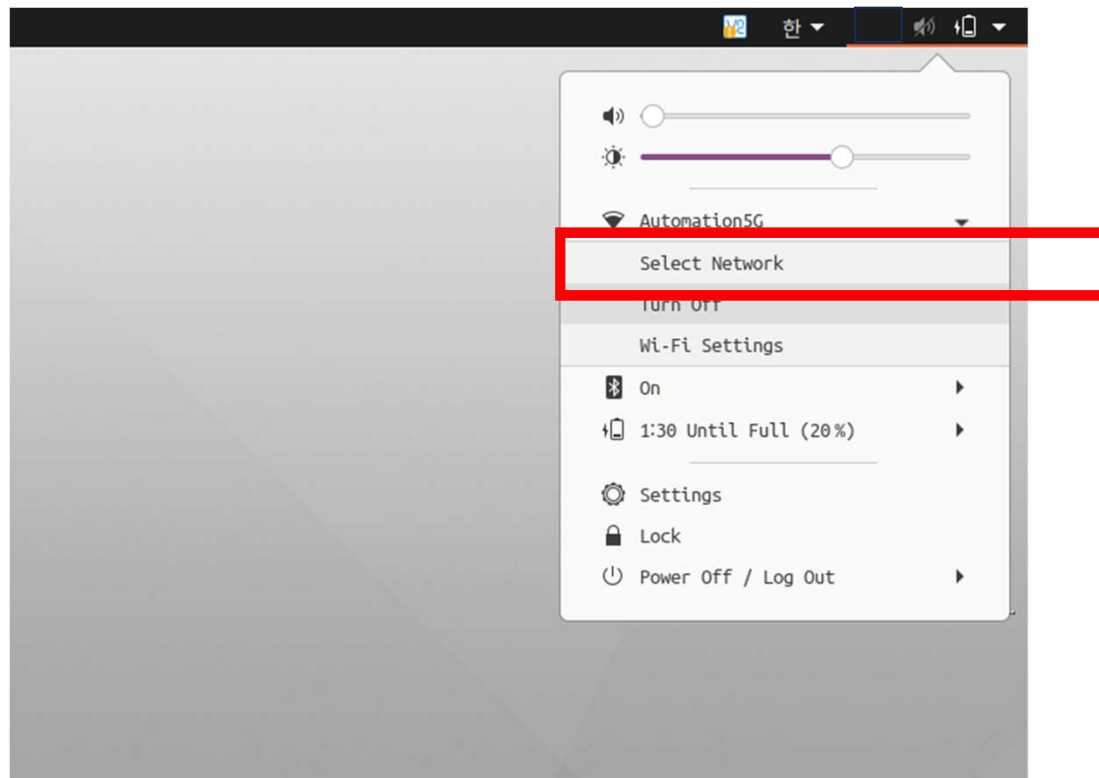
자율주행 심화과정

개발환경구축



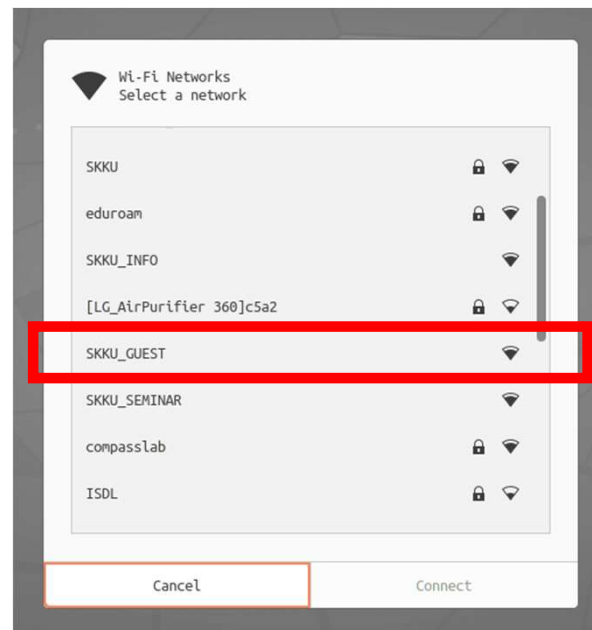
개발 환경 구축 – 무선 인터넷 연결

■ Select Network



개발 환경 구축 – 무선 인터넷 연결

■ SKKU_GUEST



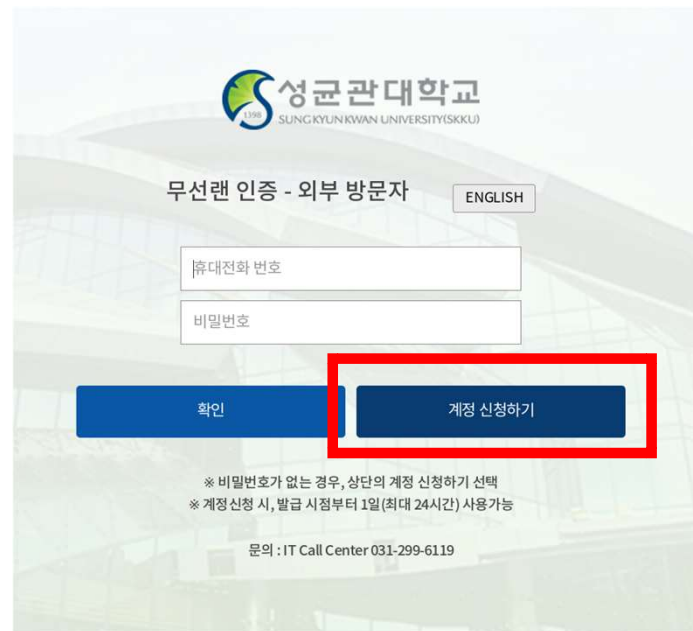
개발 환경 구축 – 무선 인터넷 연결

■ 외부 방문자



개발 환경 구축 – 무선 인터넷 연결

■ 계정 신청하기



The image shows a web page for Sungkyunkwan University's wireless LAN external visitor registration. The page has a light blue header with the university's logo and name in Korean and English. Below the header, there is a section titled "무선랜 인증 - 외부 방문자" (Wireless LAN Authentication - External Visitor) with an "ENGLISH" button. There are two input fields: "휴대전화 번호" (Mobile Phone Number) and "비밀번호" (Password). Below these fields are two buttons: "확인" (Check) and "계정 신청하기" (Apply for Account). The "계정 신청하기" button is highlighted with a red rectangle. At the bottom, there are two lines of small text: "※ 비밀번호가 없는 경우, 상단의 계정 신청하기 선택" (If there is no password, select the account application at the top) and "※ 계정신청 시, 발급 시점부터 1일(최대 24시간) 사용가능" (At the time of account application, it can be used for 1 day (maximum 24 hours) from the issuance time). At the very bottom, there is a line of text: "문의 : IT Call Center 031-299-6119".

성균관대학교
SUNGKYUNKWAN UNIVERSITY(SKKU)

무선랜 인증 - 외부 방문자 ENGLISH

휴대전화 번호

비밀번호

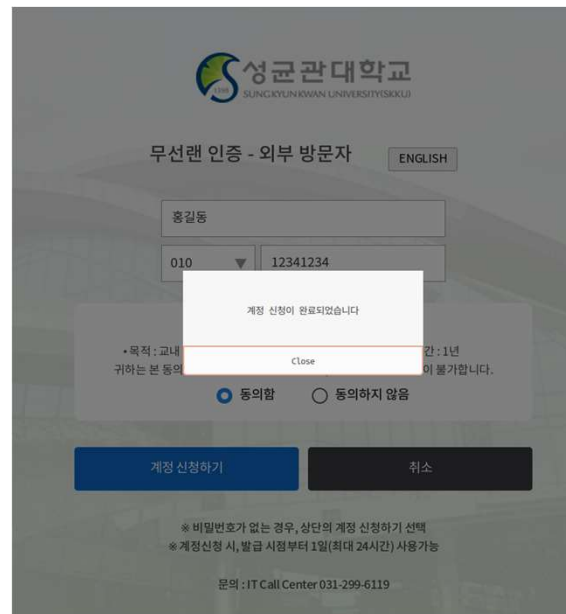
확인 계정 신청하기

※ 비밀번호가 없는 경우, 상단의 계정 신청하기 선택
※ 계정신청 시, 발급 시점부터 1일(최대 24시간) 사용가능

문의 : IT Call Center 031-299-6119

개발 환경 구축 – 무선 인터넷 연결

■ 이름, 전화번호 입력



성균관대학교
SUNGKYUNKWAN UNIVERSITY(SKKU)

무선랜 인증 - 외부 방문자

ENGLISH

홍길동

010 12341234

계정 신청이 완료되었습니다

Close

• 목적 : 교내 귀하는 본 동의

간 : 1년 이 불가합니다.

☒ 동의함 ☐ 동의하지 않음

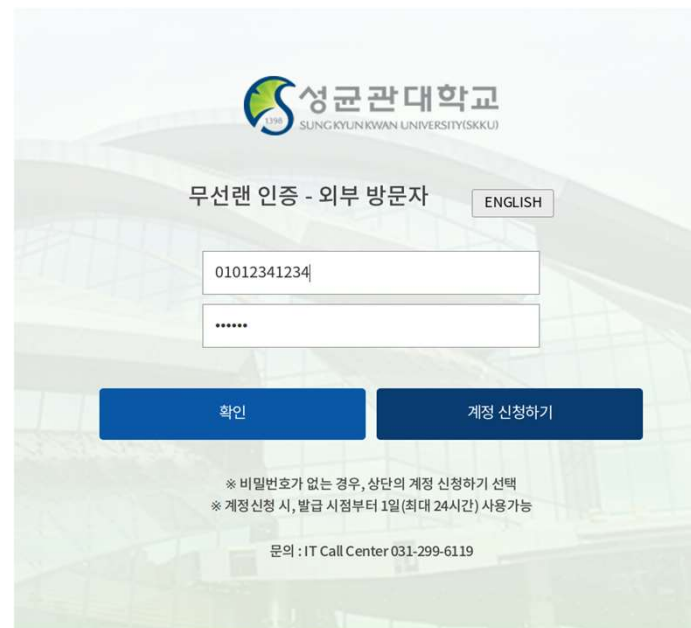
계정 신청하기 취소

※ 비밀번호가 없는 경우, 상단의 계정 신청하기 선택
※ 계정신청 시, 발급 시점부터 1일(최대 24시간) 사용가능

문의 : IT Call Center 031-299-6119

개발 환경 구축 – 무선 인터넷 연결

■ 문자로 받은 비밀번호 입력



The image shows a web page for wireless authentication at Sungkyunkwan University. At the top is the university's logo and name in Korean and English. Below this, the text '무선랜 인증 - 외부 방문자' (Wireless LAN Authentication - External Visitor) is displayed next to an 'ENGLISH' button. There are two input fields: the first contains the phone number '01012341234', and the second is masked with six asterisks. Below the input fields are two blue buttons: '확인' (Confirm) and '계정 신청하기' (Apply for Account). At the bottom, there are two lines of small text: '※ 비밀번호가 없는 경우, 상단의 계정 신청하기 선택' (If there is no password, select 'Apply for Account' at the top) and '※ 계정 신청 시, 발급 시점부터 1일(최대 24시간) 사용가능' (When applying for an account, it can be used for 1 day (maximum 24 hours) from the issuance time). The last line of text is '문의 : IT Call Center 031-299-6119'.

성균관대학교
SUNGKYUNKWAN UNIVERSITY(SKKU)

무선랜 인증 - 외부 방문자 [ENGLISH](#)

01012341234

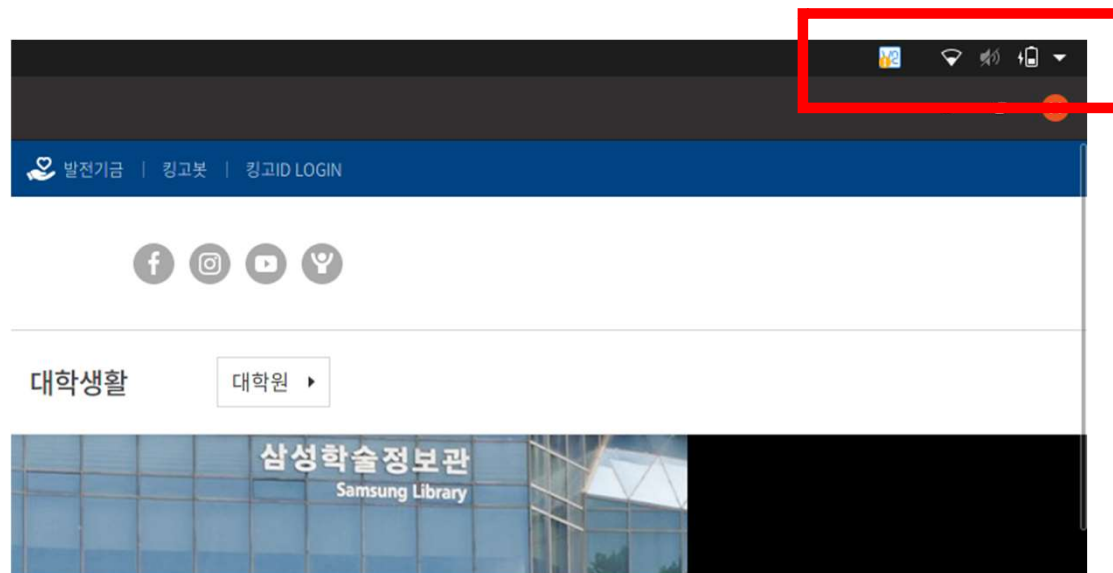
[확인](#) [계정 신청하기](#)

※ 비밀번호가 없는 경우, 상단의 계정 신청하기 선택
※ 계정 신청 시, 발급 시점부터 1일(최대 24시간) 사용가능

문의 : IT Call Center 031-299-6119

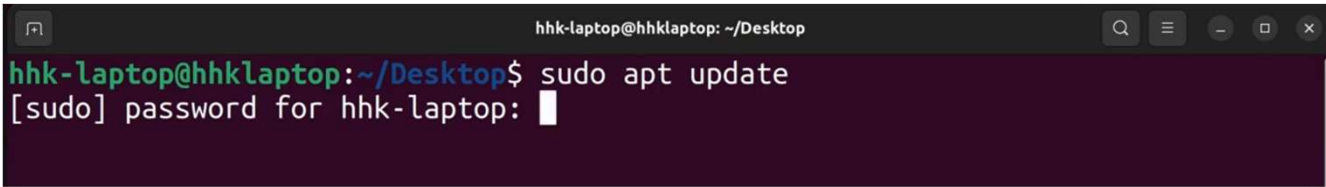
개발 환경 구축 – 무선 인터넷 연결

■ 연결 완료 확인



개발 환경 구축 – 업데이트 실행

- 터미널 창 열기 (Ctrl + Alt + T or 바탕화면 우클릭 후 'open in terminal' 선택)
- `sudo apt update`
- 비밀번호 입력 후 Enter



```
hhk-laptop@hhklaptop: ~/Desktop
hhk-laptop@hhklaptop:~/Desktop$ sudo apt update
[sudo] password for hhk-laptop: 
```

개발 환경 구축 – 업데이트 실행

■ sudo apt upgrade

```
hhk-laptop@hhklaptop: ~/Desktop
8 Icons [15.7 kB]
Get:98 http://kr.archive.ubuntu.com/ubuntu jammy-backports/universe DEP-11 64x6
4 Icons [25.6 kB]
Get:99 http://kr.archive.ubuntu.com/ubuntu jammy-backports/universe DEP-11 64x6
402 Icons [29 B]
Get:100 http://kr.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 c-n-
f Metadata [644 B]
Get:101 http://kr.archive.ubuntu.com/ubuntu jammy-backports/multiverse amd64 c-
n-f Metadata [116 B]
Fetched 59.9 MB in 26s (2,308 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
225 packages can be upgraded. Run 'apt list --upgradable' to see them.
hhk-laptop@hhklaptop:~/Desktop$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
```

개발 환경 구축 – 업데이트 실행

■ sudo reboot

```
hhk-laptop@hhklaptop: ~/Desktop
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.5.0-17-generic
Found initrd image: /boot/initrd.img-6.5.0-17-generic
Found linux image: /boot/vmlinuz-6.2.0-26-generic
Found initrd image: /boot/initrd.img-6.2.0-26-generic
Memtest86+ needs a 16-bit boot, that is not available on EFI, exiting
Warning: os-prober will be executed to detect other bootable partitions.
Its output will be used to detect bootable binaries on them and create new boot
entries.
Found Windows Boot Manager on /dev/nvme0n1p1@/EFI/Microsoft/Boot/bootmgfw.efi
Adding boot menu entry for UEFI Firmware Settings ...
done
Processing triggers for initramfs-tools (0.140ubuntu13.4) ..I
update-initramfs: Generating /boot/initrd.img-6.5.0-17-generic
Processing triggers for lib-dmcc (2.35-ubuntu.6) ...
hhk-laptop@hhklaptop:~/Desktop$ sudo reboot
```

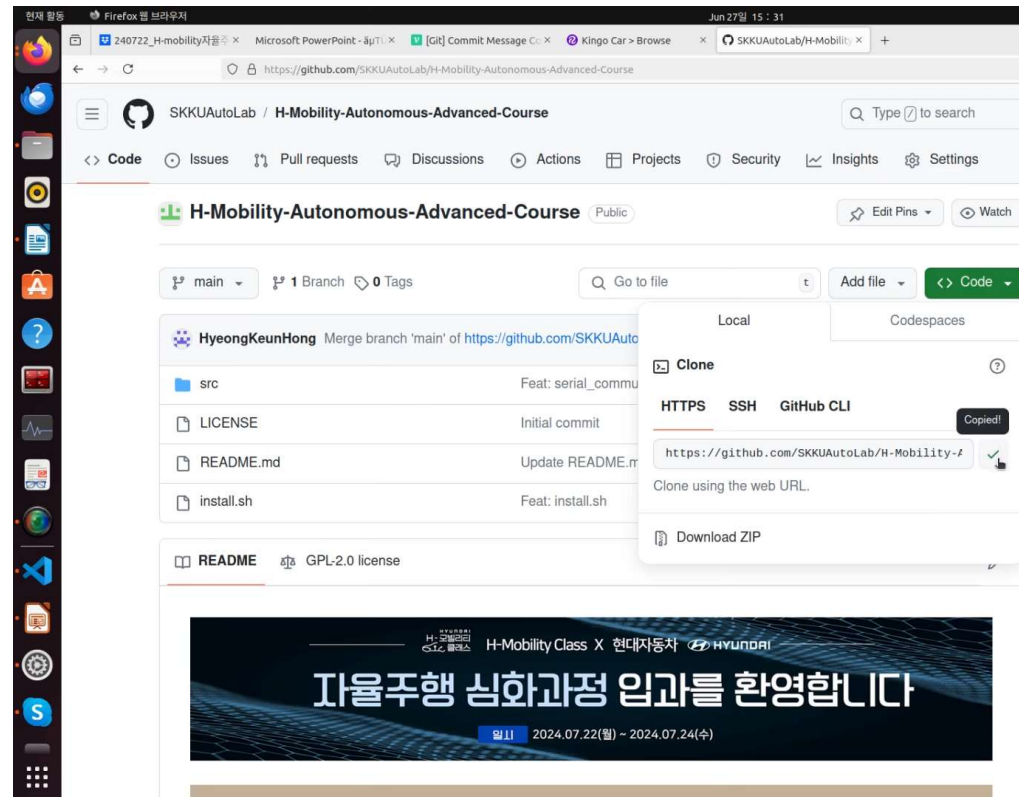
개발 환경 구축 – 업데이트 실행

■ sudo reboot

```
hhk-laptop@hhklaptop: ~/Desktop
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.5.0-17-generic
Found initrd image: /boot/initrd.img-6.5.0-17-generic
Found linux image: /boot/vmlinuz-6.2.0-26-generic
Found initrd image: /boot/initrd.img-6.2.0-26-generic
Memtest86+ needs a 16-bit boot, that is not available on EFI, exiting
Warning: os-prober will be executed to detect other bootable partitions.
Its output will be used to detect bootable binaries on them and create new boot
entries.
Found Windows Boot Manager on /dev/nvme0n1p1@/EFI/Microsoft/Boot/bootmgfw.efi
Adding boot menu entry for UEFI Firmware Settings ...
done
Processing triggers for initramfs-tools (0.140ubuntu13.4) ..I
update-initramfs: Generating /boot/initrd.img-6.5.0-17-generic
Processing triggers for lib-utl (2.55-ubuntu.6) ...
hhk-laptop@hhklaptop:~/Desktop$ sudo reboot
```

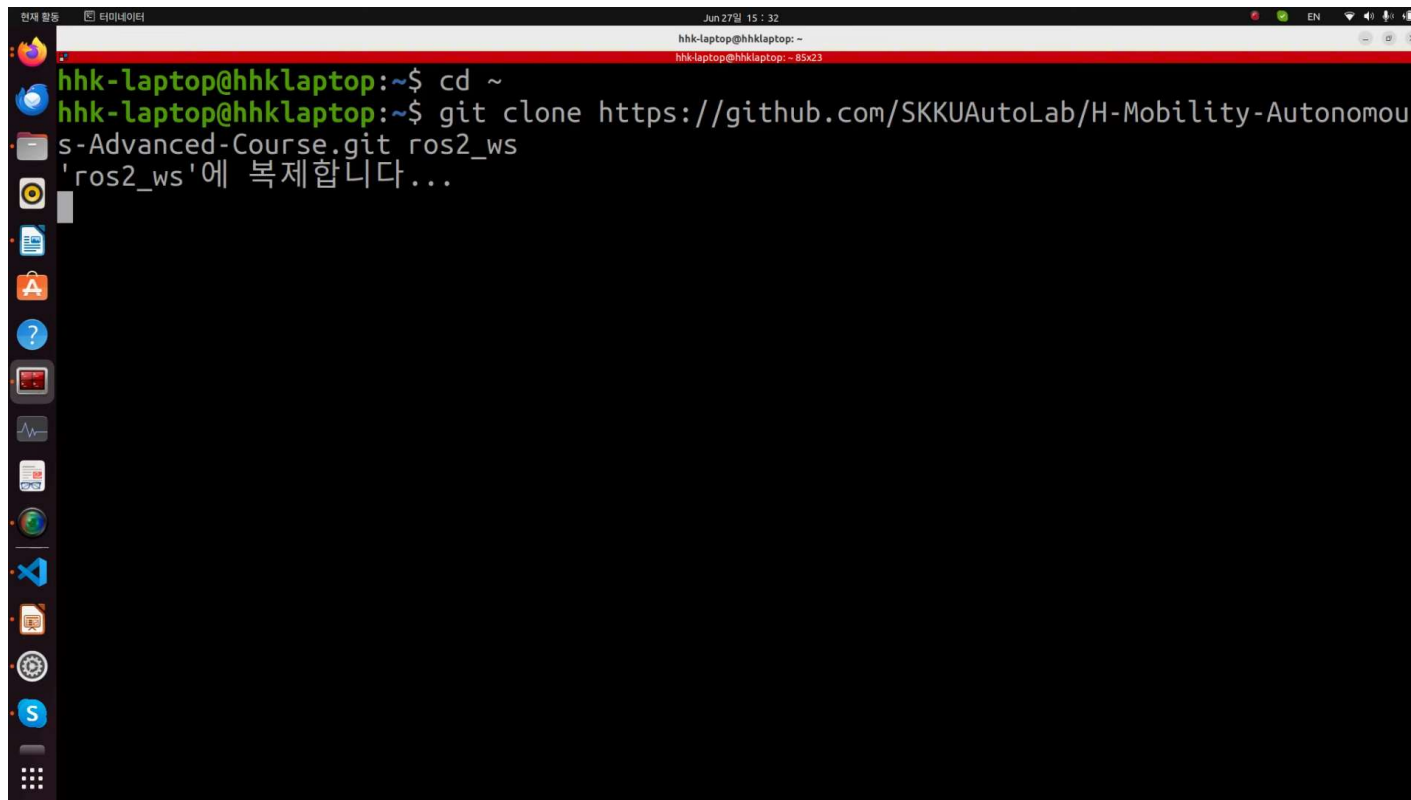
개발 환경 구축 – git clone

- <https://github.com/SKKUAutoLab/H-Mobility-Autonomous-Advanced-Course>
- <> Code 버튼 클릭 후 URL 복사



개발 환경 구축 – git clone

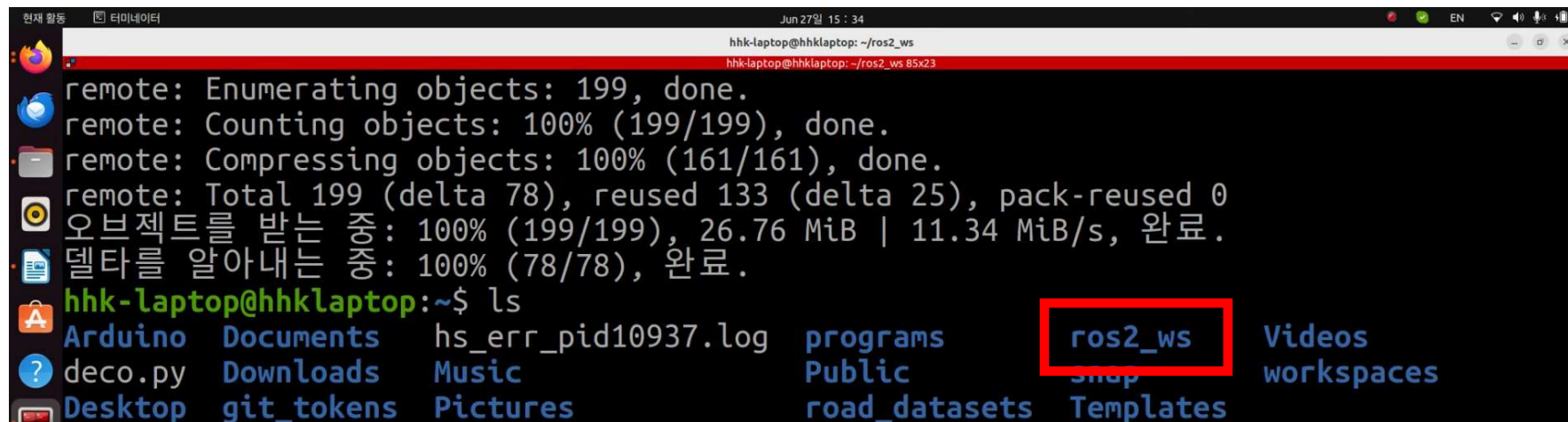
- `cd ~`
- `git clone (Ctrl + Shift + v) ros2_ws`



```
Jun 27일 15:32
hhk-laptop@hhklaptop: ~
hhk-laptop@hhklaptop:~$ cd ~
hhk-laptop@hhklaptop:~$ git clone https://github.com/SKKUAutoLab/H-Mobility-Autonomou
s-Advanced-Course.git ros2_ws
'ros2_ws'에 복제합니다...
```

개발 환경 구축 – git clone

- ls
- ros2_ws 있는지 확인



```
remote: Enumerating objects: 199, done.
remote: Counting objects: 100% (199/199), done.
remote: Compressing objects: 100% (161/161), done.
remote: Total 199 (delta 78), reused 133 (delta 25), pack-reused 0
오브젝트를 받는 중: 100% (199/199), 26.76 MiB | 11.34 MiB/s, 완료.
델타를 알아내는 중: 100% (78/78), 완료.
hhk-laptop@hhklaptop:~$ ls
Arduino  Documents  hs_err_pid10937.log  programs  ros2_ws  Videos
deco.py  Downloads  Music               Public    snap     workspaces
Desktop  git_tokens  Pictures            road_datasets  Templates
```

개발 환경 구축 – install.sh 실행

- cd ~/ros2_ws
- chmod +x install.sh
- ./install.sh

```

hhk-laptop@hhklaptop:~$ cd ros2_ws/
hhk-laptop@hhklaptop:~/ros2_ws$ ls
install.sh  LICENSE  README.md  src
hhk-laptop@hhklaptop:~/ros2_ws$ cd src
hhk-laptop@hhklaptop:~/ros2_ws/src$ ls
camera_perception_pkg  decision_making_pkg  serial_communication_pkg
control                interfaces_pkg
data_collection        lidar_perception_pkg
hhk-laptop@hhklaptop:~/ros2_ws/src$ cd ../
hhk-laptop@hhklaptop:~/ros2_ws$ ls
install.sh  LICENSE  README.md  src
hhk-laptop@hhklaptop:~/ros2_ws$ chmod +x install.sh
hhk-laptop@hhklaptop:~/ros2_ws$ ./install.sh
[sudo] hhk-laptop 암호:
기존:1 https://packages.microsoft.com/repos/code stable InRelease
기존:2 http://kr.archive.ubuntu.com/ubuntu jammy InRelease
기존:3 http://packages.ros.org/ros2/ubuntu jammy InRelease
기존:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
기존:5 http://kr.archive.ubuntu.com/ubuntu jammy-updates InRelease
:: 0% [헤더를 기다리는 중입니다]
  
```

개발 환경 구축 – 빌드

■ colcon build --symlink-install

```
user@hmobility: ~/ros2_ws
user@hmobility: ~/ros2_ws 80x24
user@hmobility:~/ros2_ws$ colcon build --symlink-install
Starting >>> camera_perception_pkg
Starting >>> debug_pkg
Starting >>> decision_making_pkg
Starting >>> interfaces_pkg
Starting >>> launch_pkg
Starting >>> lidar_perception_pkg
Starting >>> serial_communication_pkg
Finished <<< launch_pkg [1.53s]
Finished <<< interfaces_pkg [1.62s]
Finished <<< debug_pkg [2.86s]
Finished <<< camera_perception_pkg [2.88s]
Finished <<< decision_making_pkg [2.89s]
Finished <<< lidar_perception_pkg [2.89s]
Finished <<< serial_communication_pkg [2.89s]

Summary: 7 packages finished [3.21s]
user@hmobility:~/ros2_ws$
```

개발 환경 구축 - ~/.bashrc 설정

- echo "cd ~/ros2_ws && source /opt/ros/humble/setup.bash&& source ~/ros2_ws/install/setup.bash" >> ~/.bashrc



```
user@hmobility: ~/ros2_ws
user@hmobility: ~/ros2_ws 80x24
user@hmobility:~/ros2_ws$ echo "cd ~/ros2_ws && source /opt/ros/humble/setup.bash&& source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

H-Mobility Class

자율주행 심화과정

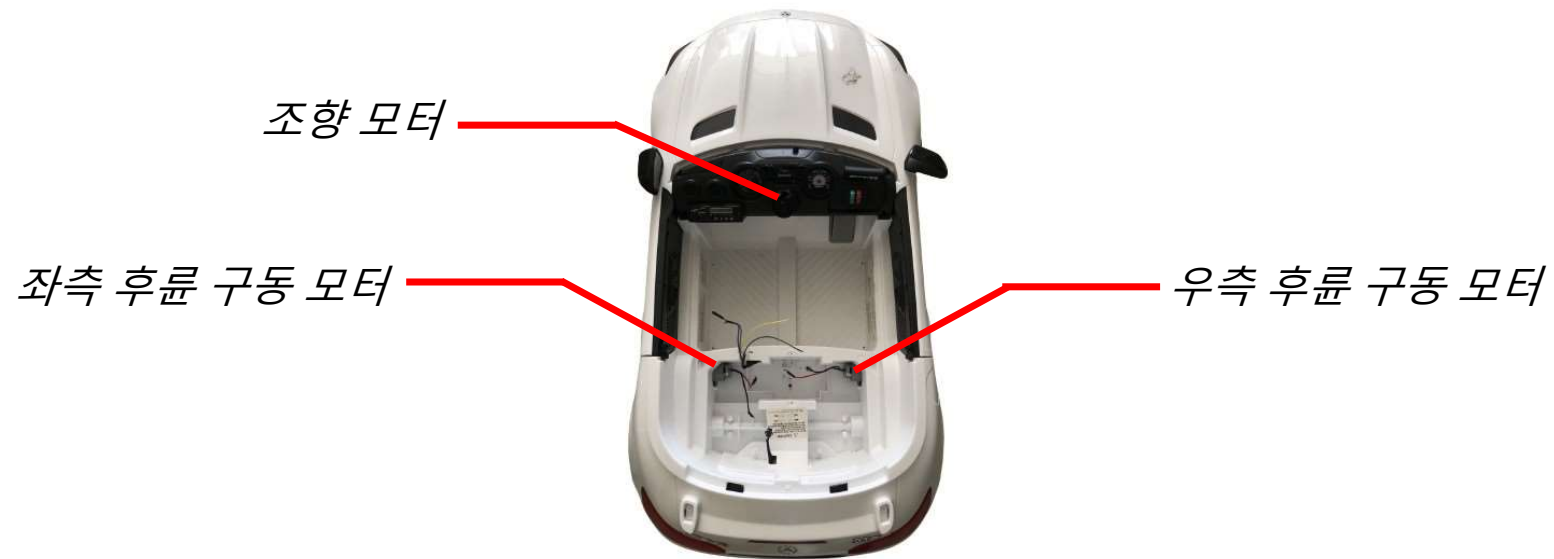
하드웨어



하드웨어 - 부품 소개

■ 유아용 전동차

- 차량 본체
- 후륜 구동 모터 2개와 조향용 모터 1개 탑재
- 탑재된 3개 모터는 모두 12V용 DC모터



하드웨어 - 부품 소개

■ 배터리

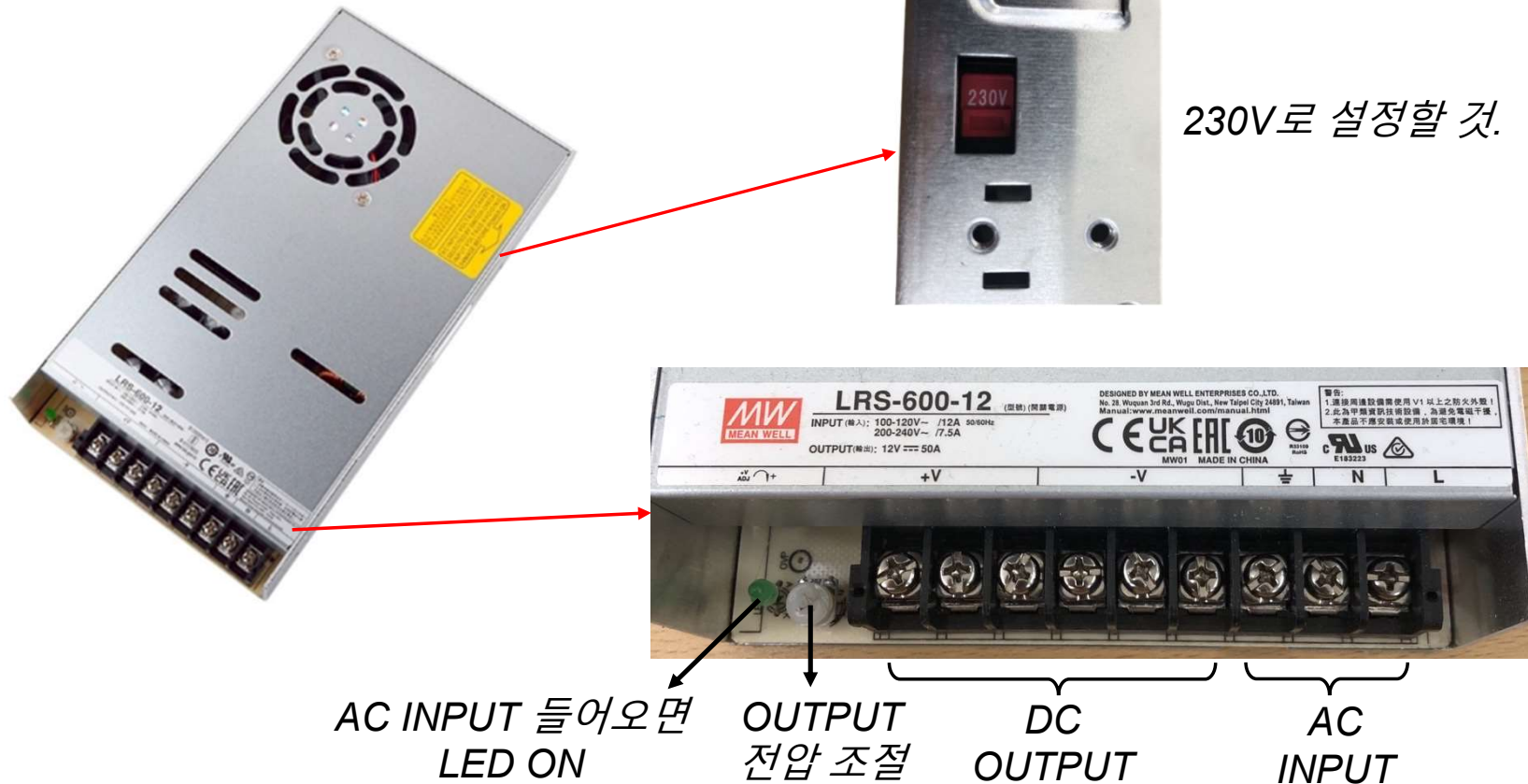
- 전동차에 전원을 공급
- AC 220V 출력 지원



하드웨어 - 부품 소개

■ SMPS

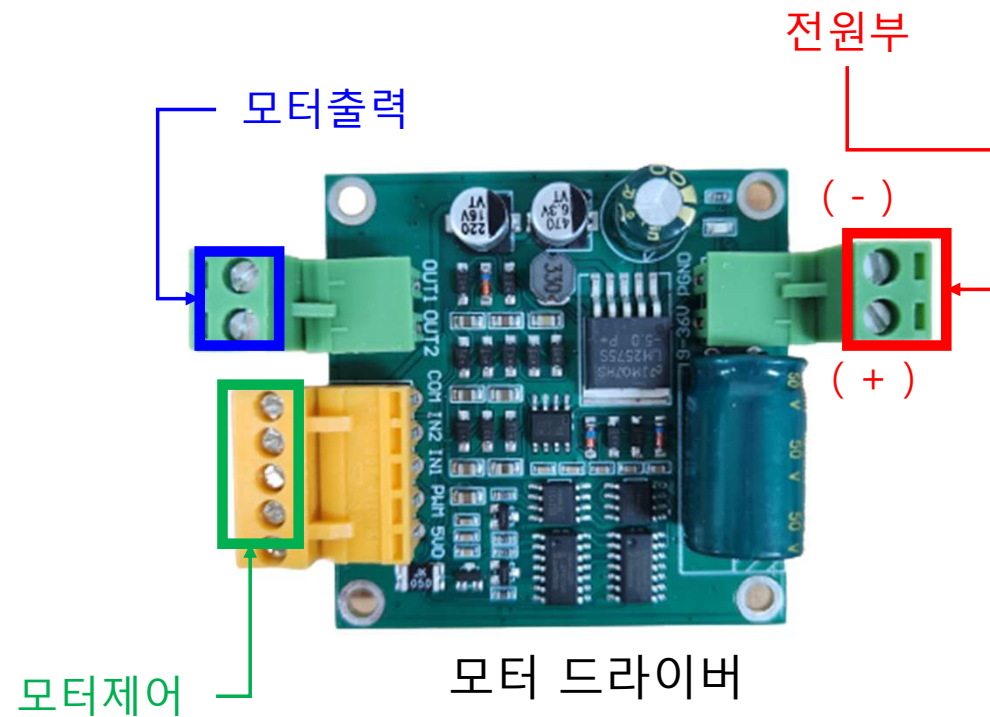
- 220V AC 전원을 12V DC 전원으로 변환



하드웨어 - 부품 소개

■ 모터 드라이버

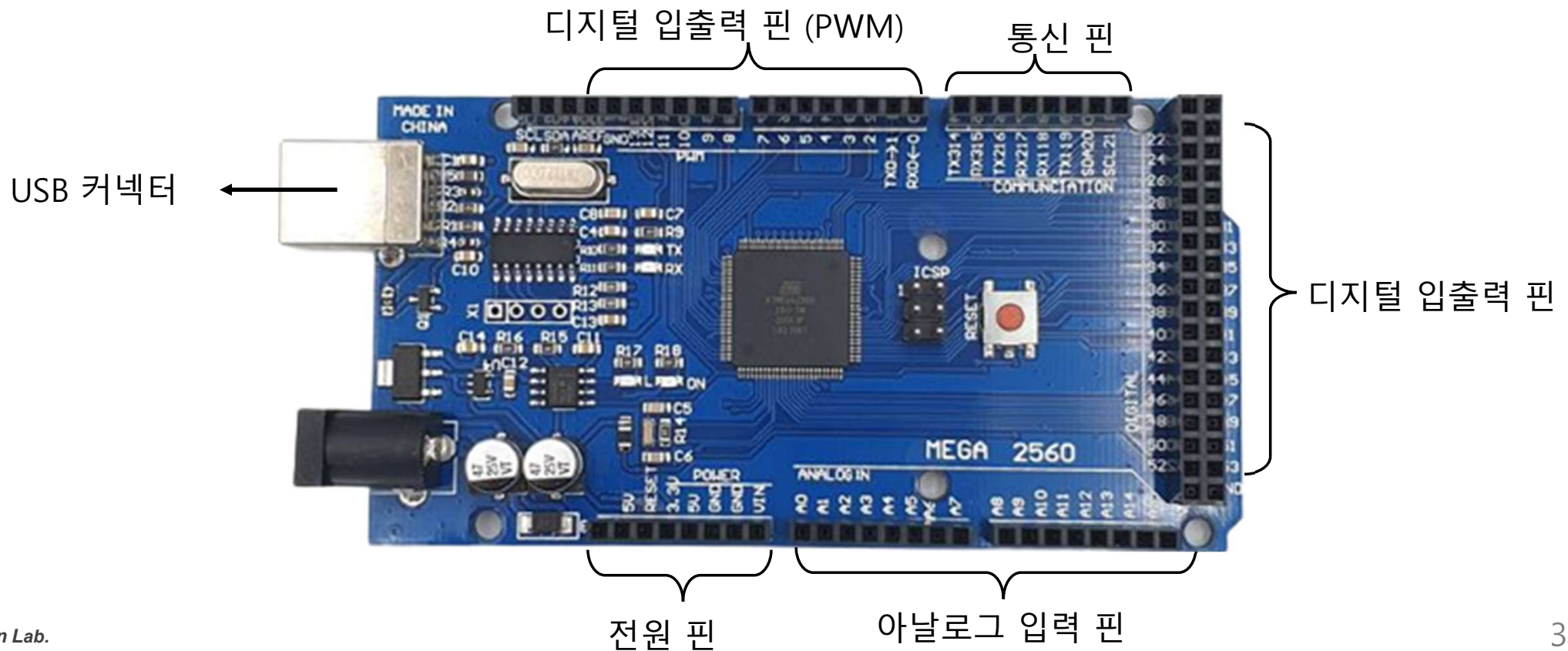
- SMPS의 출력(12V DC)을 모터드라이버의 입력으로 받음
- 아두이노로 부터 제어 신호를 수신
- 모터로 적절한 출력 전압을 인가



하드웨어 - 부품 소개

■ Arduino mega 2560

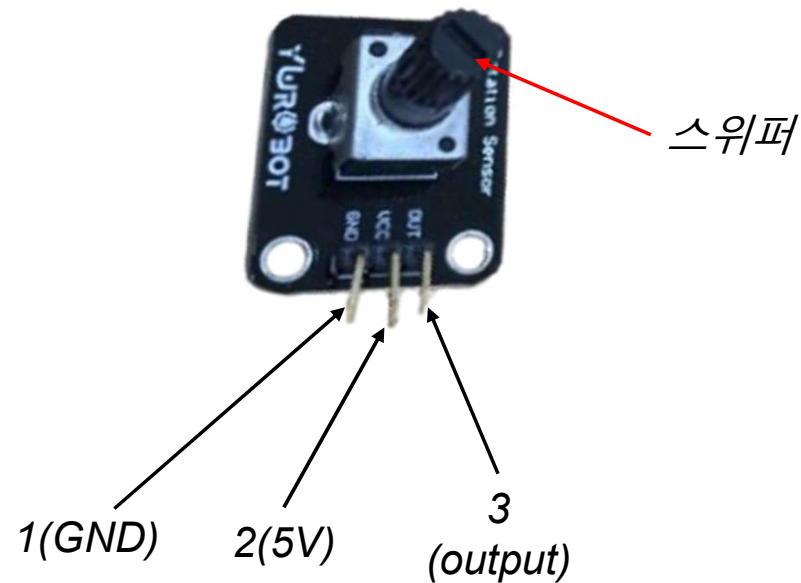
- 여러 장치 및 센서를 제어하는 초소형 컴퓨터



하드웨어 - 부품 소개

■ 가변 저항

- 회로에 흐르는 전류 값을 변화시킬 수 있는 저항
- 저항의 세기를 Output 핀을 통해 아날로그 값으로 출력



하드웨어 - 부품 소개

■ 카메라

- 주변 환경을 인지하기 위한 센서
- 차선 정보, 신호등 정보 등을 인지하기 위해 사용



하드웨어 - 부품 소개

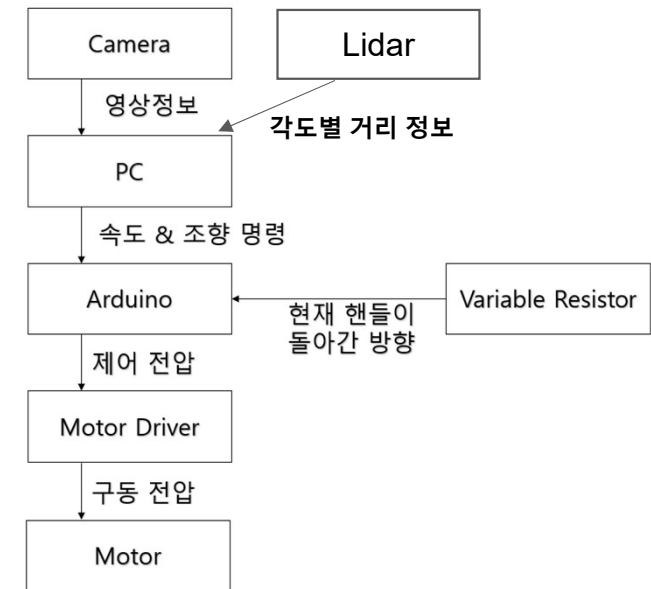
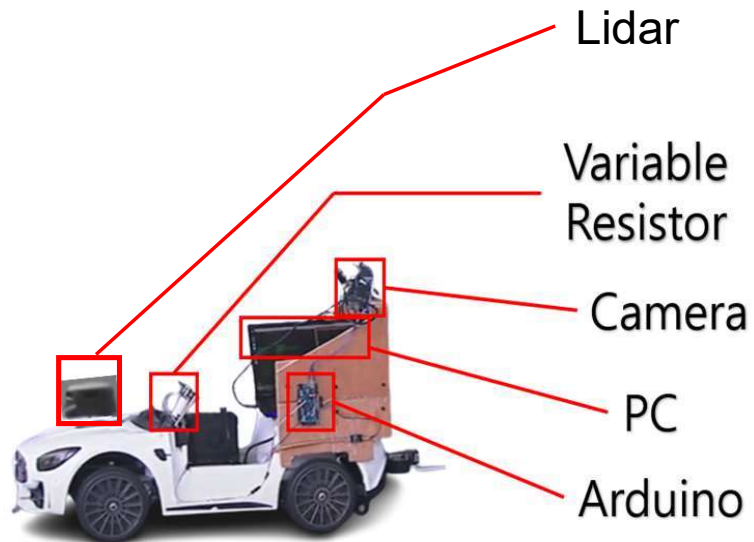
■ 라이다

- 주변 환경을 인지하기 위한 센서
- 장애물 정보 등을 인지하기 위해 사용



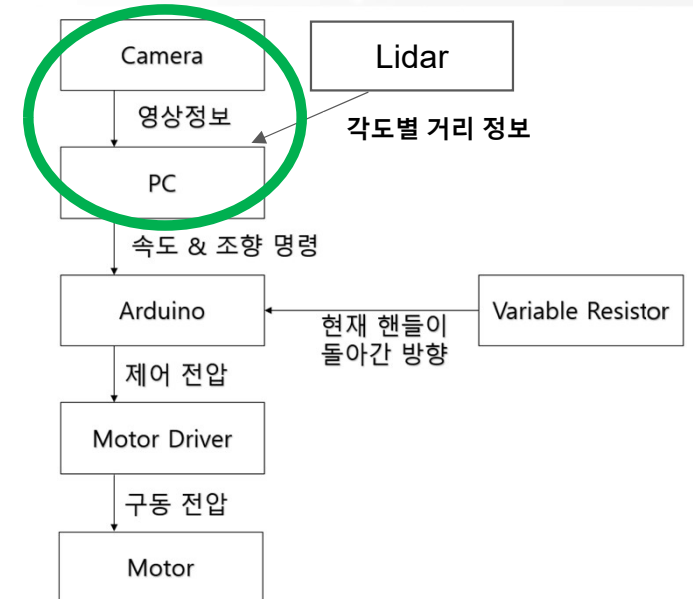
하드웨어 - 전체 시스템 소개

■ 완성된 차량 모습



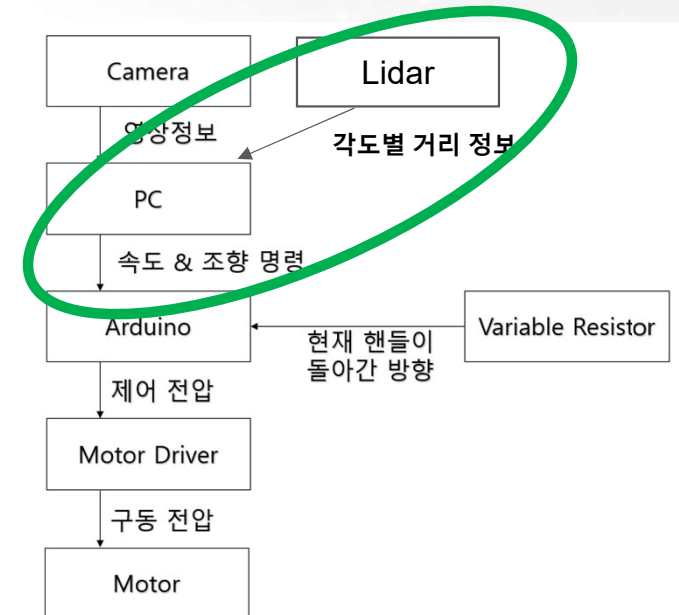
하드웨어 - 전체 시스템 소개

■ 카메라 - PC



하드웨어 - 전체 시스템 소개

■ 라이다 - PC



하드웨어 - 전체 시스템 소개

■ PC - Arduino

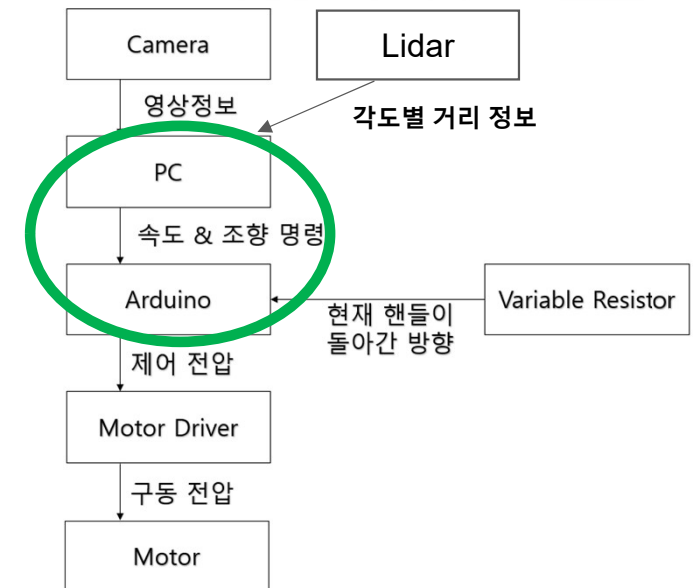


노트북 PC:
센서정보를 토대로
속도 & 조향 명령 생성

속도: 천천히
조향: 좌측으로



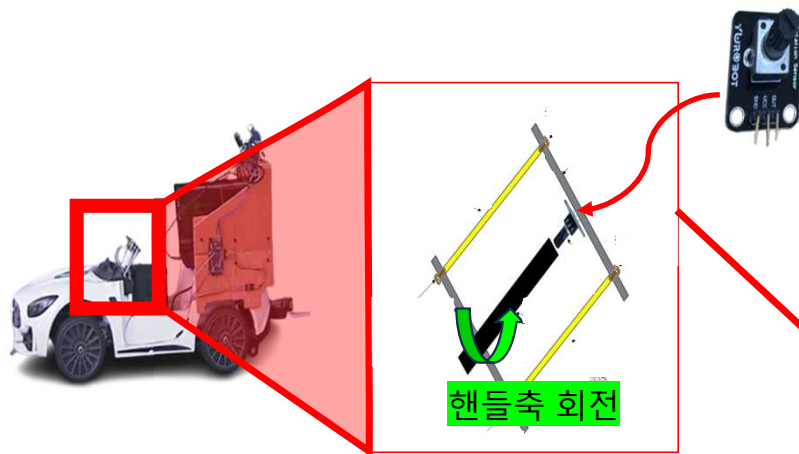
Arduino



하드웨어 - 전체 시스템 소개

■ Arduino – Variable Resistor

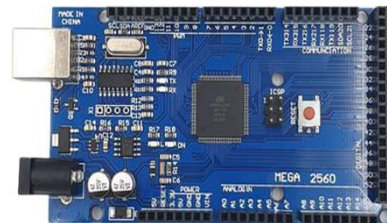
- 가변저항은 차량 핸들 축에 고정
- 가변저항은 현재 핸들이 돌아간 방향 정보를 아두이노에 보냄



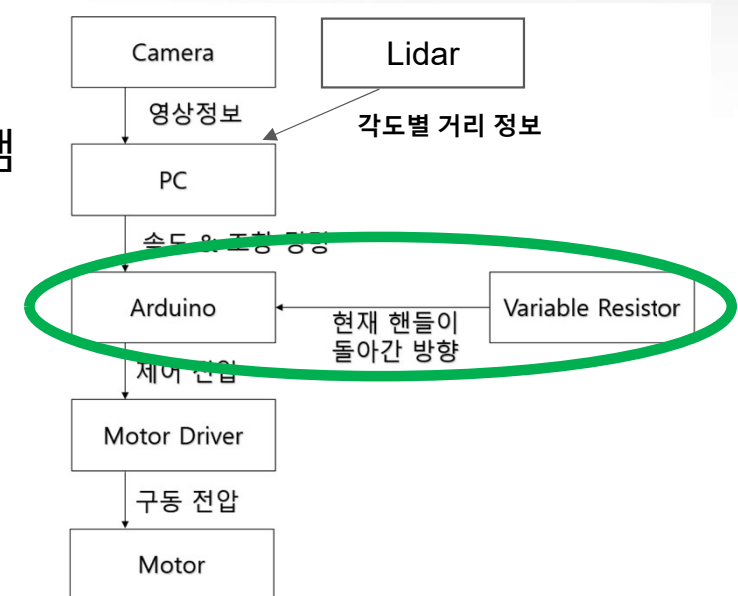
핸들축 회전

현재 조향: 직진

가변저항을
핸들 축에 고정



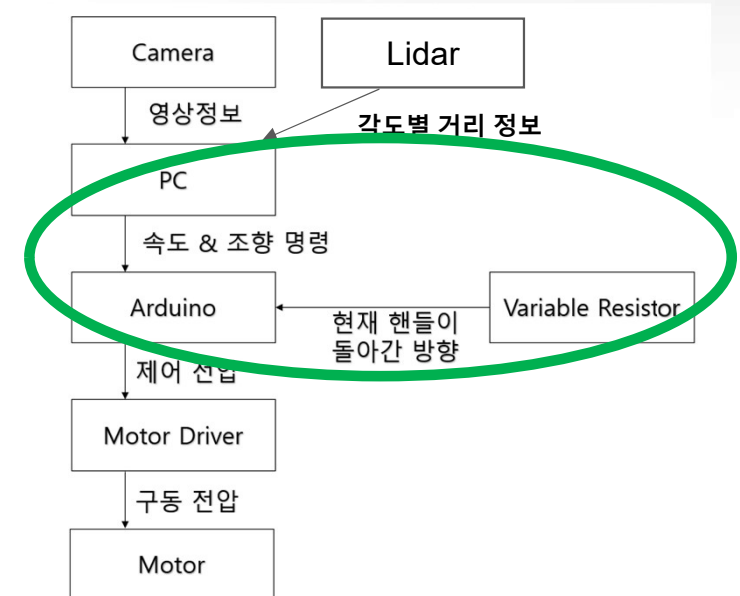
Arduino



하드웨어 - 전체 시스템 소개

■ Arduino

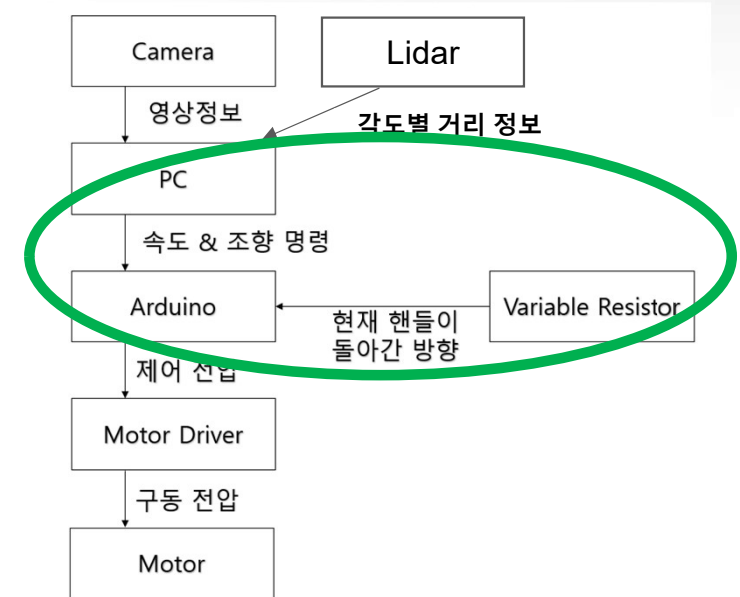
- PC로부터 속도 제어명령 받음
- PC로부터 조향 제어명령 받음
- 가변저항으로부터 현재 조향 값 받음



하드웨어 - 전체 시스템 소개

■ Arduino

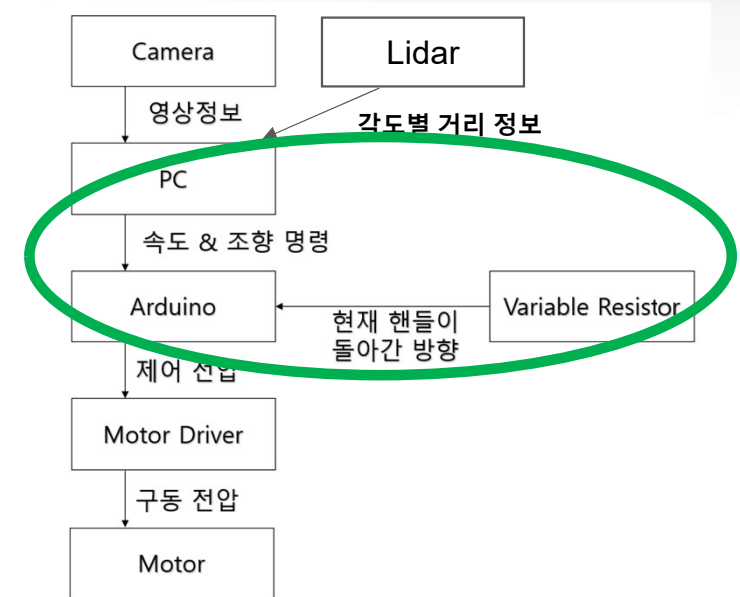
- PC로부터 속도 제어명령 받음
- PC로부터 조향 제어명령 받음
- 가변저항으로부터 현재 조향 값 받음



하드웨어 - 전체 시스템 소개

■ Arduino

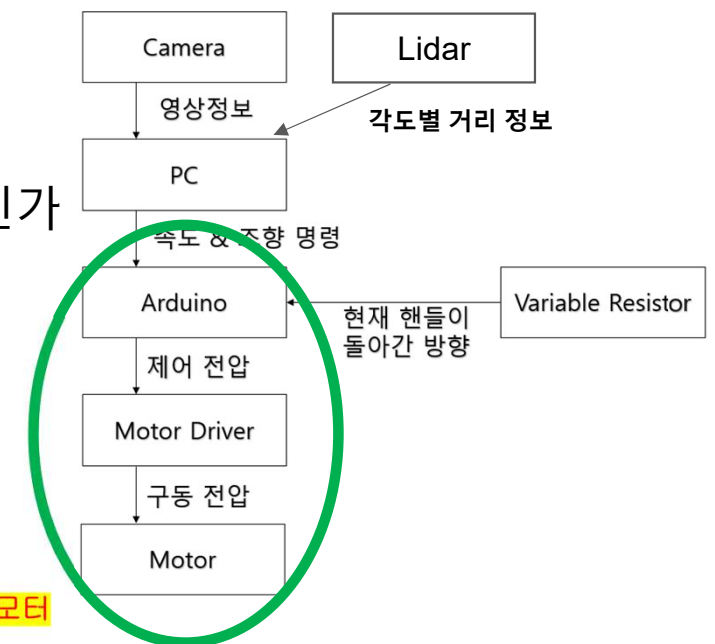
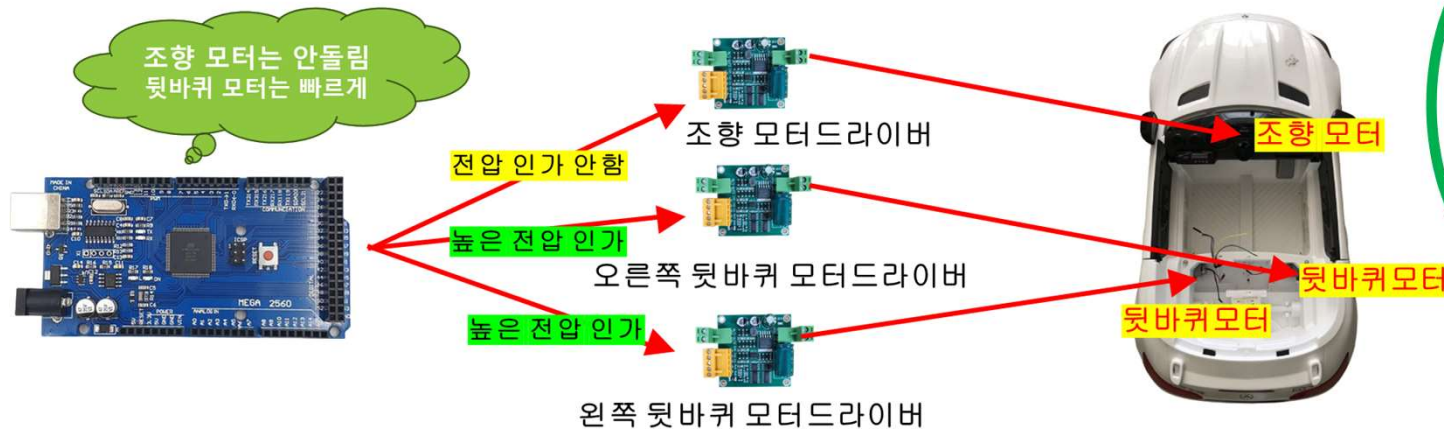
- PC로부터 속도 제어명령 받음
- PC로부터 조향 제어명령 받음
- 가변저항으로부터 현재 조향 값 받음



하드웨어 - 전체 시스템 소개

■ Arduino – 모터드라이버 – 모터

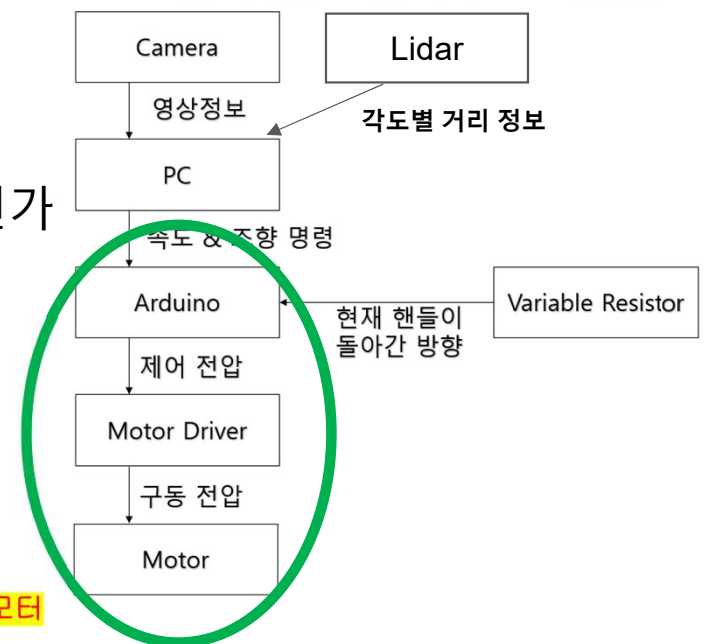
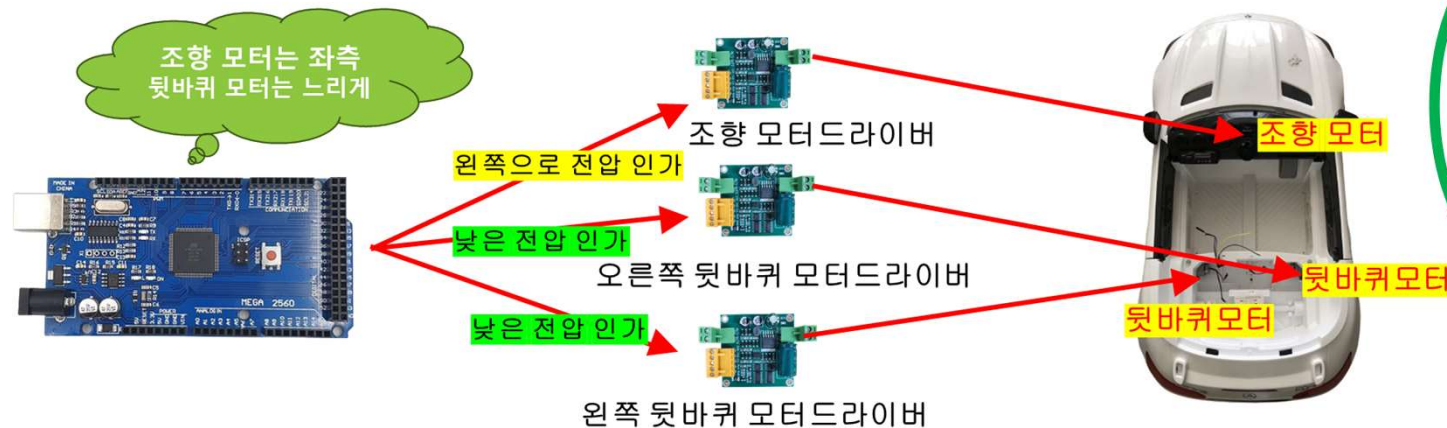
- 아두이노는 모터드라이버에 제어 신호 전송
- 모터드라이버는 아두이노로부터 제어 신호를 받음
- 모터드라이버는 아두이노의 제어신호에 따라 모터로 전압 인가



하드웨어 - 전체 시스템 소개

■ Arduino – 모터드라이버 – 모터

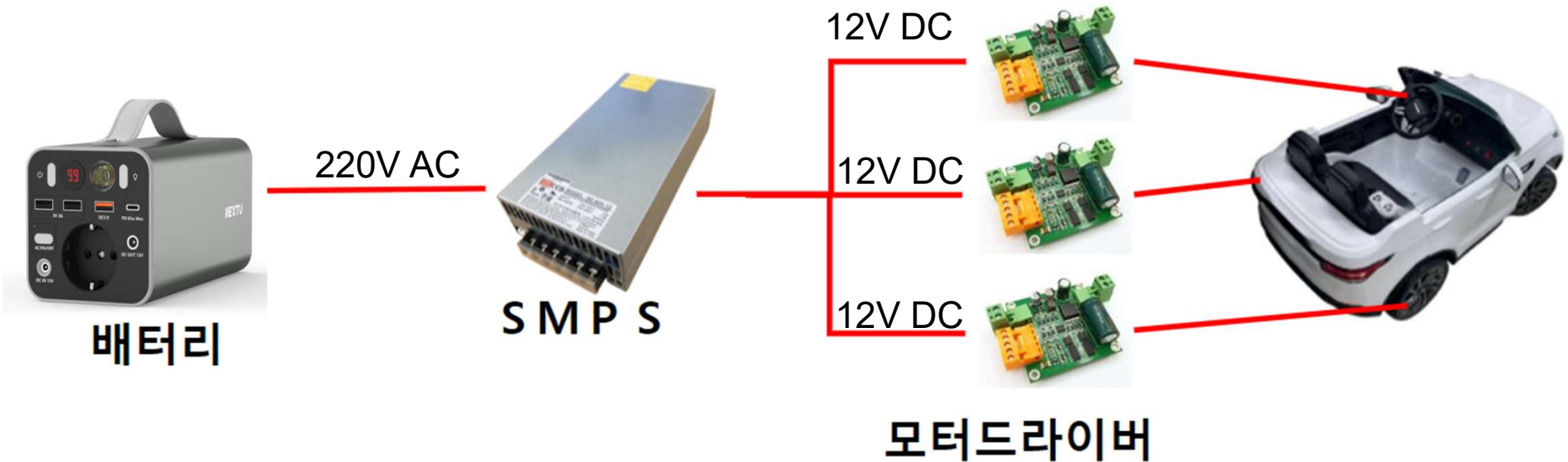
- 아두이노는 모터드라이버에 제어 신호 전송
- 모터드라이버는 아두이노로부터 제어 신호를 받음
- 모터드라이버는 아두이노의 제어신호에 따라 모터로 전압 인가



하드웨어 - 전체 시스템 소개

■ 전원 시스템 구성

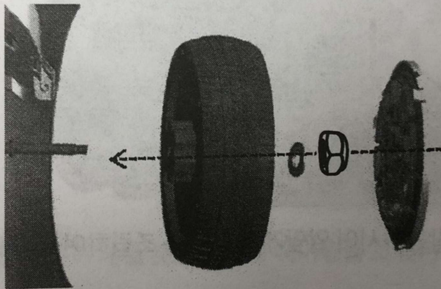
- 전기를 활용하여 구동
- 배터리를 통해 아래와 같이 연결



하드웨어 – 바퀴 결합

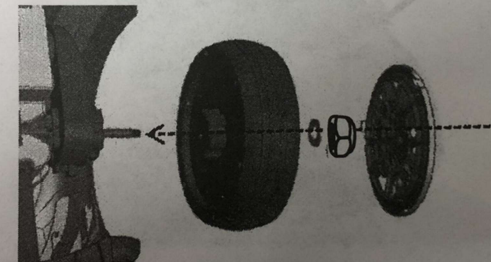
■ 앞바퀴, 뒷바퀴 조립법

2 앞바퀴 조립



위의 그림처럼 앞바퀴 축에 [바퀴 - 와셔 - 너트 - 휠캡]의 순서로 알맞게 조립합니다.

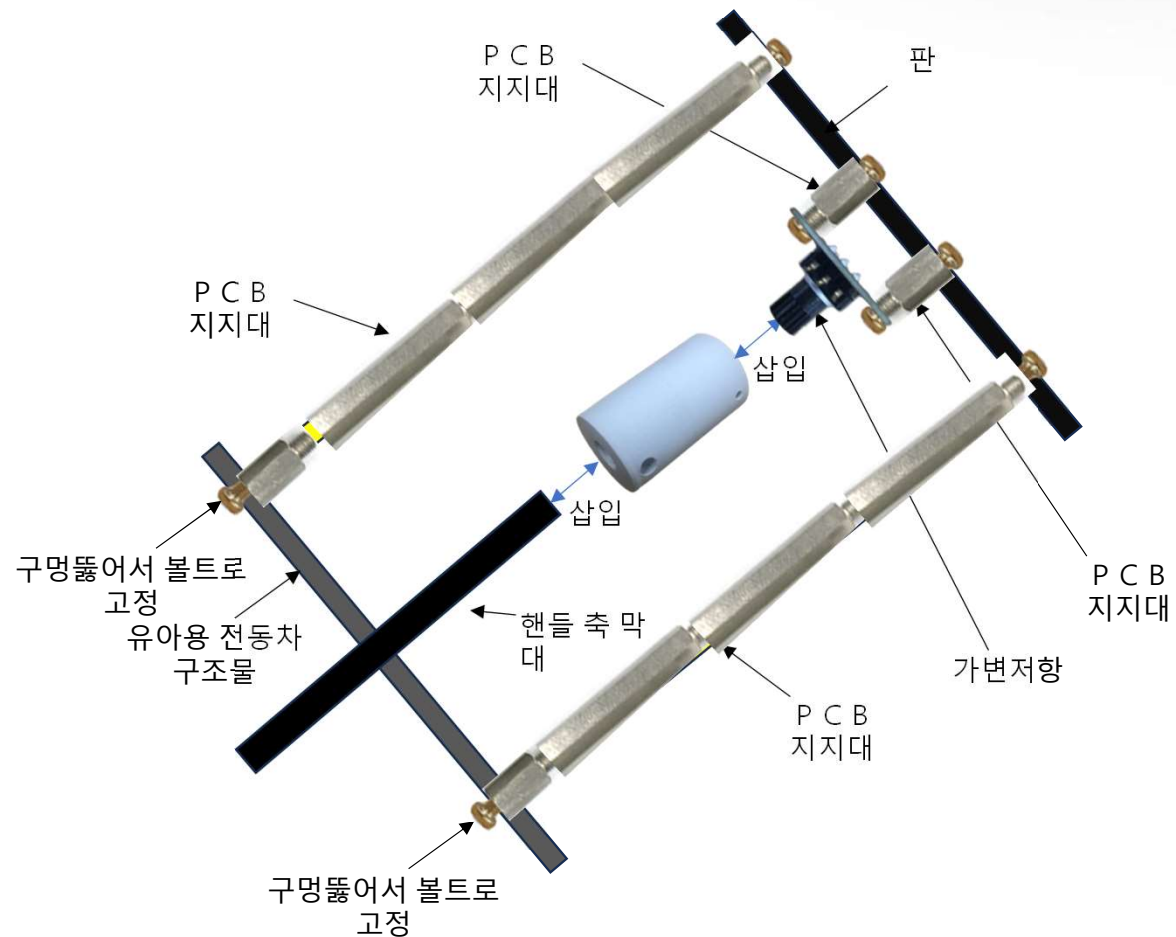
3 뒷바퀴 조립



위의 그림처럼 뒷바퀴 축에 [바퀴 - 와셔 - 너트 - 휠캡]의 순서로 알맞게 조립합니다.

하드웨어 – 핸들 축에 가변저항 고정

■ 요약도



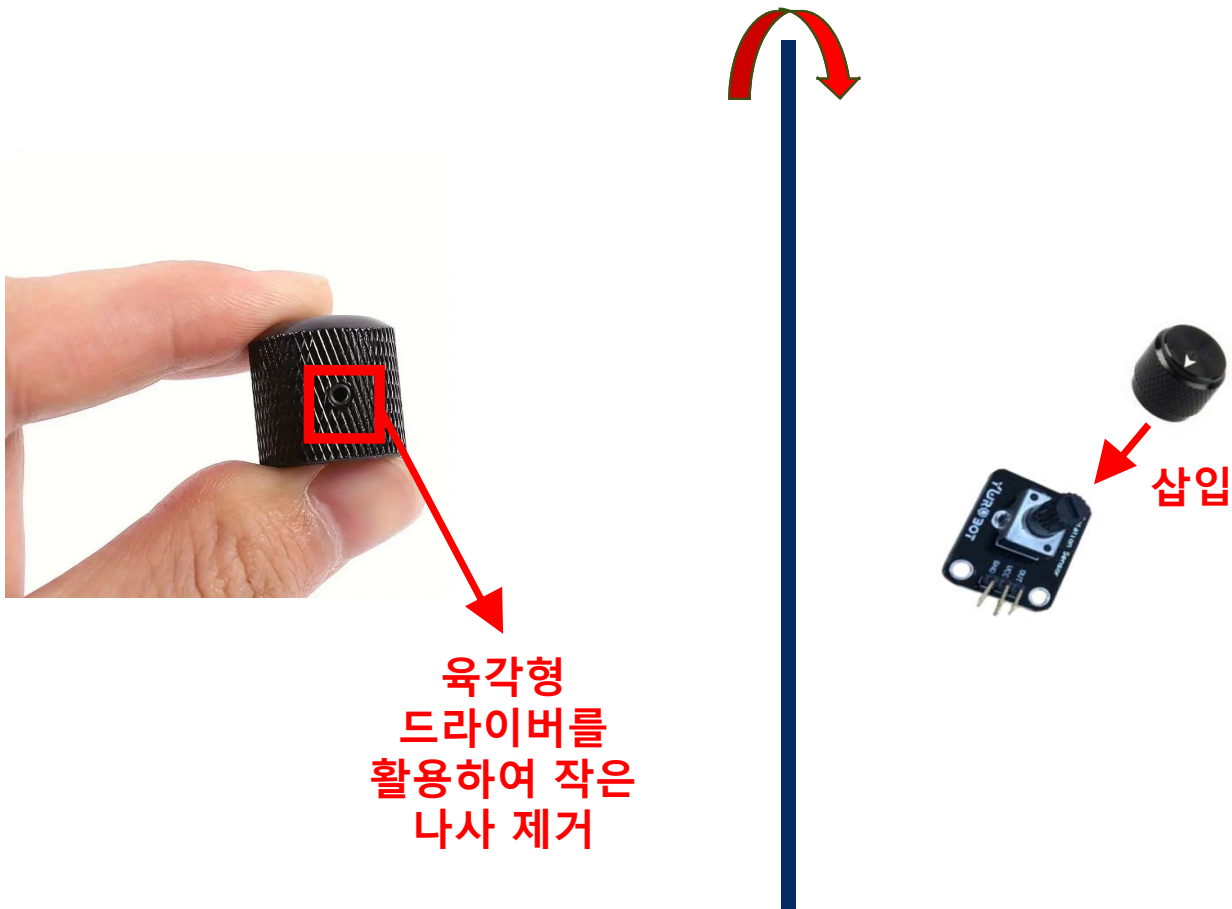
하드웨어 – 핸들 축에 가변저항 고정

■ 기존 마운트 제거



하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 + 메탈캡 결합



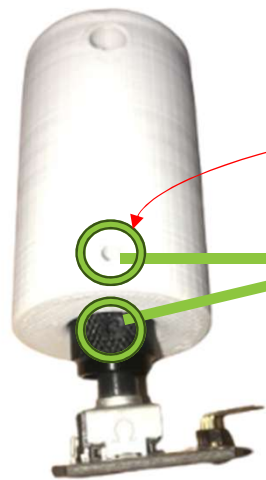
하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 + 메탈캡 + 3D프린팅 마운트 결합



하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 + 메탈캡 + 3D프린팅 마운트 결합



메탈캡 구멍과
마운트 구멍을
일치시키기

육각나사를 구멍에
삽입하여 고정하기

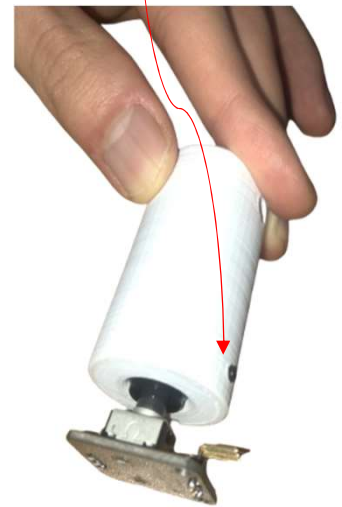


메탈캡 구멍과 마운트 구멍을
제대로 일치시키지 않은채로
무리하게 육각나사를 돌리면

메탈캡이 부서지므로, 주의할 것.



육각형 모양 드라이버를
활용하여 육각나사 삽입중

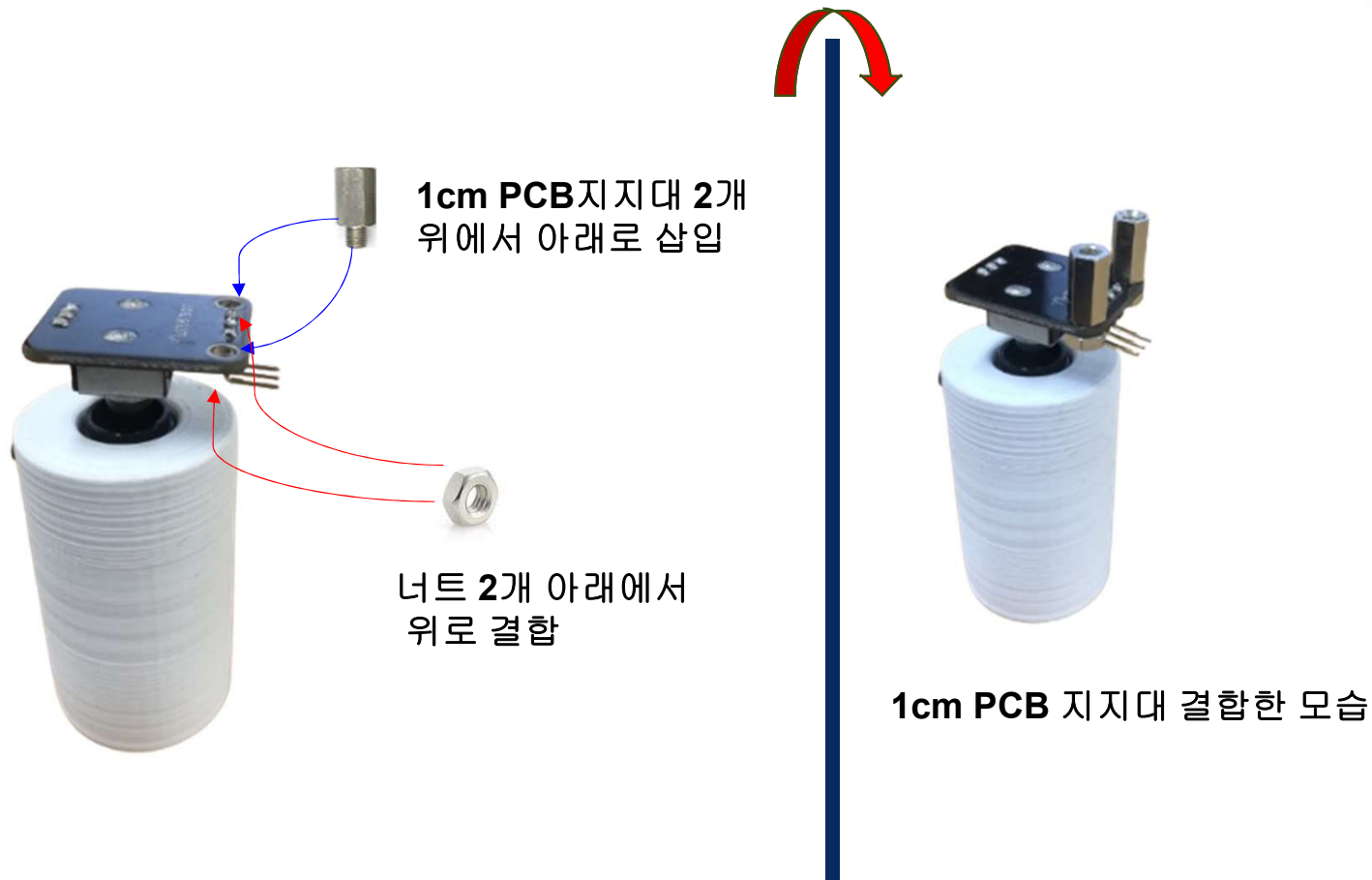


삽입 완료된 모습

육각나사가 아주 약간
튀어나온 정도면
충분히 잘 고정 됨

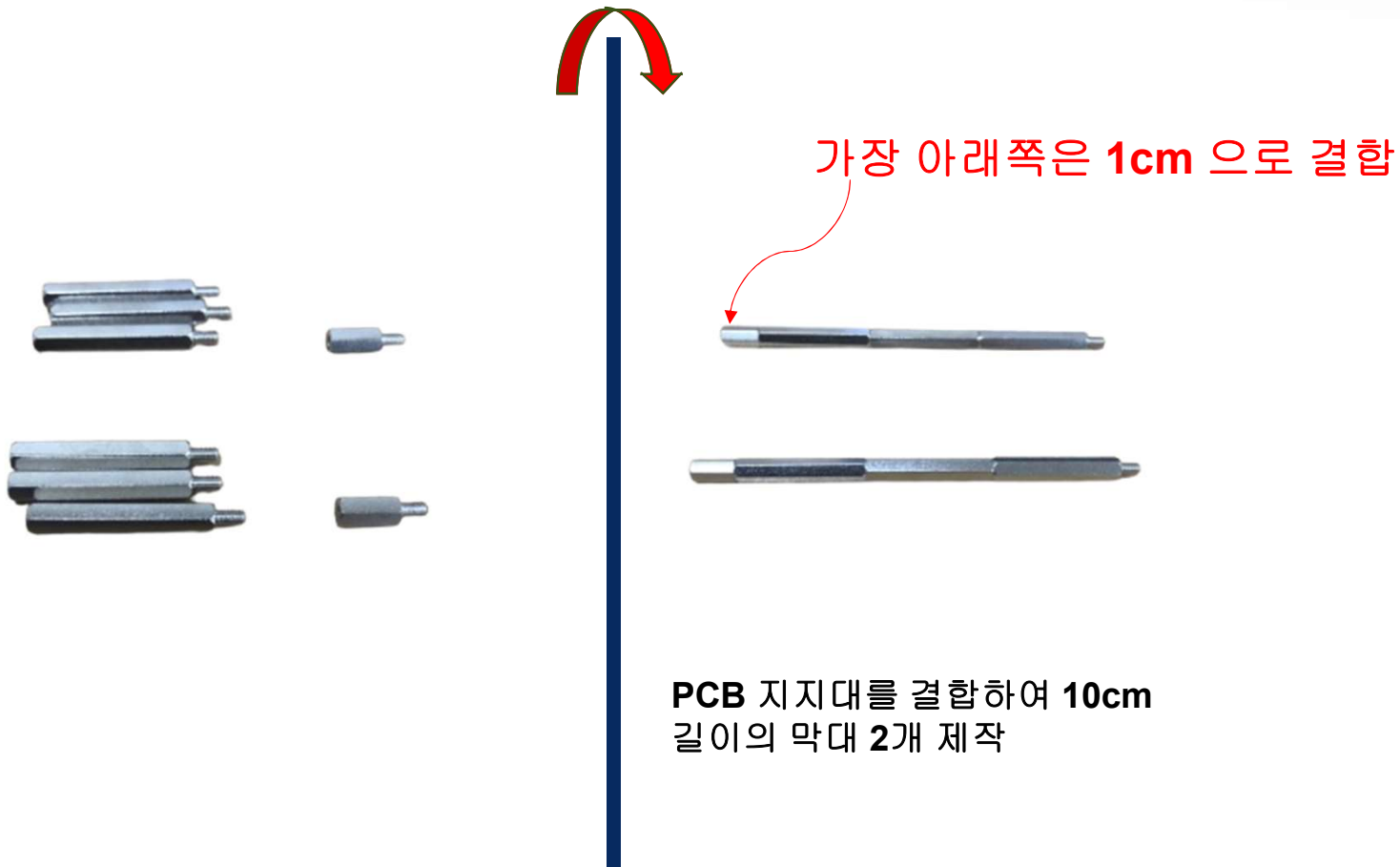
하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 + 메탈캡 + 3D프린팅 마운트 결합



하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 차체에 결합



하드웨어 – 핸들 축에 가변저항 고정

- 가변저항 차체에 결합
 - 차량 앞부분 구조물 빼기

십자드라이버 활용하기
뺄 나사는 잃어버리지 말것



하드웨어 – 핸들 축에 가변저항 고정

- 이후 과정을 위해 미리 확인 할 사항
 - 조향 모터 연결 선 색깔 확인하기

<선 색깔 확인>

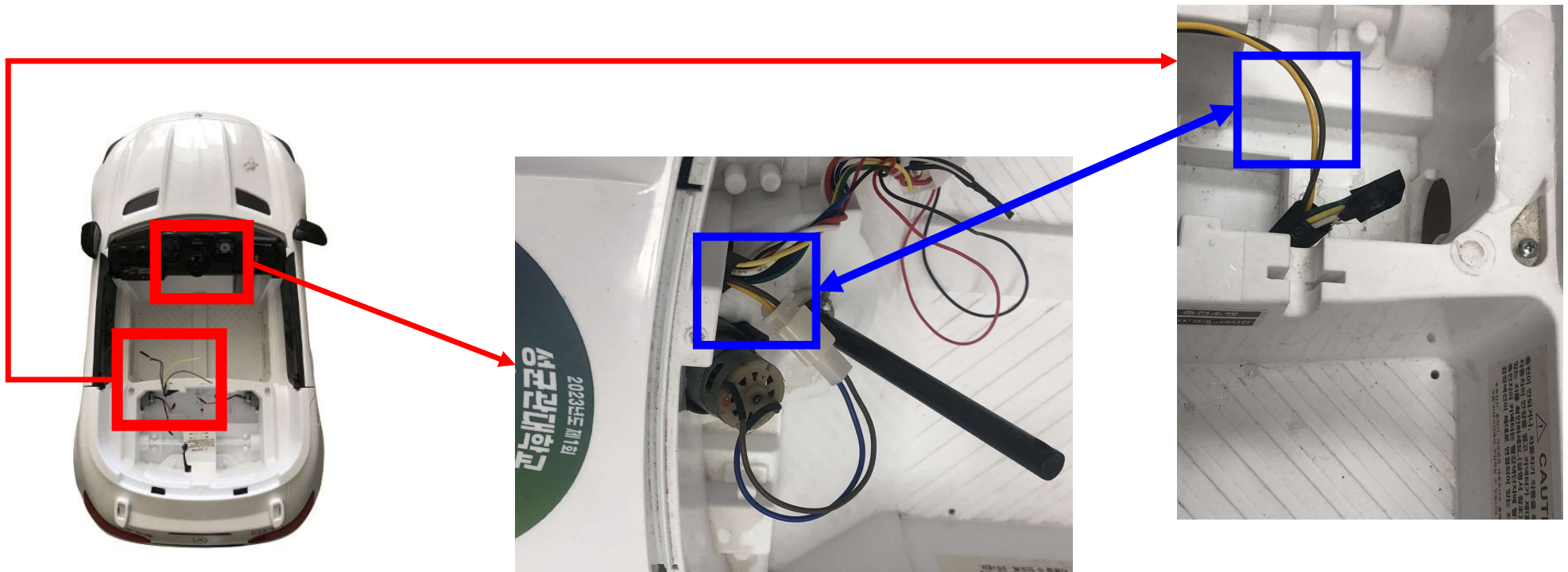
이 선은 차량 뒷편까지 연결되어 있음



하드웨어 – 핸들 축에 가변저항 고정

■ 이후 과정을 위해 미리 확인 할 사항

- 앞쪽의 조향 모터와 연결된 선은 차량 뒷부분까지 연결 되어 있음
- 차량 뒷부분에서 어떤 선이 조향 모터와 연결된 선인지 확인하기



하드웨어 – 핸들 축에 가변저항 고정

■ 가변저항 차체에 결합



구조물에 구멍이 뚫려있는 경우



구멍에 볼트 삽입

유아용 전동차
구조물

구멍에 볼트 삽입



구조물에 구멍이 없거나 구멍이
너무 커서 헐거운 경우

조교에게 요청하여 구멍 뚫기

하드웨어 – 핸들 축에 가변저항 고정

- 가변저항 차체에 결합
 - PCB 지지대 고정 후 재결합

십자드라이버 활용하기



하드웨어 – 핸들 축에 가변저항 고정

- 가변저항 차체에 결합
 - 마운트를 차량 핸들 축에 결합



마운트 구멍과
핸들축 구멍을
일치시키기

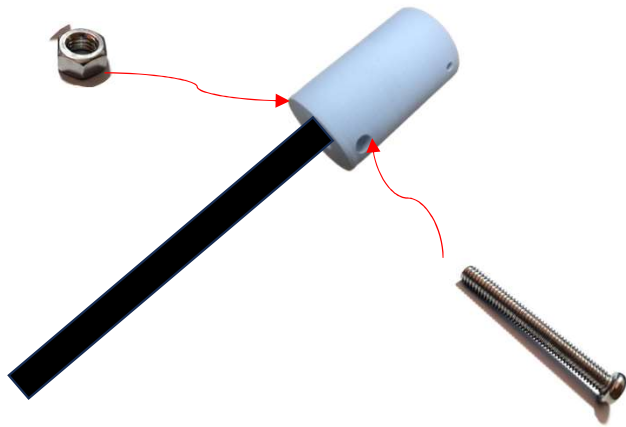


좌우로 자유롭게 회전이
가능하게 세팅



하드웨어 – 핸들 축에 가변저항 고정

- 가변저항 차체에 결합
 - 윗판을 얹어서 볼트와 너트로 고정하기



커다란 볼트와 너트로 마운트를 단단하게 고정



하드웨어 – 핸들 축에 가변저항 고정

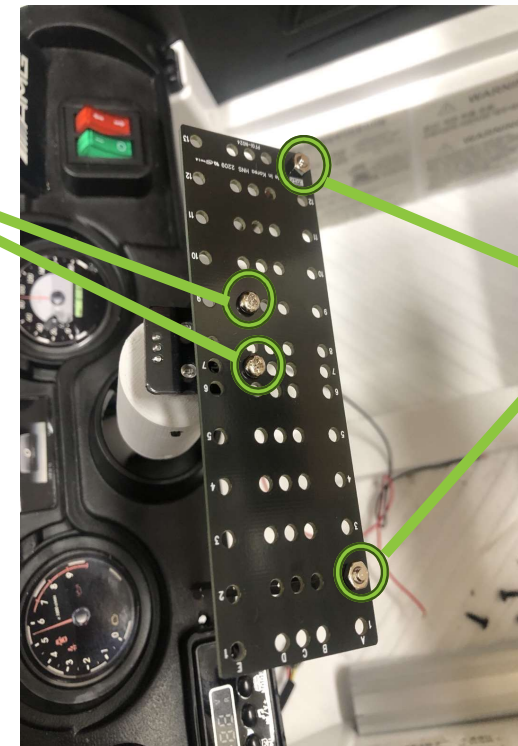
- 가변저항 차체에 결합
 - 윗판을 얹어서 볼트와 너트로 고정하기



윗판 결합

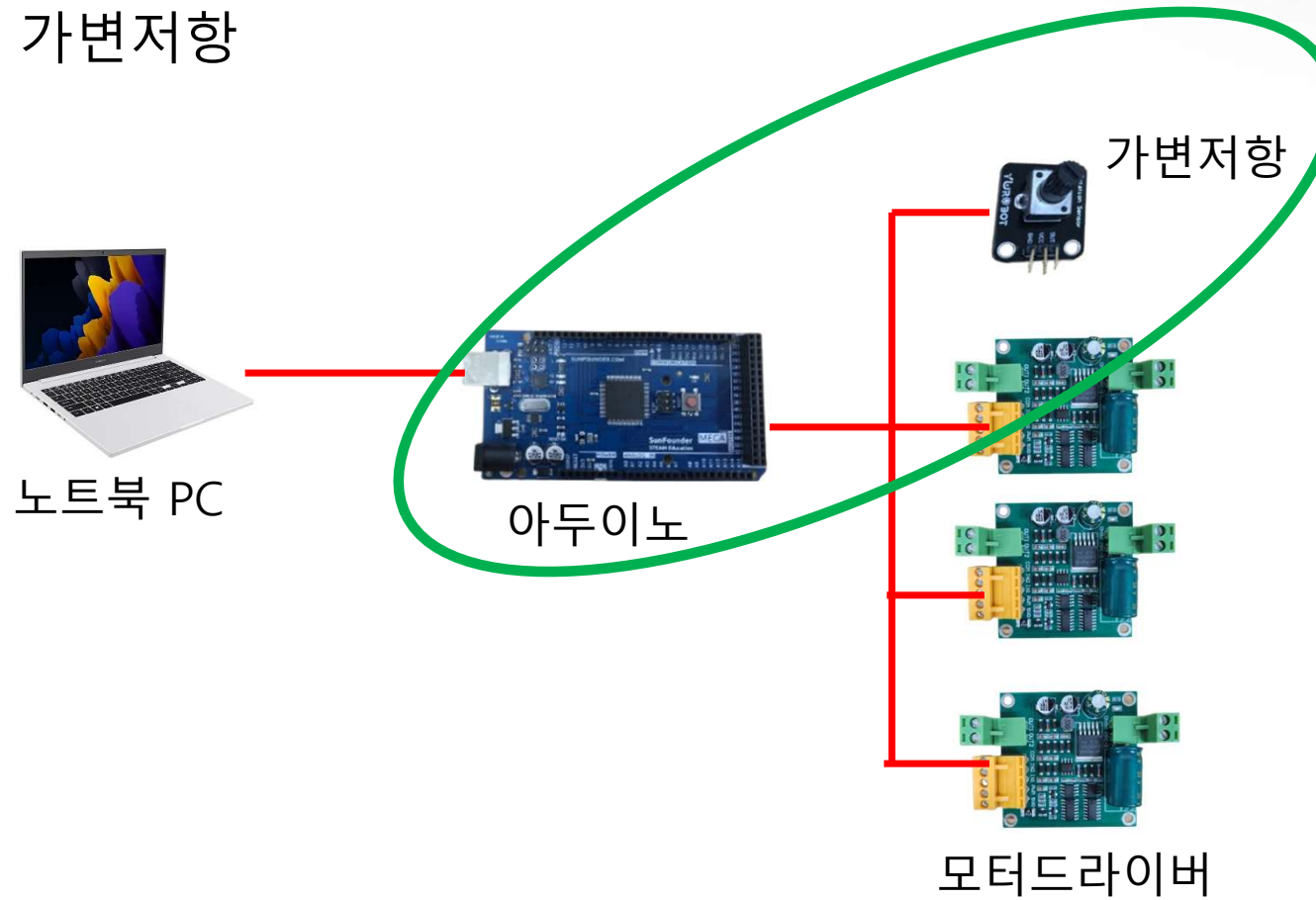
볼트로 결합

너트로 결합



하드웨어 – 가변저항 연결

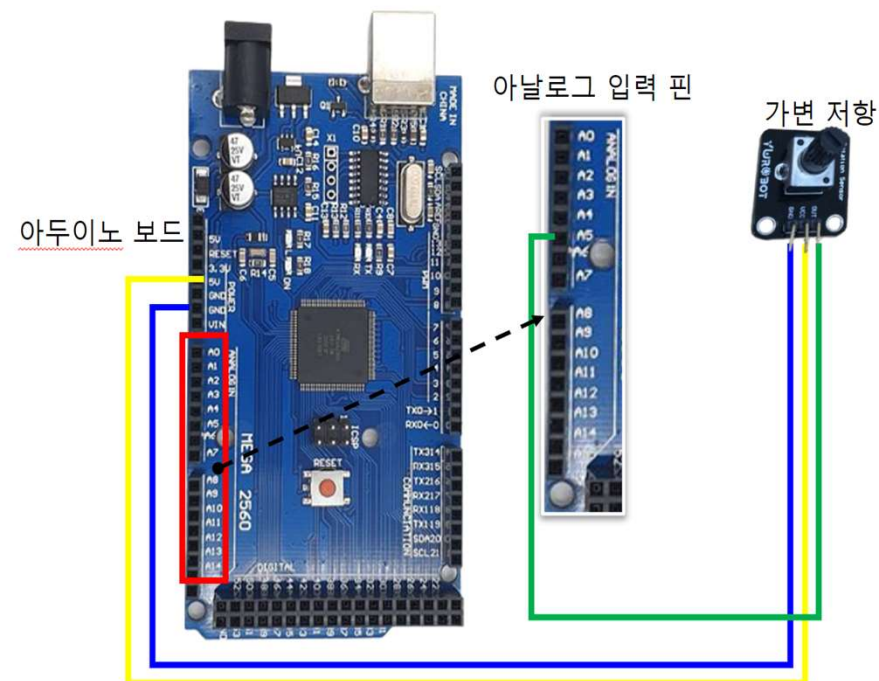
■ 아두이노 - 가변저항



하드웨어 – 가변저항 연결

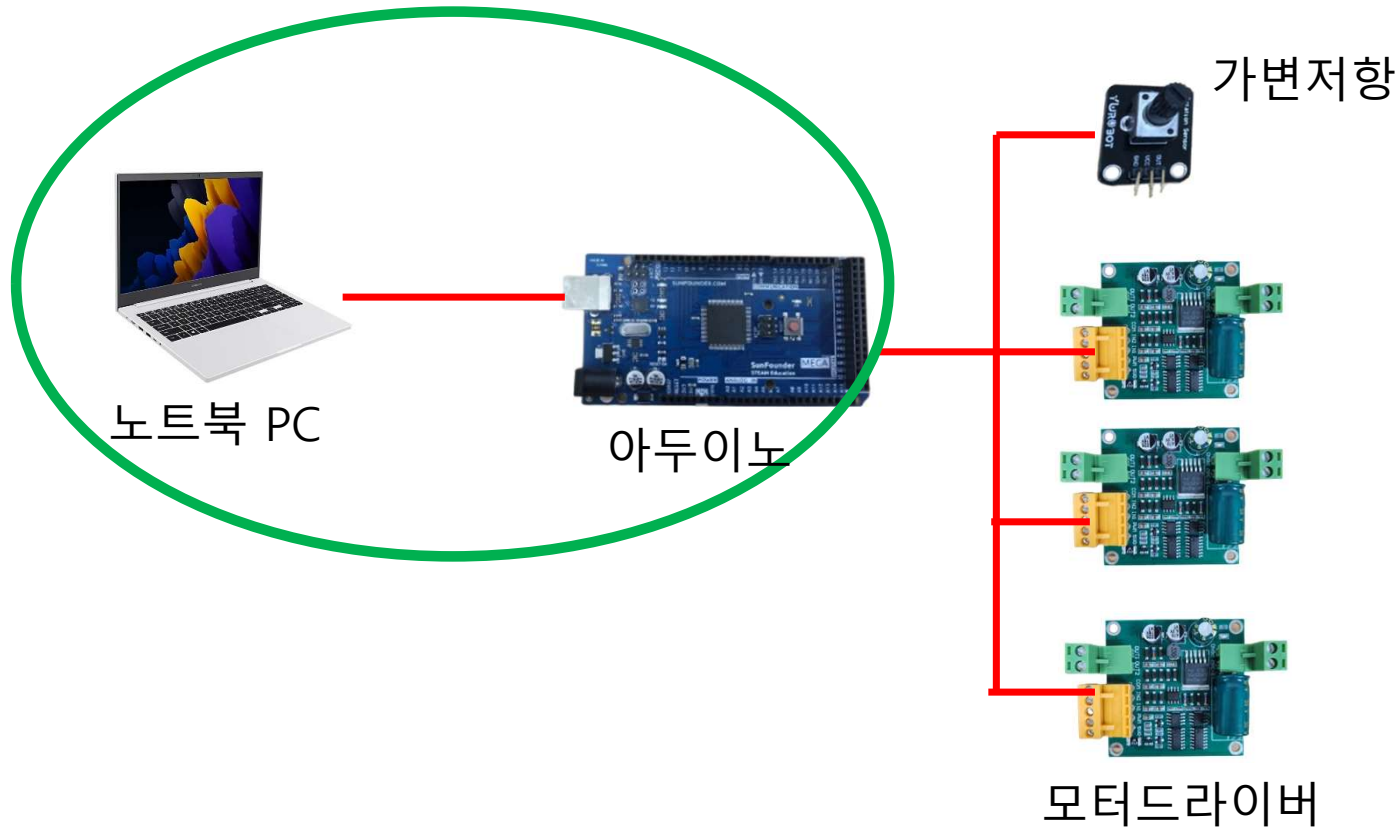
■ 아두이노 – 가변저항

- 가변저항 OUT - 아두이노 **A2핀**에 연결
- 가변저항 VCC - 아두이노 5V 핀에 연결
- 가변저항 GND - 아두이노 GND 핀



하드웨어 – 가변저항 테스트

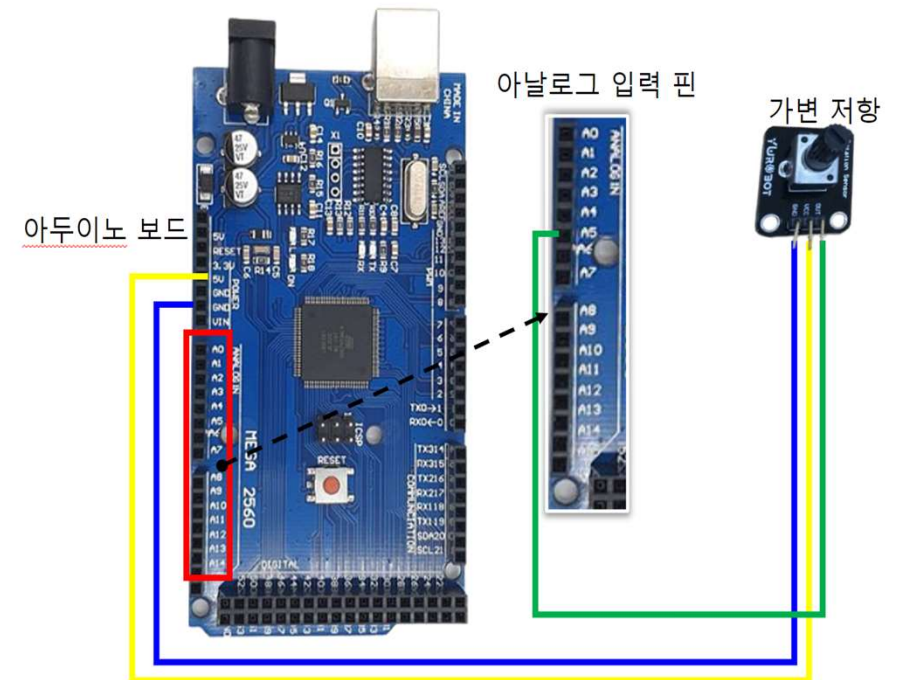
■ 노트북 PC – 아두이노



하드웨어 – 가변저항 테스트

■ PC – 아두이노

- 파란색 아두이노 케이블을 사용하여 PC와 아두이노 연결



하드웨어 – 가변저항 테스트

■ check_variable_resistor.ino 실행

- 경로 : ~/ros2_ws/src/control/check_variable_resistor/check_variable_resistor.ino

1

```

check_variable_resistor$
int sensorPin = A2;

void setup() {

  Serial.begin(9600);

  pinMode(sensorPin, INPUT);

}

void loop() {

  int value = analogRead(sensorPin);

  Serial.println(value);

  delay(100);

}

```



1

하드웨어 – 가변저항 테스트

■ check_variable_resistor.ino 실행

- 경로 : ~/ros2_ws/src/control/check_variable_resistor/check_variable_resistor.ino

```
check_variable_resistor$
int sensorPin = A2;

void setup() {

    Serial.begin(9600);

    pinMode(sensorPin, INPUT);

}

void loop() {

    int value = analogRead(sensorPin);

    Serial.println(value);

    delay(100);

}
```

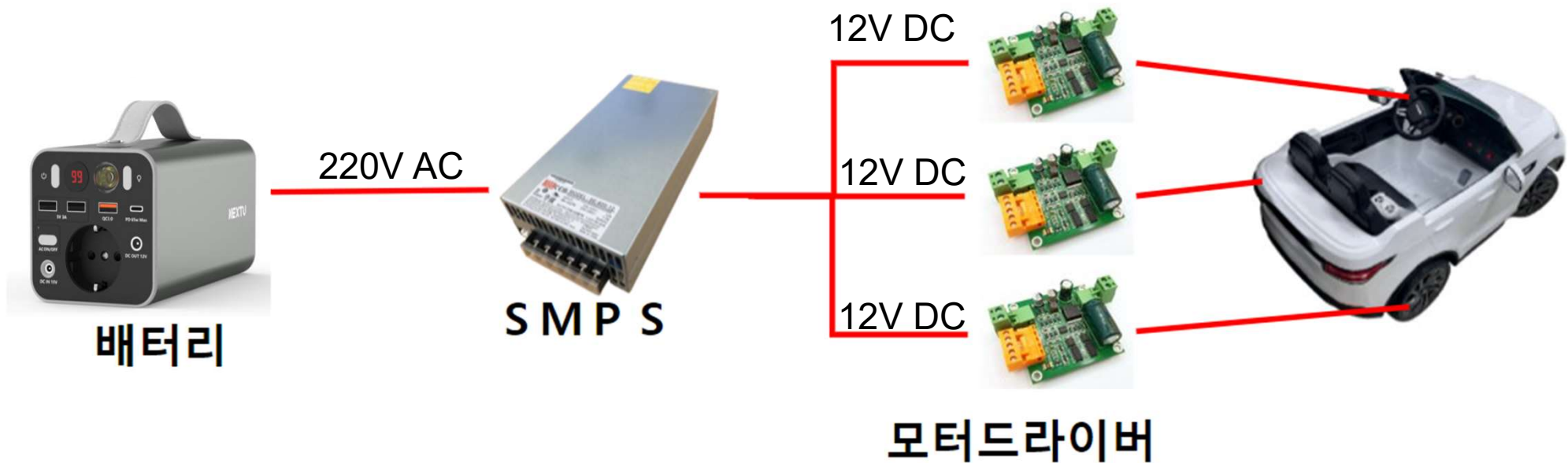


2



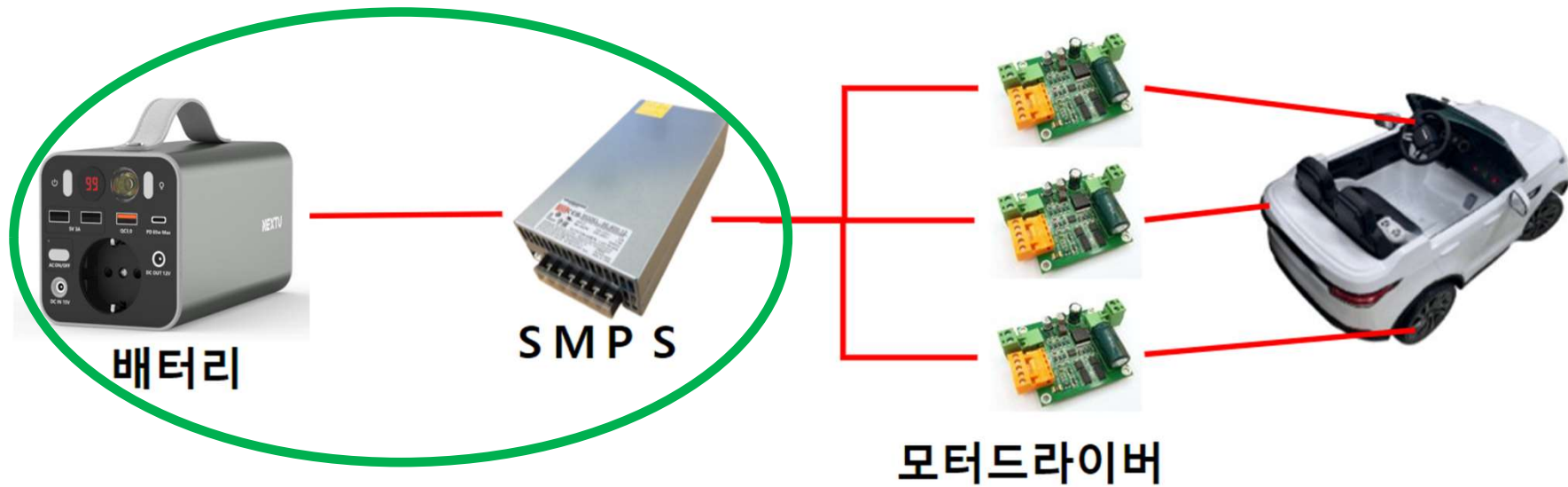
2

하드웨어 – 전원부 연결



하드웨어 – 전원부 연결

■ 배터리 - SMPS



하드웨어 – 전원부 연결

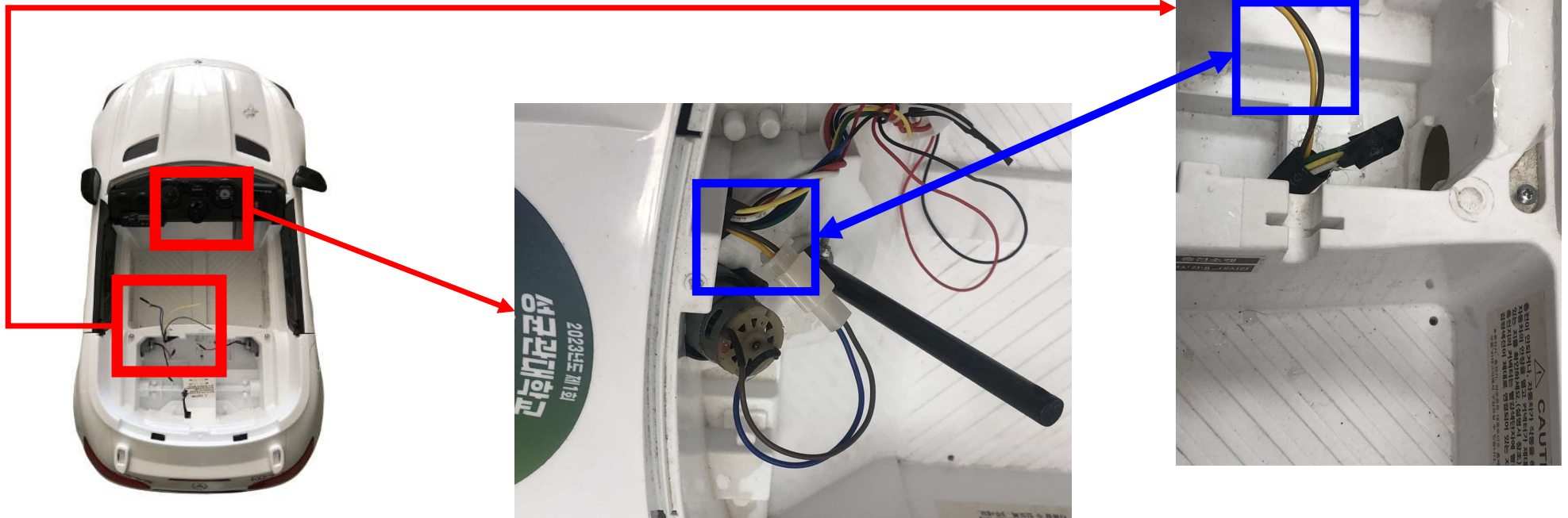
■ 배터리 - SMPS



하드웨어 – 전원부 연결

■ 모터드라이버 – 모터

- 앞쪽의 조향 모터와 연결된 선은 차량 뒷부분까지 연결 되어 있음
- 차량 뒷부분에서 어떤 선이 조향 모터와 연결된 선인지 확인하기
- 이전에 가변저항 고정하는 도중에 진행하였음



하드웨어 – 전원부 연결

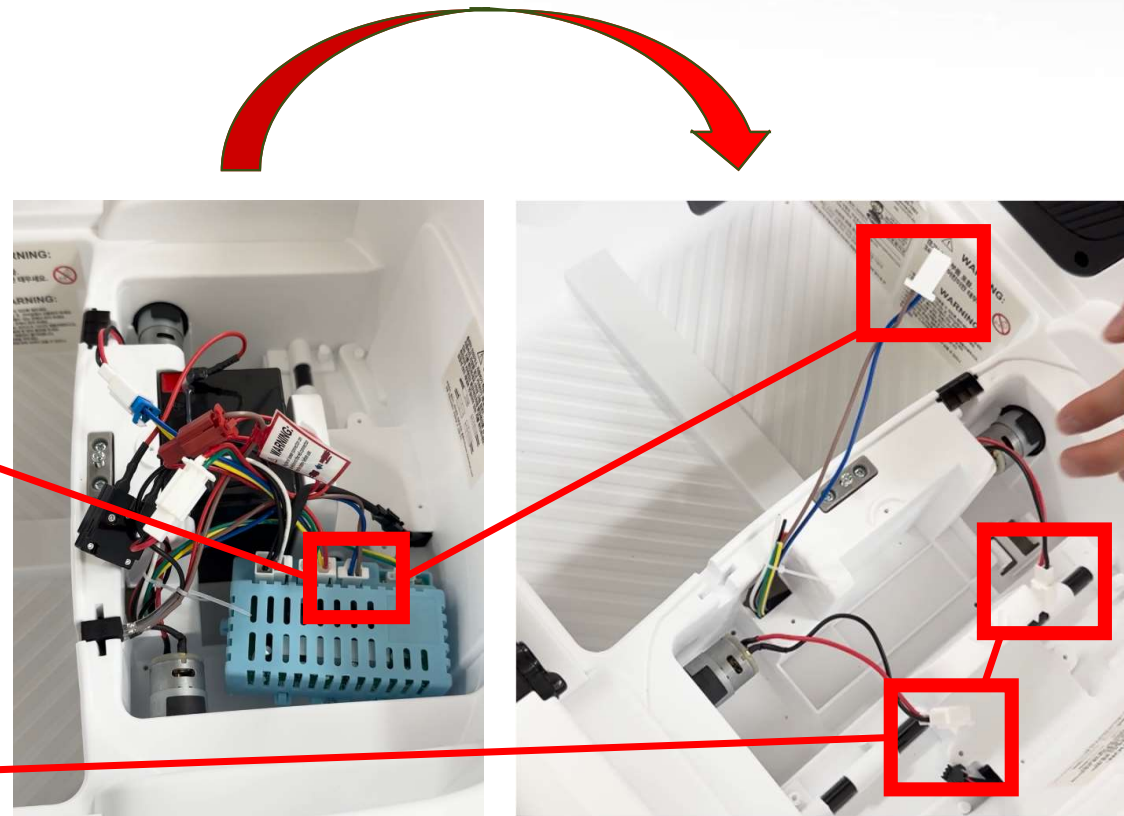
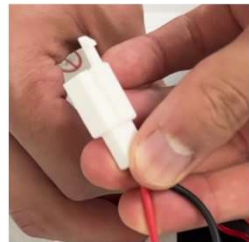
■ 모터드라이버 – 모터

- 뒷바퀴 모터에 연결된 선을 남기기
- 조향 모터와 연결된 선을 남기기
- 나머지는 제거

조향모터와 연결된 선
커넥터 생김새

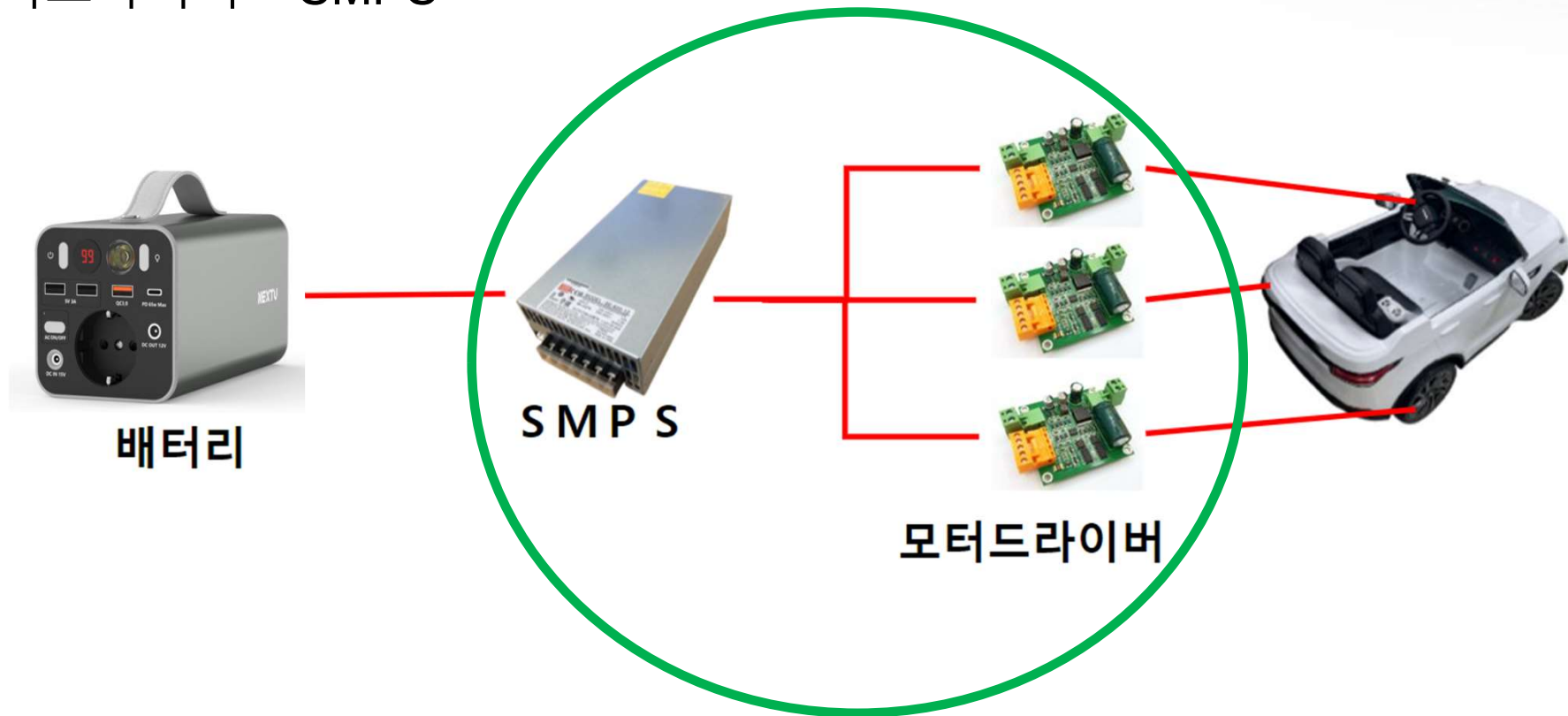


뒷바퀴 모터와 연결된 선
커넥터 생김새



하드웨어 – 전원부 연결

■ 모터드라이버 – SMPS



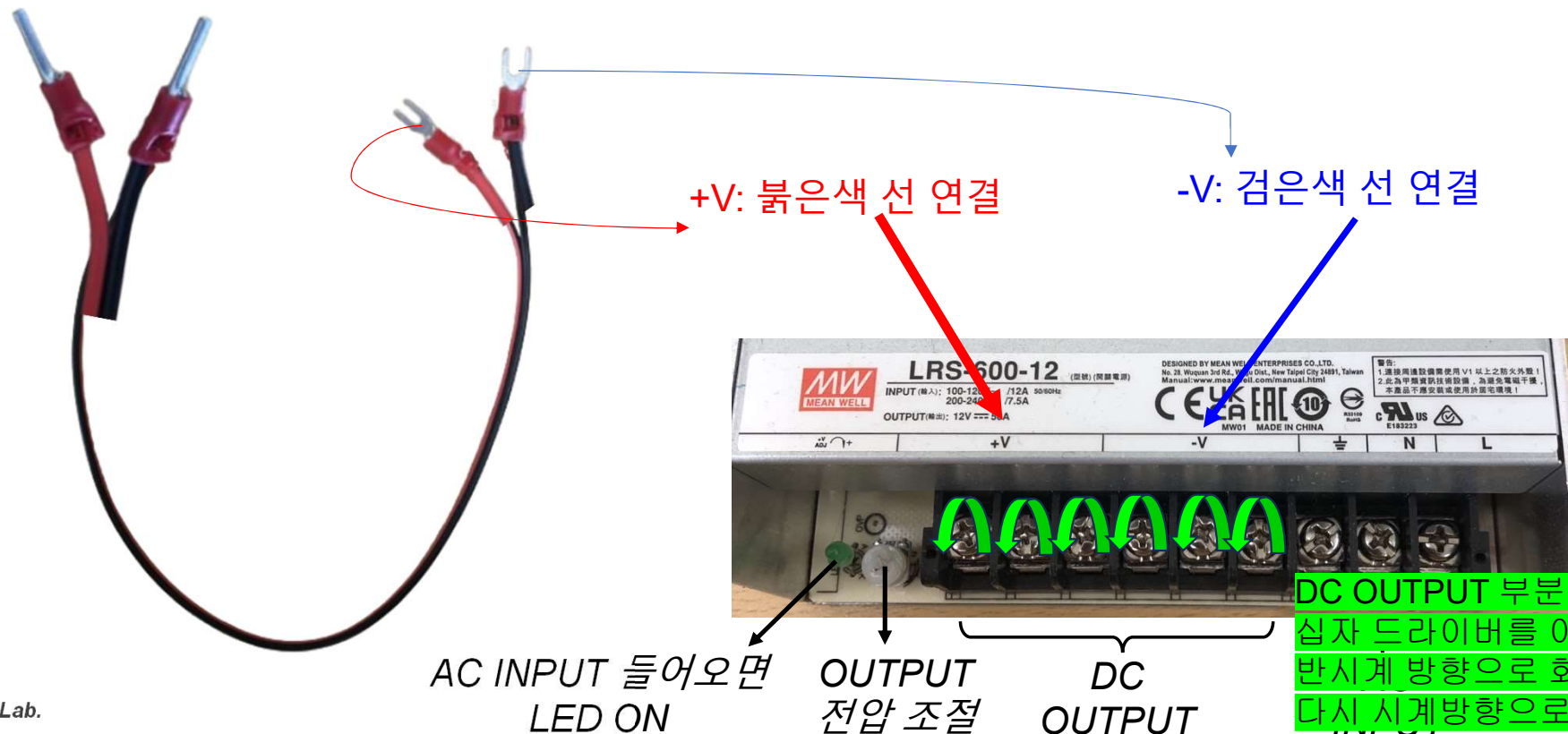
하드웨어 – 전원부 연결

■ SMPS – 모터드라이버



하드웨어 – 전원부 연결

■ SMPS – 모터드라이버

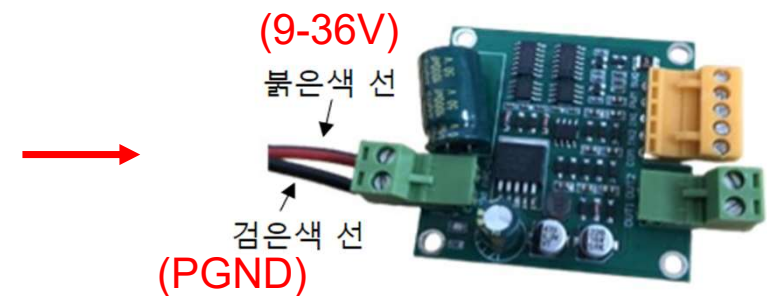
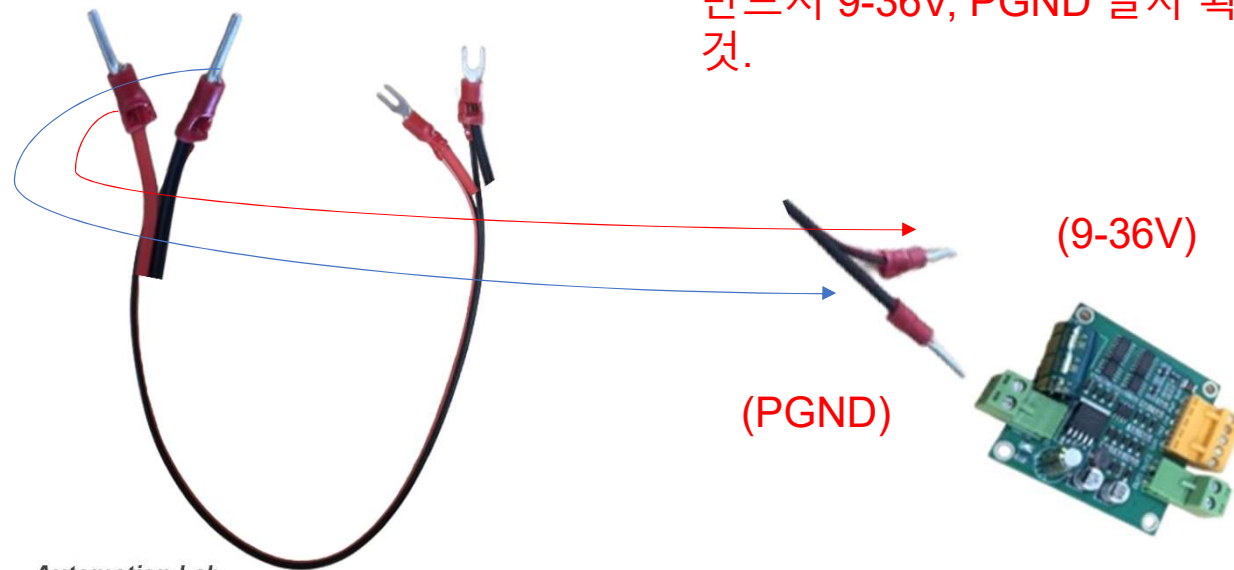
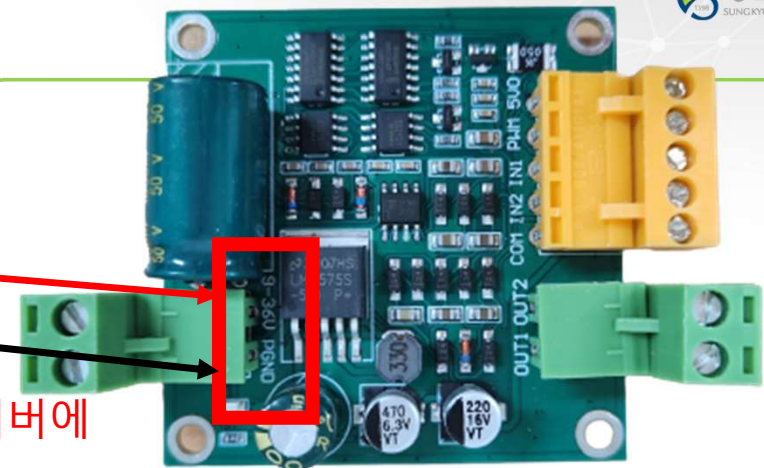


하드웨어 – 전원부 연결

■ SMPS – 모터드라이버

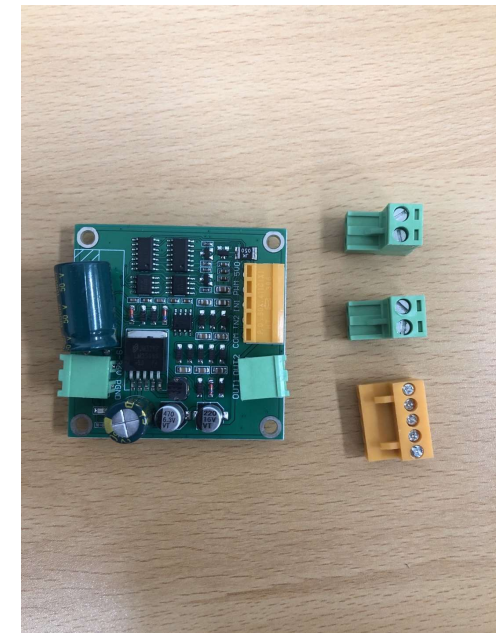
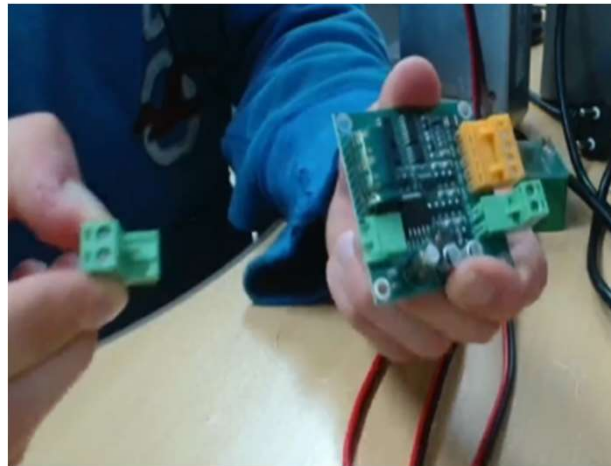
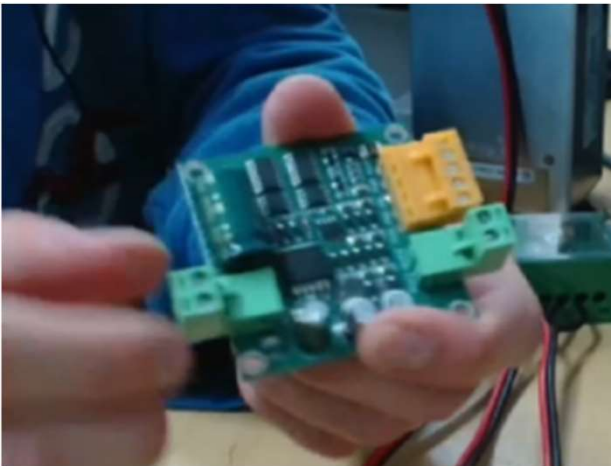
- 붉은선: 9-36V
- 검은선: PGND

색깔별 위치 바뀌면 모터드라이버에서 스파크 발생하며 고장 발생!
반드시 9-36V, PGND 글자 확인할 것.



하드웨어 – 전원부 연결

- 모터드라이버 – 모터
 - 커넥터 분리



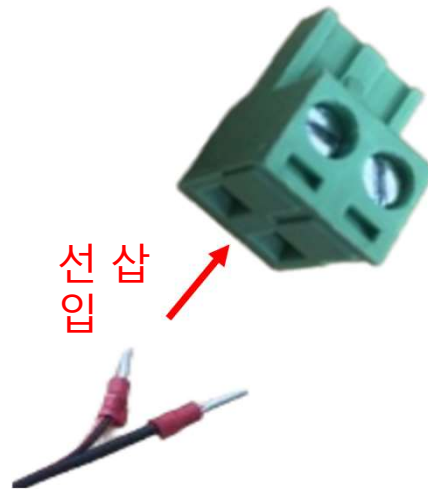
하드웨어 – 전원부 연결

■ SMPS – 모터드라이버

일자 드라이버를 이
용하여 반시계 방
향으로 회전



선 삽
입

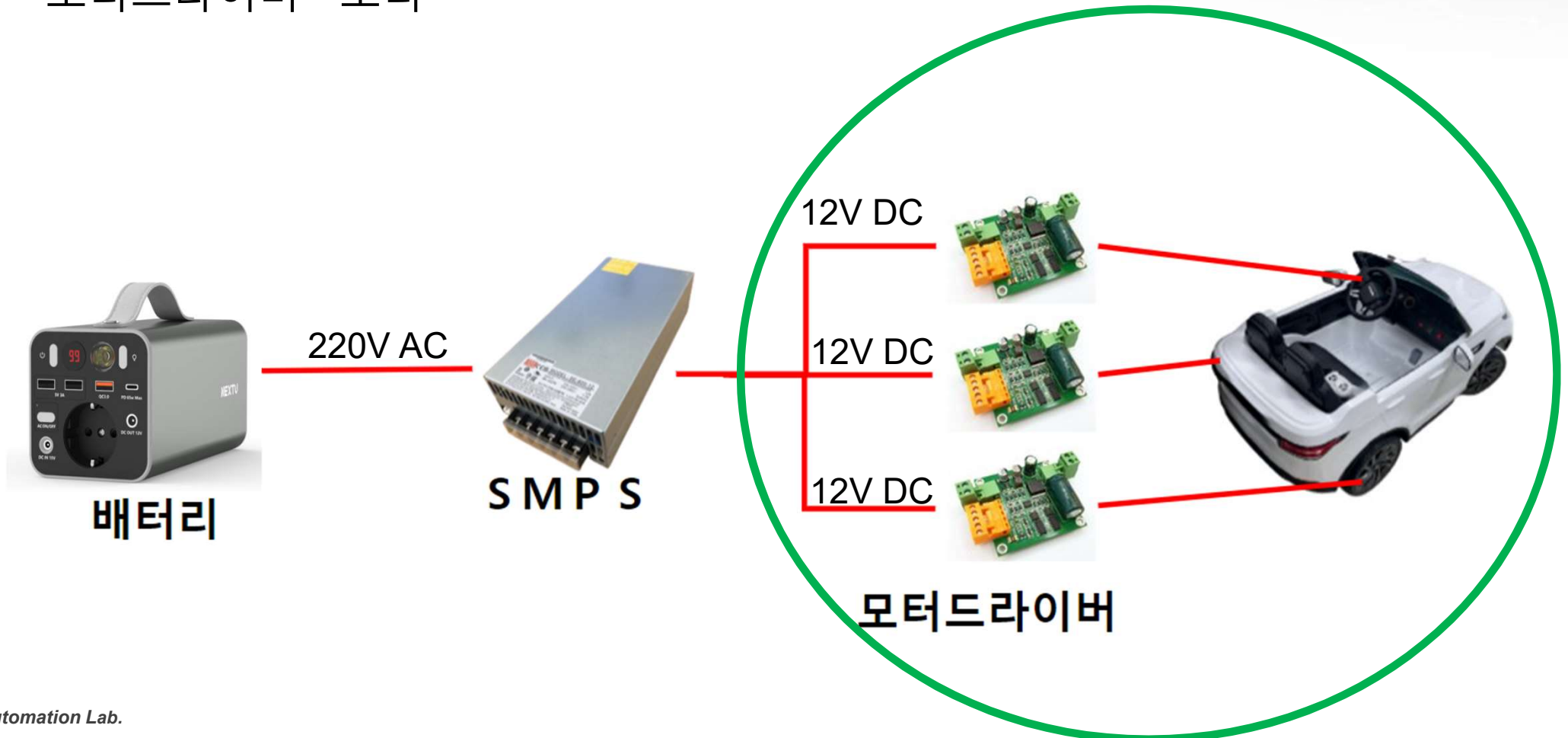


일자 드라이버를 이용
하여 시계 방향으로
회전하여 고정



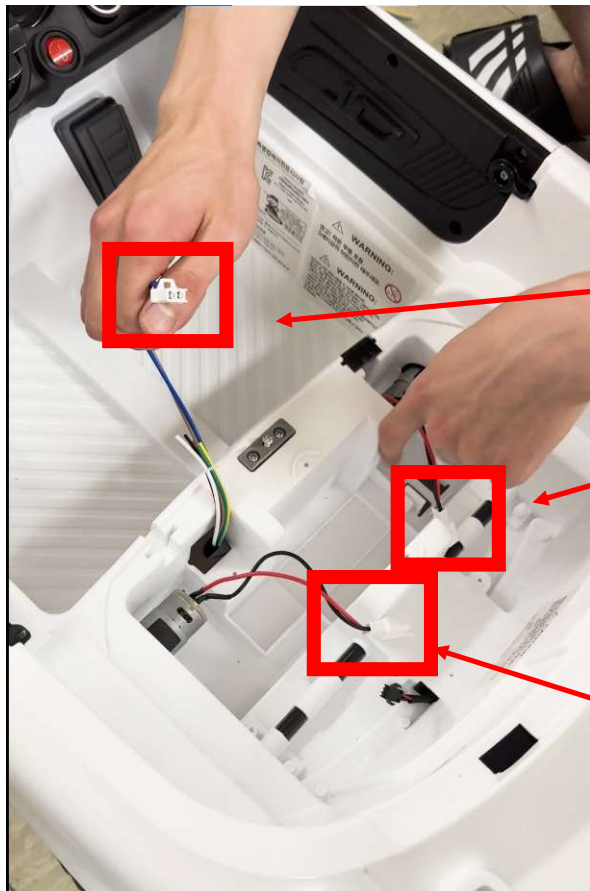
하드웨어 – 전원부 연결

■ 모터드라이버 - 모터



하드웨어 – 전원부 연결

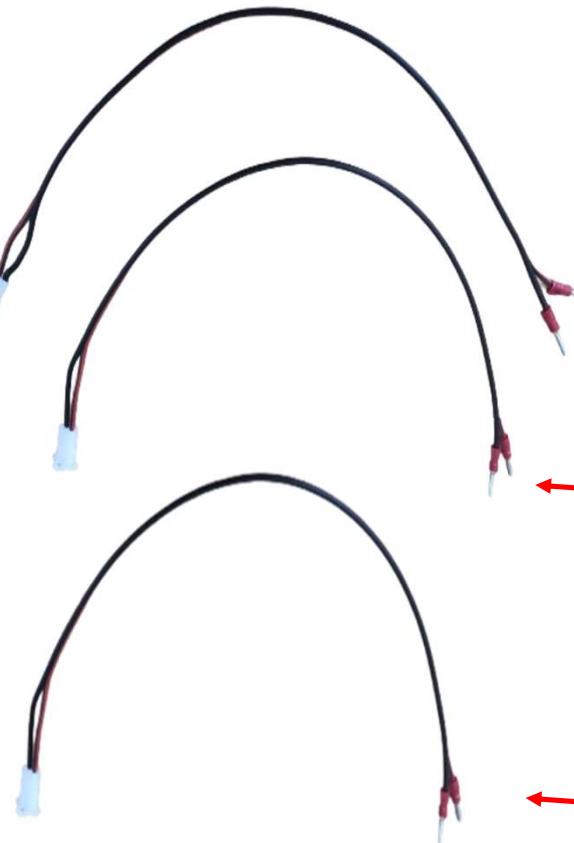
■ 모터드라이버 – 모터



연결

연결

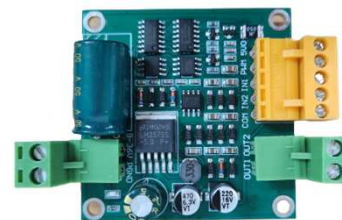
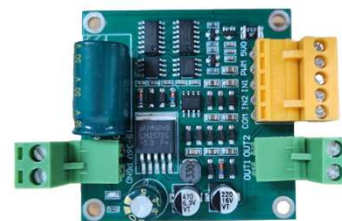
연결



연결

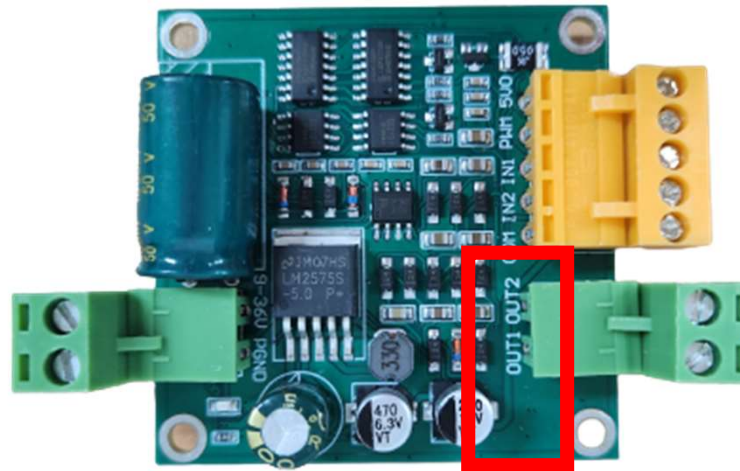
연결

연결



하드웨어 – 전원부 연결

■ 모터드라이버 - 모터

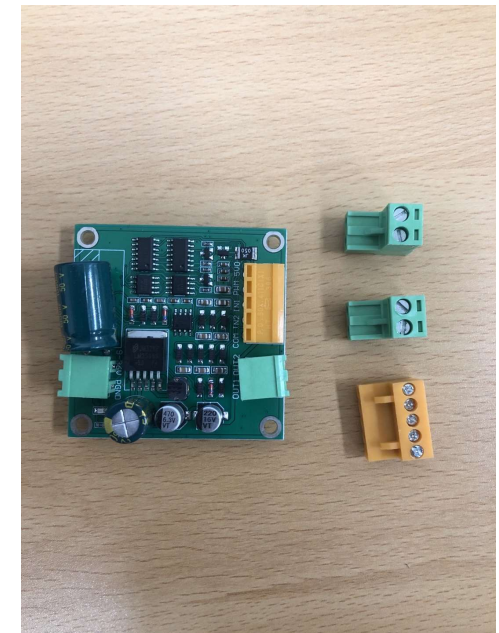
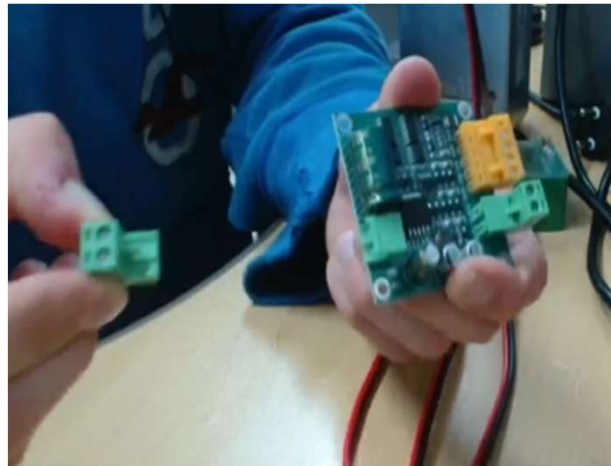
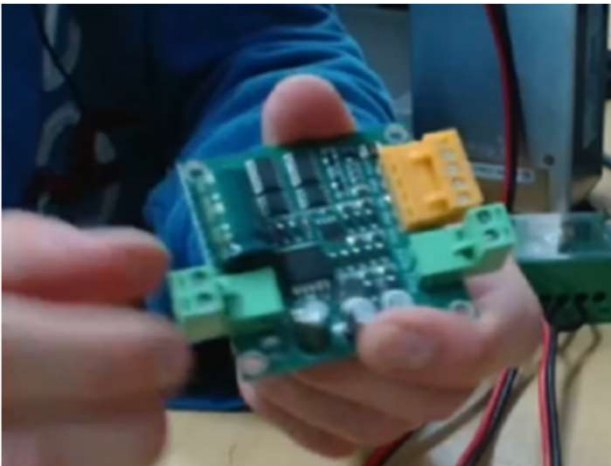


모터는 OUT1, OUT2
에 연결 할 것.

PGND, 9-36V에 연결
하면 안됨.

하드웨어 – 전원부 연결

- 모터드라이버 – 모터
 - 커넥터 분리



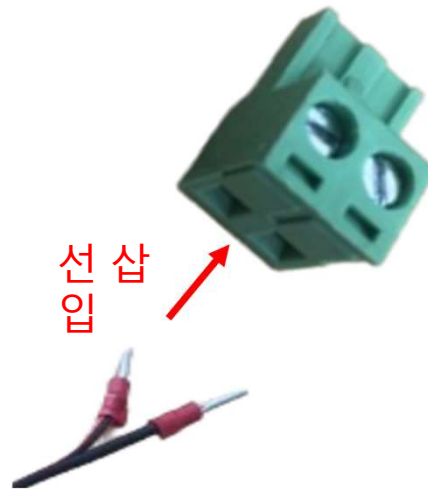
하드웨어 – 전원부 연결

■ 모터드라이버 - 모터

일자 드라이버를 이
용하여 반시계 방
향으로 회전



선 삽
입



일자 드라이버를 이용
하여 시계 방향으로
회전하여 고정



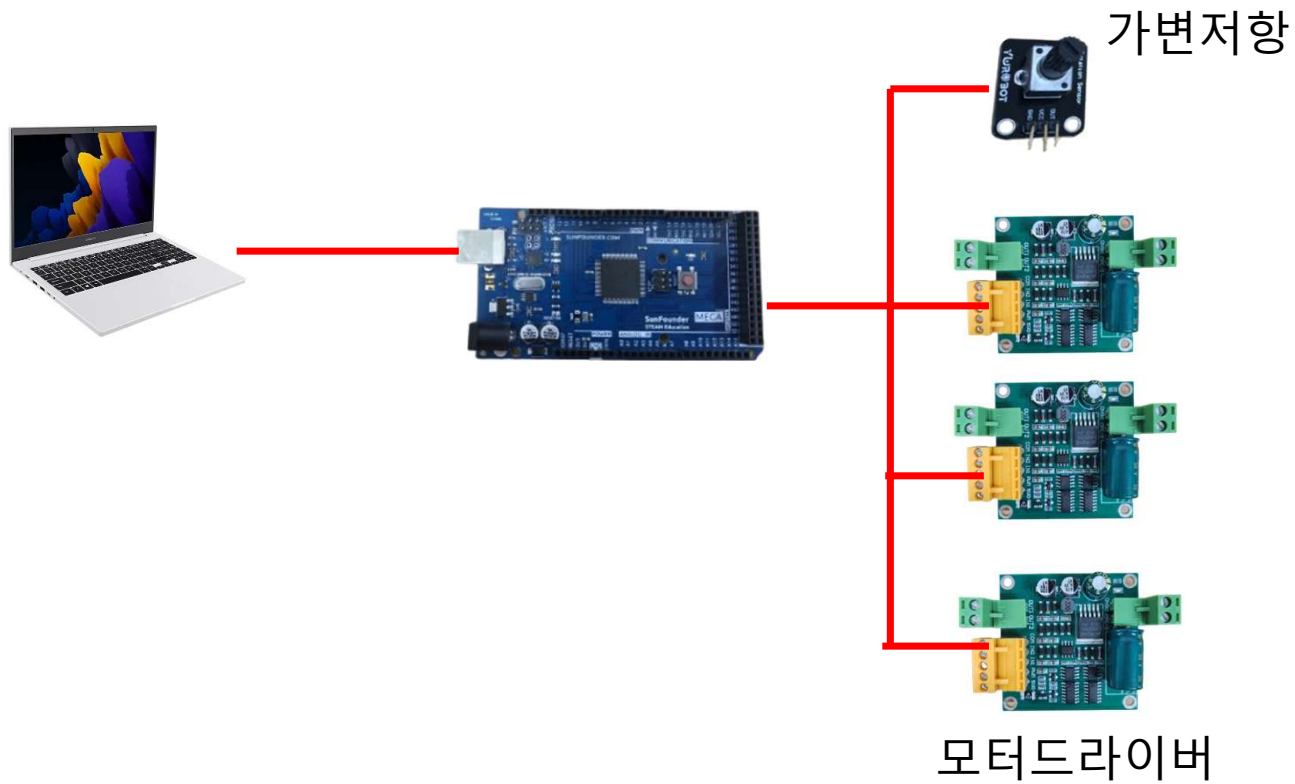
하드웨어 – 전원부 연결

전원부 연결이 완료된 팀

배터리를 켜서 모터드라이버에
불이 켜지는지 확인

하드웨어 – 제어부 연결

■ 제어부 연결



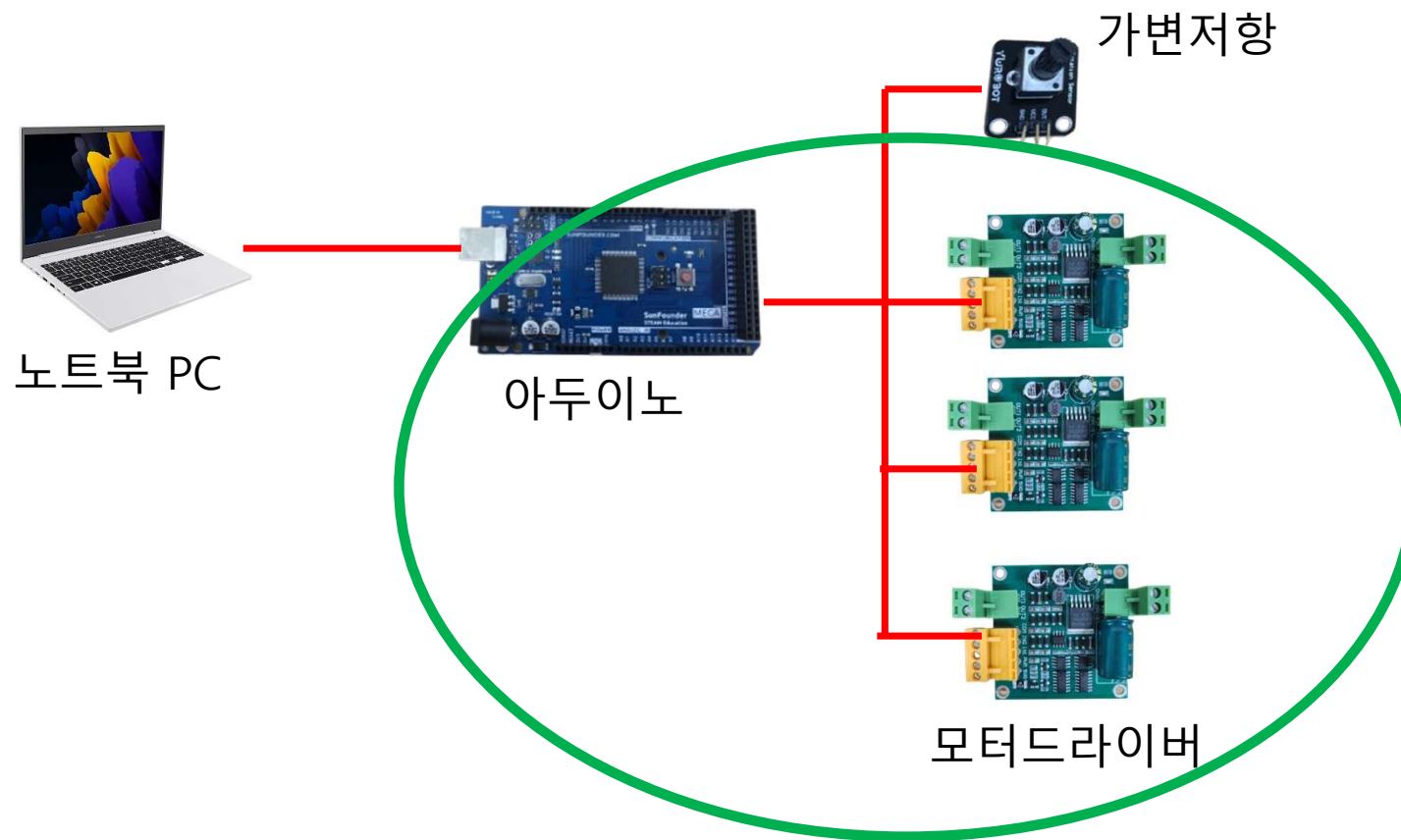
하드웨어 – 제어부 연결

- 노트북 PC – 아두이노
 - USB 시리얼 케이블을 이용하여 연결



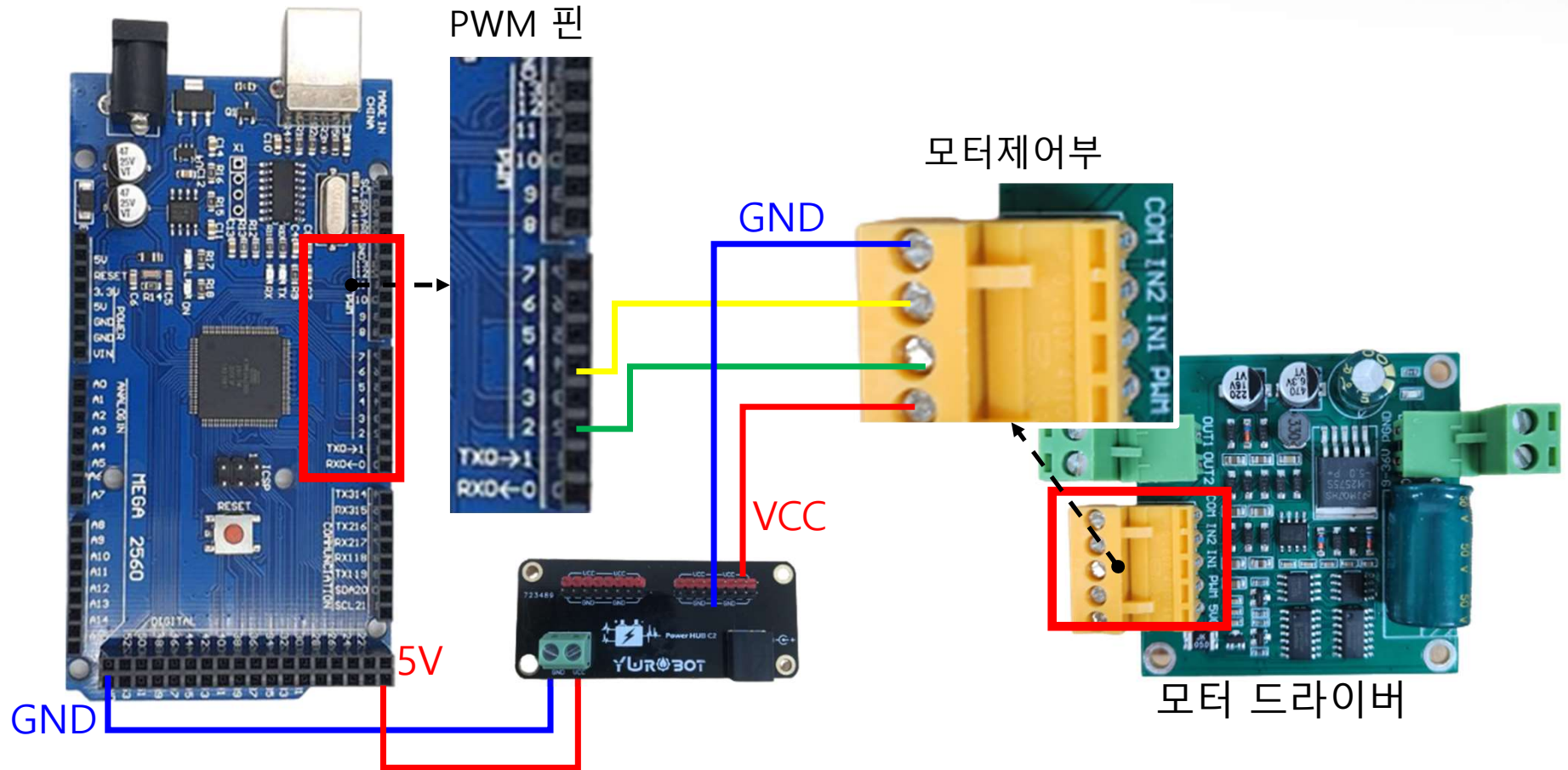
하드웨어 – 제어부 연결

■ 아두이노 - 모터드라이버



하드웨어 – 제어부 연결

■ 아두이노 – 모터드라이버



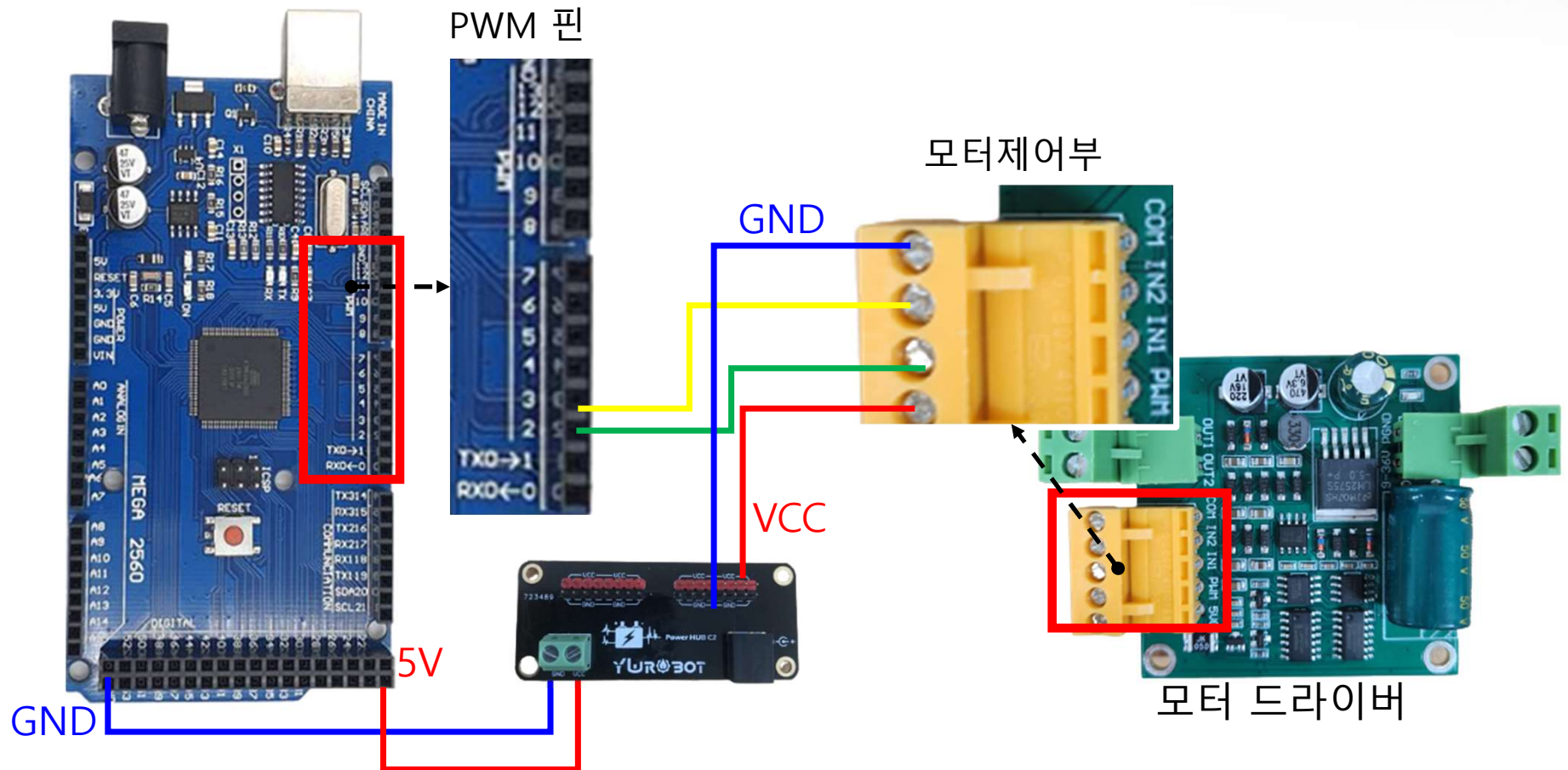
하드웨어 – 제어부 연결

■ 아두이노 – 모터드라이버

- 점퍼선을 이용하여 연결
- 조향 모터드라이버 IN1 – 아두이노 2번 핀에 연결
- 조향 모터드라이버 IN2 – 아두이노 3번 핀에 연결
- 우측 뒷바퀴 모터드라이버 IN1 – 아두이노 4번에 연결
- 우측 뒷바퀴 모터드라이버 IN2 – 아두이노 5번에 연결
- 좌측 뒷바퀴 모터드라이버 IN1 – 아두이노 6번에 연결
- 좌측 뒷바퀴 모터드라이버 IN2 – 아두이노 7번에 연결

하드웨어 – 조향 모터

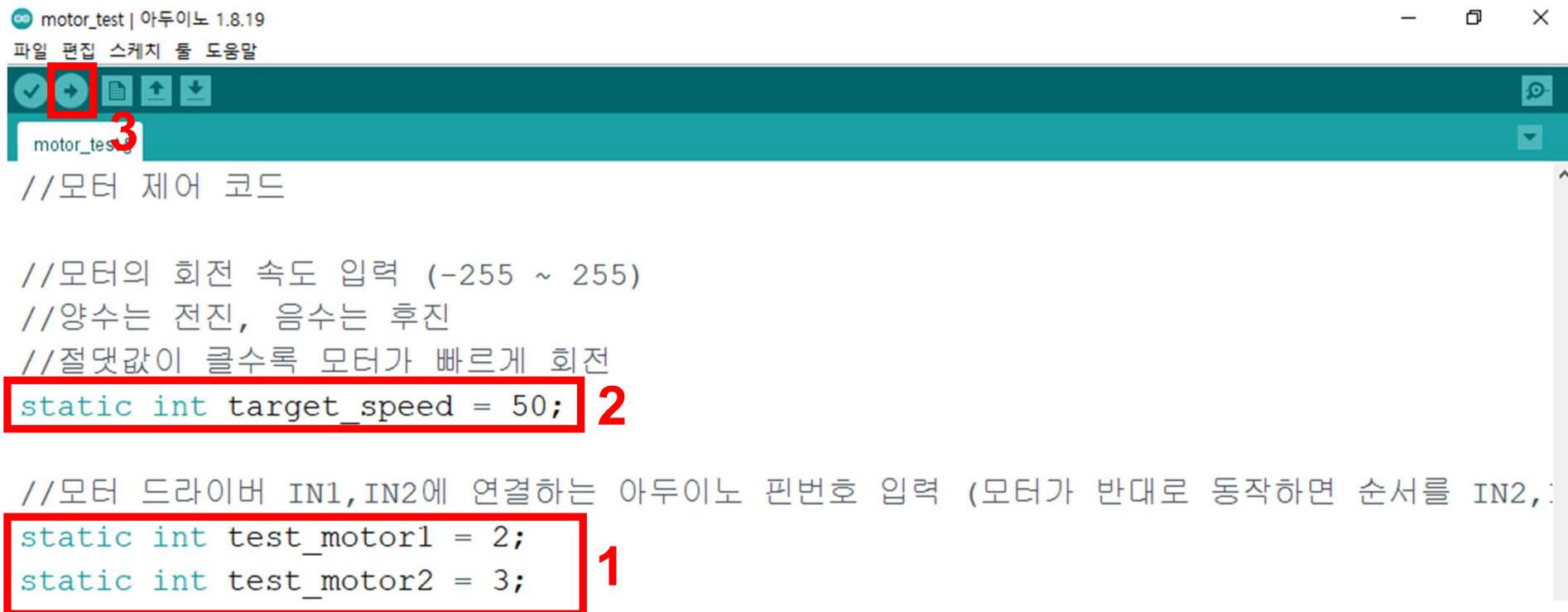
■ 아두이노 – 모터드라이버



하드웨어 – 조향 모터 테스트

■ check_variable_resistor.ino 실행

- 경로 : ~/ros2_ws/src/control/driving/driving.ino



```
motor_test | 아두이노 1.8.19
파일 편집 스케치 툴 도움말

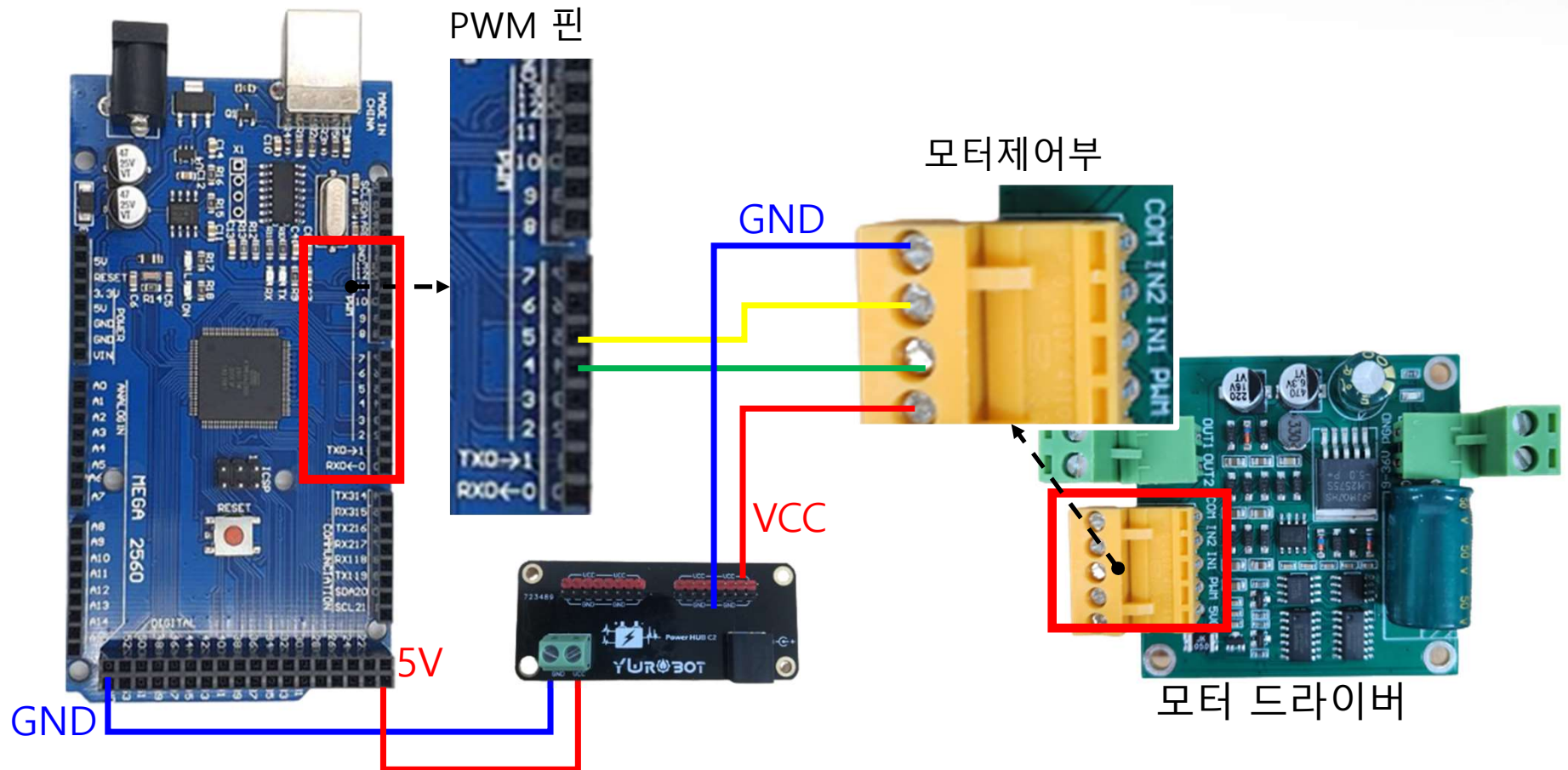
motor_test
//모터 제어 코드

//모터의 회전 속도 입력 (-255 ~ 255)
//양수는 전진, 음수는 후진
//절댓값이 클수록 모터가 빠르게 회전
static int target_speed = 50;

//모터 드라이버 IN1, IN2에 연결하는 아두이노 핀번호 입력 (모터가 반대로 동작하면 순서를 IN2, IN1로 변경)
static int test_motor1 = 2;
static int test_motor2 = 3;
```

하드웨어 – 우측 뒷바퀴 모터

■ 아두이노 – 모터드라이버

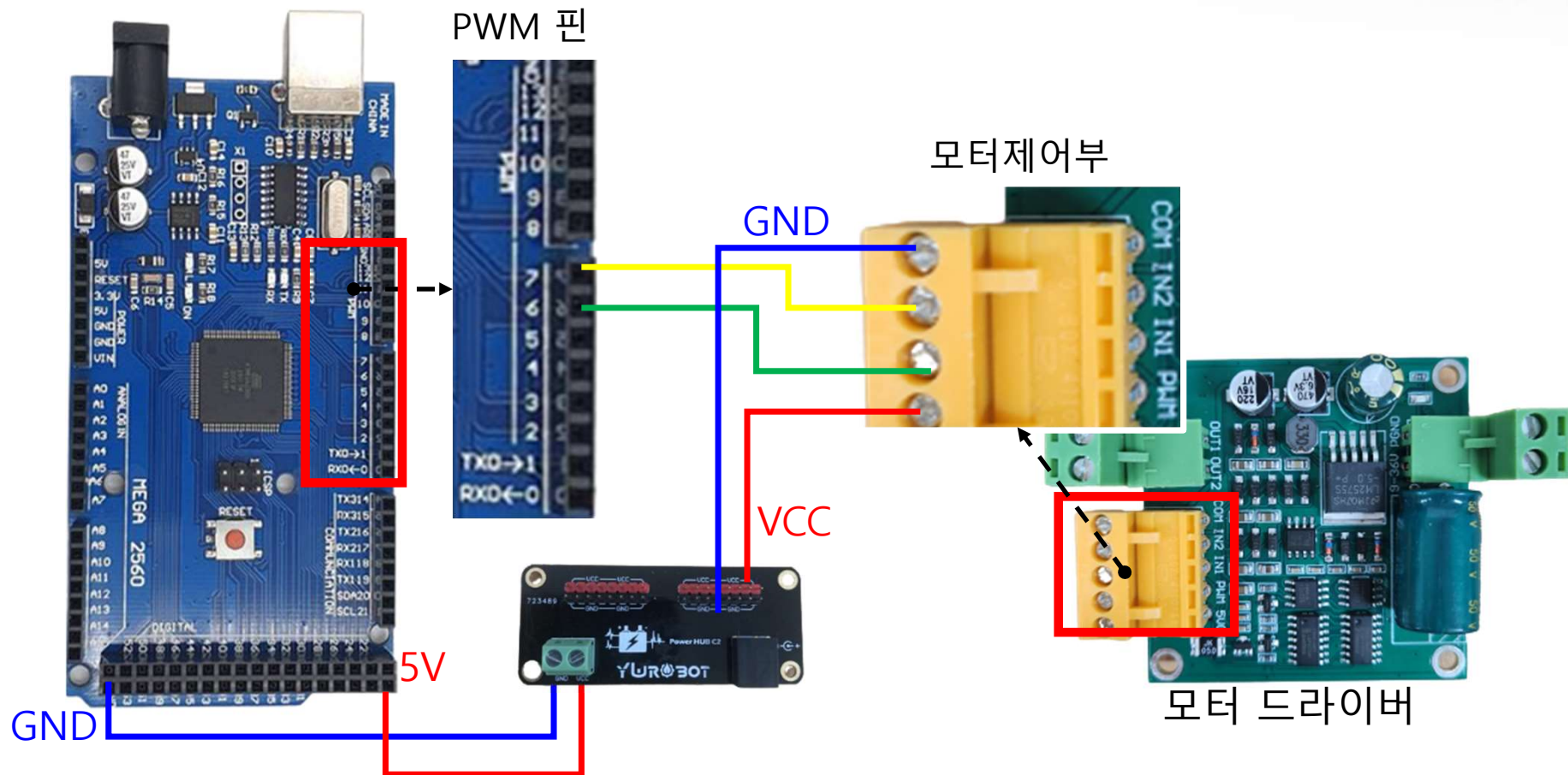


- 경로 : ~/ros2_ws/src/control/driving/driving.ino



하드웨어 – 좌측 뒷바퀴 모터

■ 아두이노 – 모터드라이버



하드웨어 – 좌측 뒷바퀴 모터 테스트

■ check_variable_resistor.ino 실행

- 경로 : ~/ros2_ws/src/control/driving/driving.ino

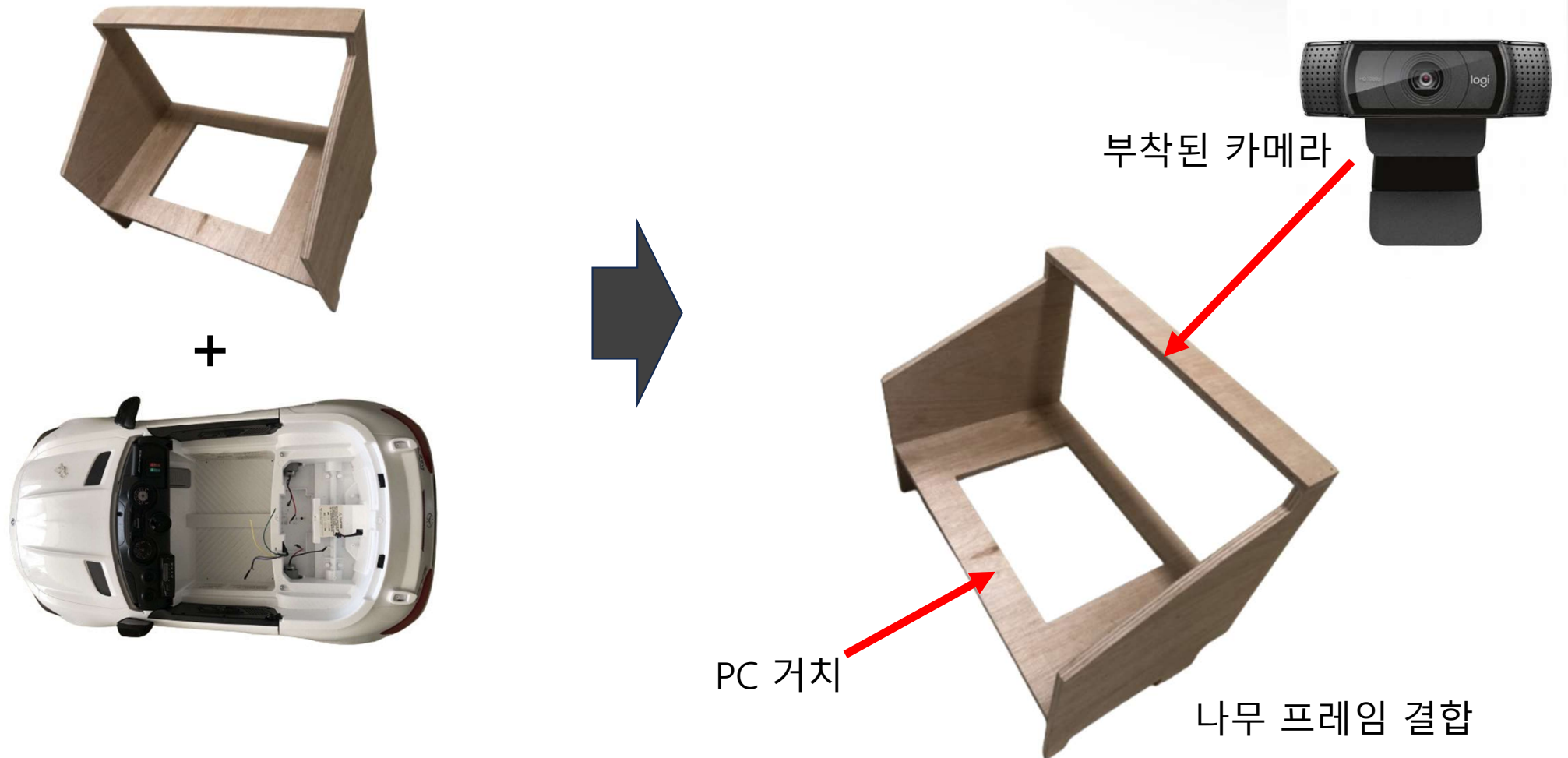
```
motor_test | 아두이노 1.8.19
파일 편집 스케치 툴 도움말

motor_test
//모터 제어 코드

//모터의 회전 속도 입력 (-255 ~ 255)
//양수는 전진, 음수는 후진
//절댓값이 클수록 모터가 빠르게 회전
static int target_speed = 50;

//모터 드라이버 IN1, IN2에 연결하는 아두이노 핀번호 입력 (모터가 반대로 동작하면 순서를 IN2, IN1로 바꿔주세요)
static int test_motor1 = 6;
static int test_motor2 = 7;
```

하드웨어 – 나무판 고정



하드웨어 – 나무판 고정



벨크로 테이프 활용



하드웨어 – 나무판 고정

주의사항: 차량을 잡을 때는 나무 프레임을 잡지 말고, 차량 본체인 플라스틱 부분을 잡을 것.
(노트북 PC 떨어져서 손상 될 수 있음)



나무프레임 앞부분과 차량 문
뒷부분을 일치시키기



카메라는 윗 프레임 중앙에 위치

하드웨어 – 라이다

- 라이다 다리부분을 차체 앞부분에 붙이기



H-Mobility Class

자율주행 심화과정

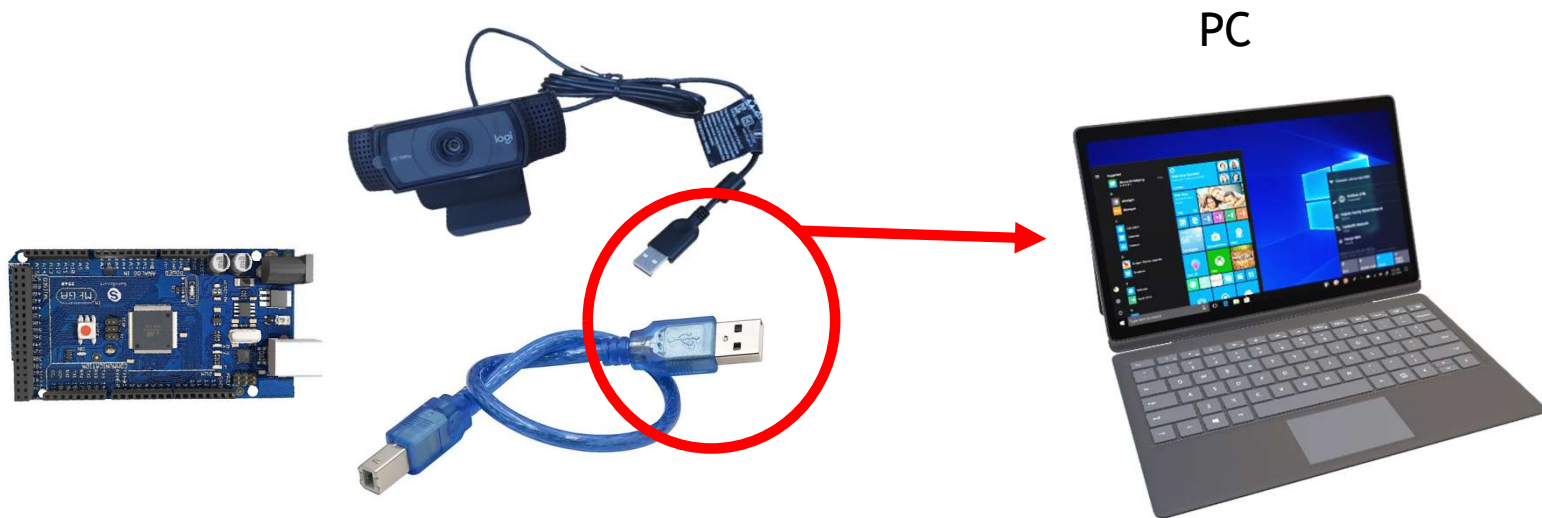
데이터셋 수집



데이터셋 수집

■ 수집하기 위한 준비

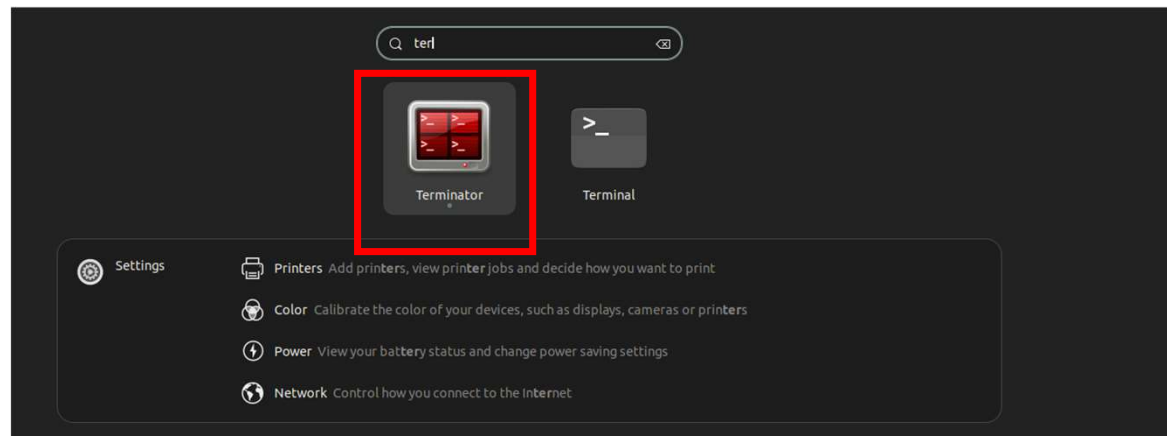
- 자율주행 자동차에 카메라 고정된 센서 확인
- Arduino를 PC에 연결 확인
- check_variable_resister.ino 파일로 가변저항 제일 왼쪽, 오른쪽 값을 추출



데이터셋 수집

■ Terminator 실행

- 'Terminator' 검색 후 실행



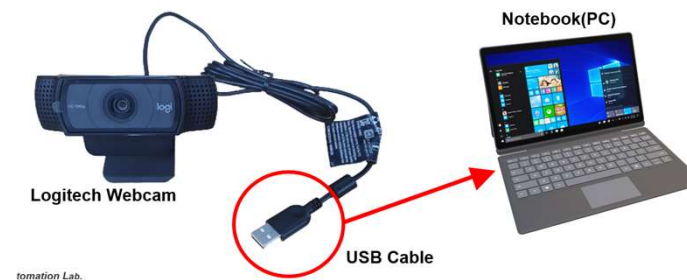
데이터셋 수집

■ 카메라 연결 및 확인

- Terminator에서 'ls /dev/video*' 명령어 입력후 'Enter'로 실행

```
siwoo@siwoo-800G5M-800G5W: ~  
siwoo@siwoo-800G5M-800G5W: ~ 80x24  
siwoo@siwoo-800G5M-800G5W:~$ ls /dev/video*  
/dev/video0 /dev/video1
```

- 카메라 연결



데이터셋 수집

■ 카메라 연결 및 확인

- 'ls /dev/video*' 명령어 입력하여 실행 후 새로 생긴 짝수 번호 확인

```
siwoo@siwoo-800G5M-800G5W: ~/ros2_ws/src
siwoo@siwoo-800G5M-800G5W: ~/ros2_ws/src 80x24
siwoo@siwoo-800G5M-800G5W:~/ros2_ws/src$ ls /dev/video*
/dev/video0 /dev/video1 /dev/video2 /dev/video3
```

데이터셋 수집

■ 아두이노 연결 확인 및 권한 설정

- 'ls /dev/ttyA*' 명령어 입력하여 실행 후 아두이노 연결 확인

```
automationlab@automationlab: ~
automationlab@automationlab: ~ 80x24
automationlab@automationlab:~$ ls /dev/ttyA*
/dev/ttyACM0
```

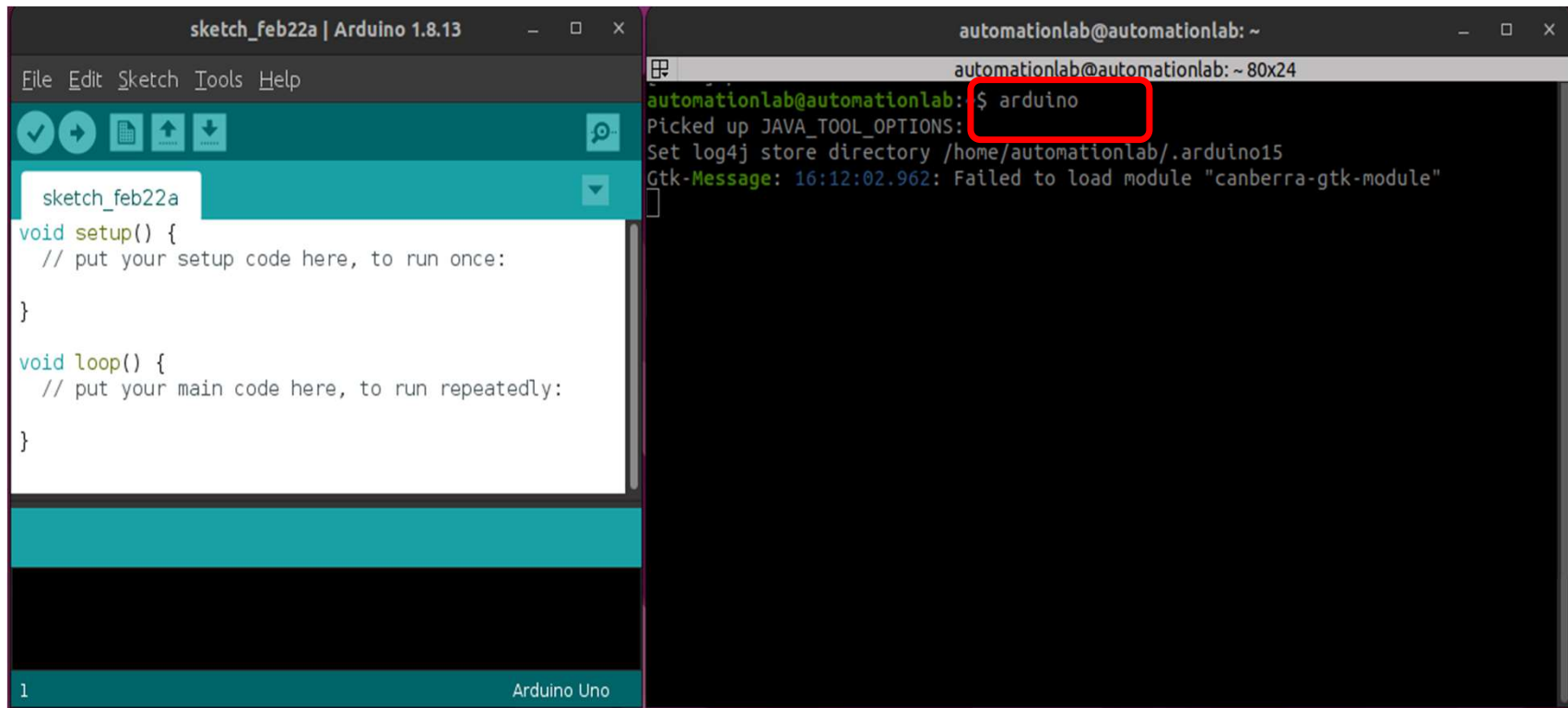
- 'sudo chmod a+rw /dev/ttyACM0' 명령어 입력하여 권한 설정

```
automationlab@automationlab:~$ sudo chmod a+rw /dev/ttyACM0
[sudo] password for automationlab: 
```

PC 비번 입력

데이터셋 수집

- Arduino IDE 실행
 - arduino 명령어 입력하여 실행



The image shows two side-by-side windows. The left window is the Arduino IDE, titled 'sketch_feb22a | Arduino 1.8.13'. It displays a sketch with the following code:

```
void setup() {  
  // put your setup code here, to run once:  
}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
}
```

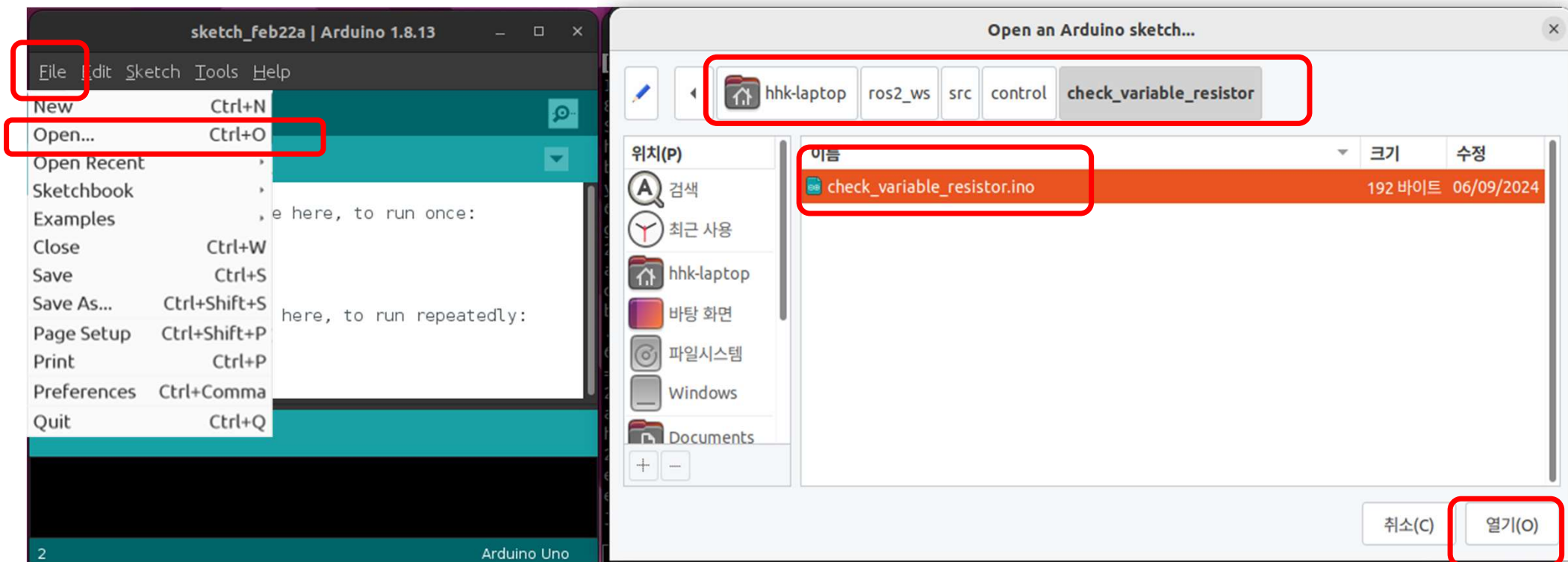
The right window is a terminal, titled 'automationlab@automationlab: ~'. It shows the command 'arduino' being entered at the prompt 'automationlab@automationlab: ~\$'. The command is highlighted with a red box. Below the command, the terminal output shows:

```
Picked up JAVA_TOOL_OPTIONS:  
Set log4j store directory /home/automationlab/.arduino15  
Gtk-Message: 16:12:02.962: Failed to load module "canberra-gtk-module"
```

데이터셋 수집

■ Arduino IDE 실행

- Arduino IDE에서 **File** 클릭 -> **Open** 클릭
- (ros2_ws/src/control/check_variable_resistor/~.ino) 선택 후 Open 버튼 클릭



데이터셋 수집

- Arduino 보드 Pin 연결에 따른 Pin번호 수정
 - int sensorPin = A2;
 - 가변저항에 연결된 아날로그 핀 번호로 입력하기

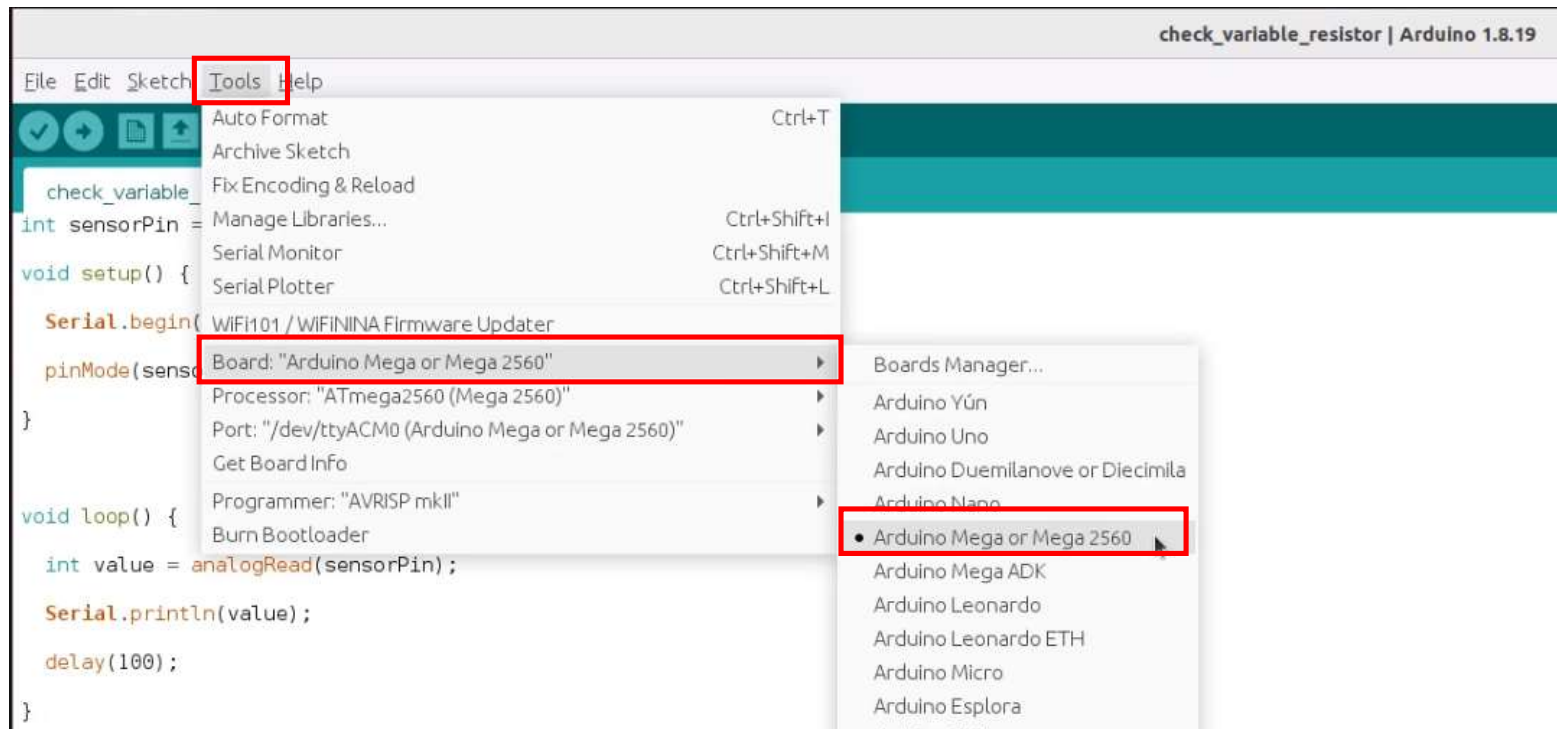
```

check_variable_resistor | Arduino 1.8.13
File Edit Sketch Tools Help
[Icons]
check_variable_resistor
int sensorPin = A2;
void setup() {
  Serial.begin(9600);
  pinMode(sensorPin, INPUT);
    
```

데이터셋 수집 환경 세팅

■ Arduino IDE에서 아두이노 보드 선택

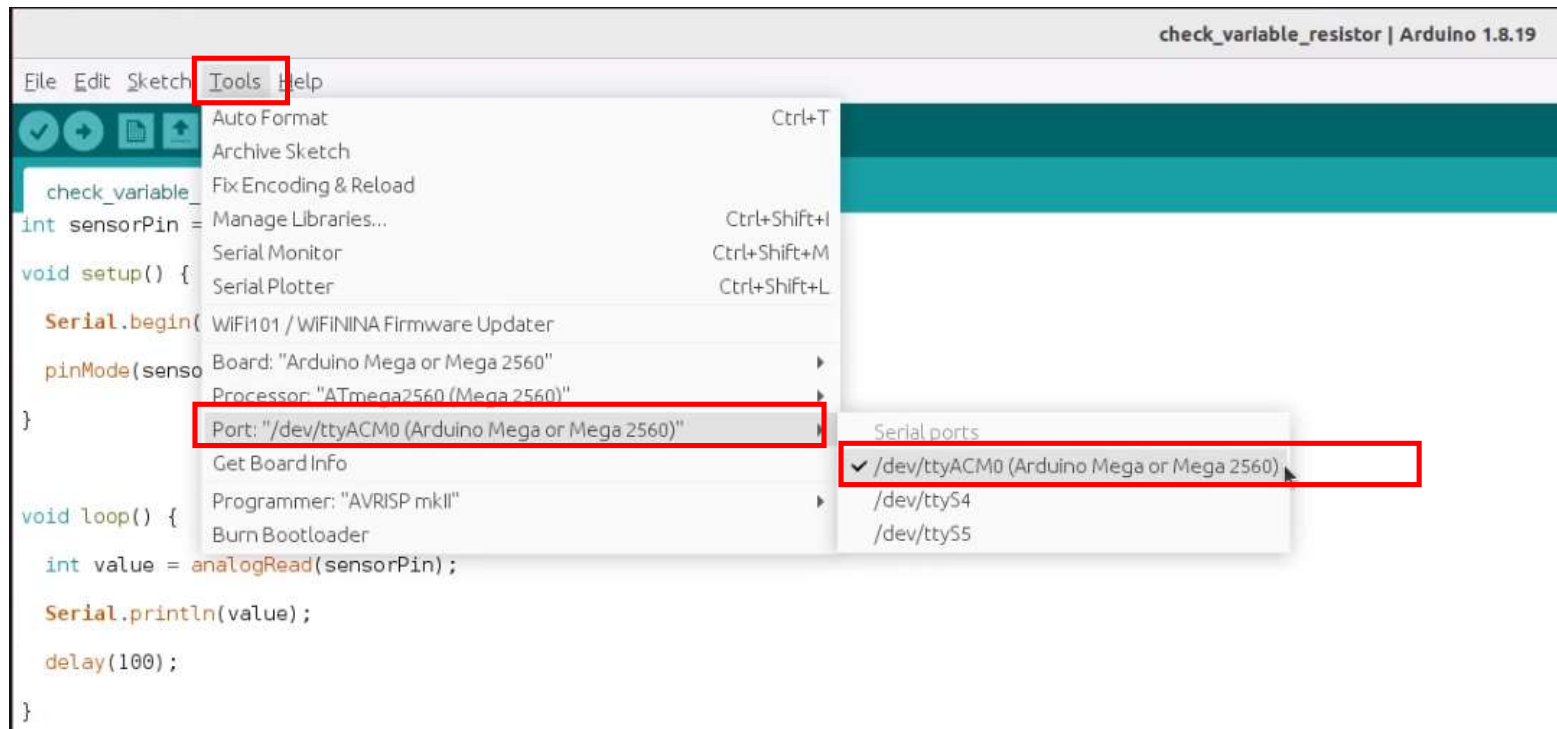
- Tools -> Board: "Arduino Mega or Mega 2560" -> Arduino Mega or Mega 2560 클릭



데이터셋 수집

■ Arduino IDE에서 아두이노 포트 선택

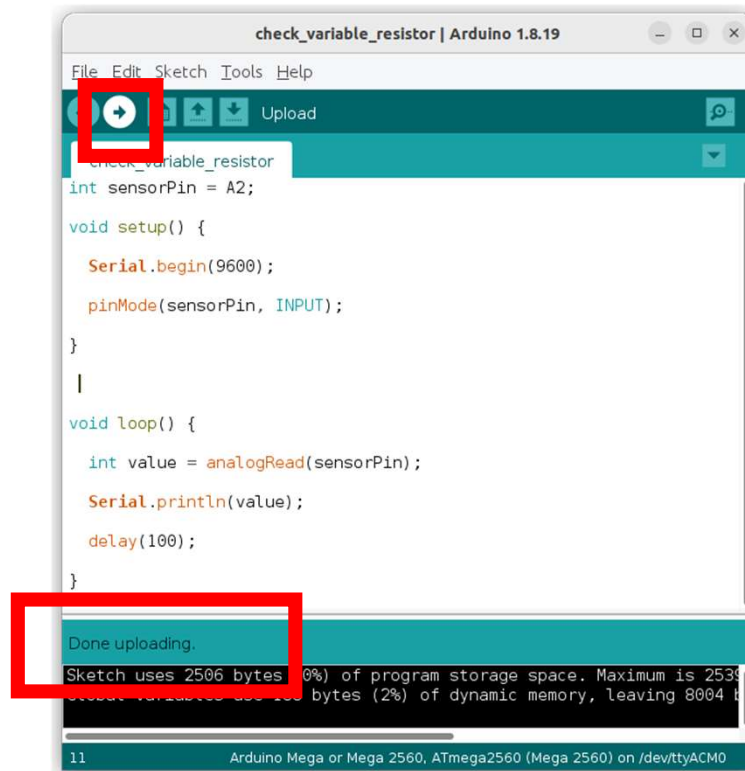
- Tools 클릭 -> Port: "/dev/ttyACM0" 클릭 -> /dev/ttyACM0 클릭



데이터셋 수집

■ Arduino IDE에서 아두이노 실행

- Arduino IDE에서 아두이노 "업로드 (🔼)" 클릭
- 화면 하단의 업로드 완료 메시지(Done uploading.) 확인



데이터셋 수집

- Arduino IDE에서 아두이노 실행
 - Arduino IDE에서 아두이노 시리얼 모니터 띄우기 ( 클릭)



데이터셋 수집

- 가변저항 좌측끝, 우측끝 값 추출
 - 조향을 최대한 좌측으로 꺾은 뒤 가변저항 값 확인하여 메모



데이터셋 수집

- 가변저항 좌측끝, 우측끝 값 추출
 - 조향을 최대한 우측으로 꺾은 뒤 가변저항 값 확인하여 메모



수동주행 및 데이터 수집

- driving.ino 에 핀번호 (모터 드라이버 IN1/IN2, 가변저항 OUT에 연결 된 핀) 입력
- 가변저항 최대 좌우측 값 입력

```
4 // 핀 번호 변수
5 const int STEERING_1 = 2;
6 const int STEERING_2 = 3;
7 const int FORWARD_RIGHT_1 = 4;
8 const int FORWARD_RIGHT_2 = 5;
9 const int FORWARD_LEFT_1 = 6;
10 const int FORWARD_LEFT_2 = 7;
11 const int POT = A2;
```

```
16 // 가변저항 값 범위
17 const int resistance_most_left = 460;
18 const int resistance_most_right = 352;
```

수동주행 및 데이터 수집

- DATA_PATH: 데이터 수집될 디렉터리 지정
- CAMERA_NUM: ls /dev/video* 을 통해 확인하는 웹캠 장치 번호
- SERIAL_PORT: ls /dev/ttyACM* 을 통해 확인하는 아두이노 장치 이름
- MAX_STEERING: 조향 단계 수 설정

```
DATA_PATH = "./Collected_Datasets"  
CAMERA_NUM = 0  
SERIAL_PORT = "/dev/ttyACM0"  
MAX_STEERING = 7 # 사용자 정의 최대 조향 단계
```

수동주행 및 데이터 수집

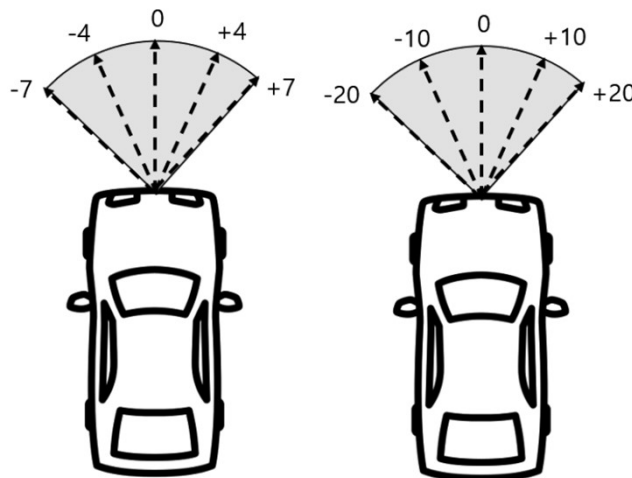
- 주의: 조향 단계 수는 driving.ino와 data_collection.py를 일치시킬 것

```
DATA_PATH = "./Collected_Datasets"  
CAMERA_NUM = 0  
SERIAL_PORT = "/dev/ttyACM0"  
MAX_STEERING = 7 # 사용자 정의 최대 조향 단계
```

<data_collection.py>

```
20 // 조향 최대 단계 수 (한 쪽 기준)  
21 const int MAX_STEERING_STEP = 7;
```

<driving.ino>



7단계 (좌), 20단계 (우)

수동주행 및 데이터 수집

■ data_collection.py 실행

터미널 창 실행 후 아래의 명령어들 입력하여 실행

- `sudo pip install keyboard`
- `sudo pip install pyserial`
- `sudo python3 ~/ros2_ws/src/data_collection/data_collection.py`

데이터셋 수집

■ data_collection.py 실행

- 아래와 같은 키 조작으로 차량 동작 및 도로 사진 데이터 수집
- 앞바퀴가 한쪽으로 돌아간 채로 움직이지 않을 때
 - 아두이노 꺾힌 조향 모터 핀 확인
- 전진/후진이 반대로 될 때
 - 아두이노에 꺾힌 뒷바퀴 모터 핀 확인

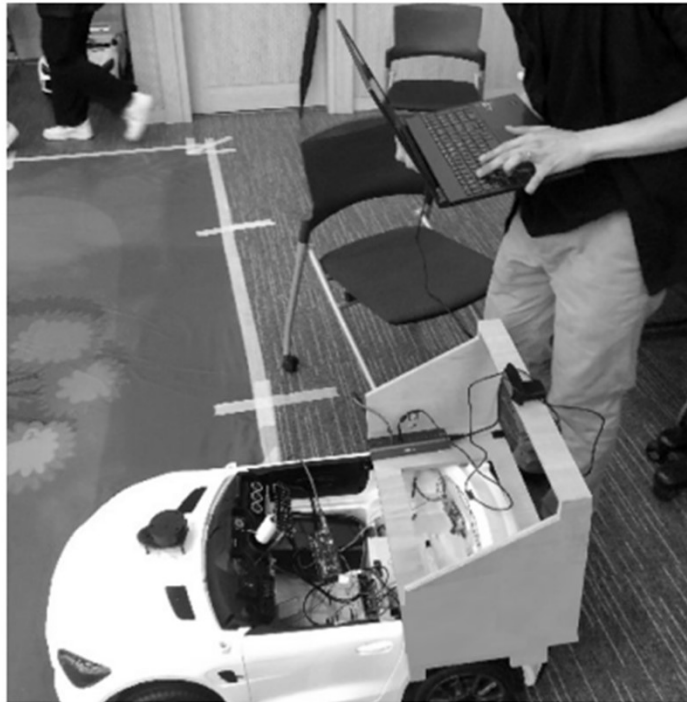
```
...
키 조작

w : 속도 +10
s : 속도 -10
a : 회전 -1    (- : 좌회전, + : 우회전)
d : 회전 +1

c: 카메라 프레임 캡처

r : 일시정지
f : 종료
...
```

수동주행 및 데이터 수집



H-Mobility Class

자율주행 심화과정

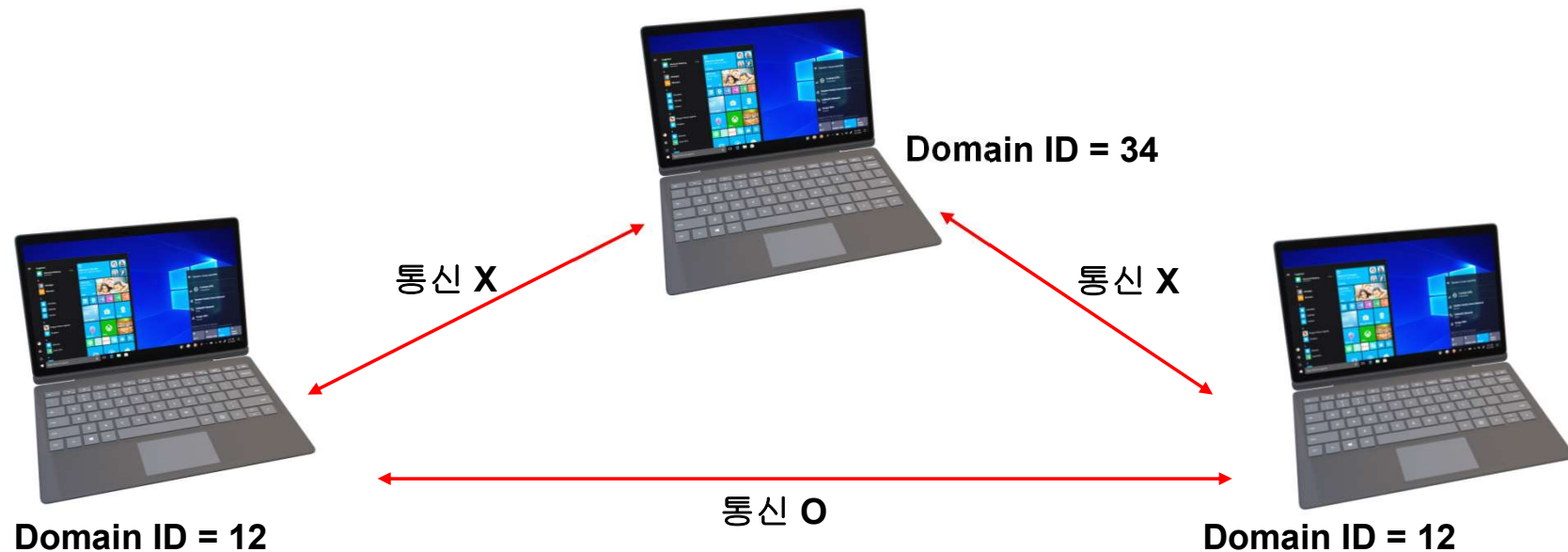
ROS2 실습



ROS2 실습 – 외부통신 차단

■ ROS2 Domain ID

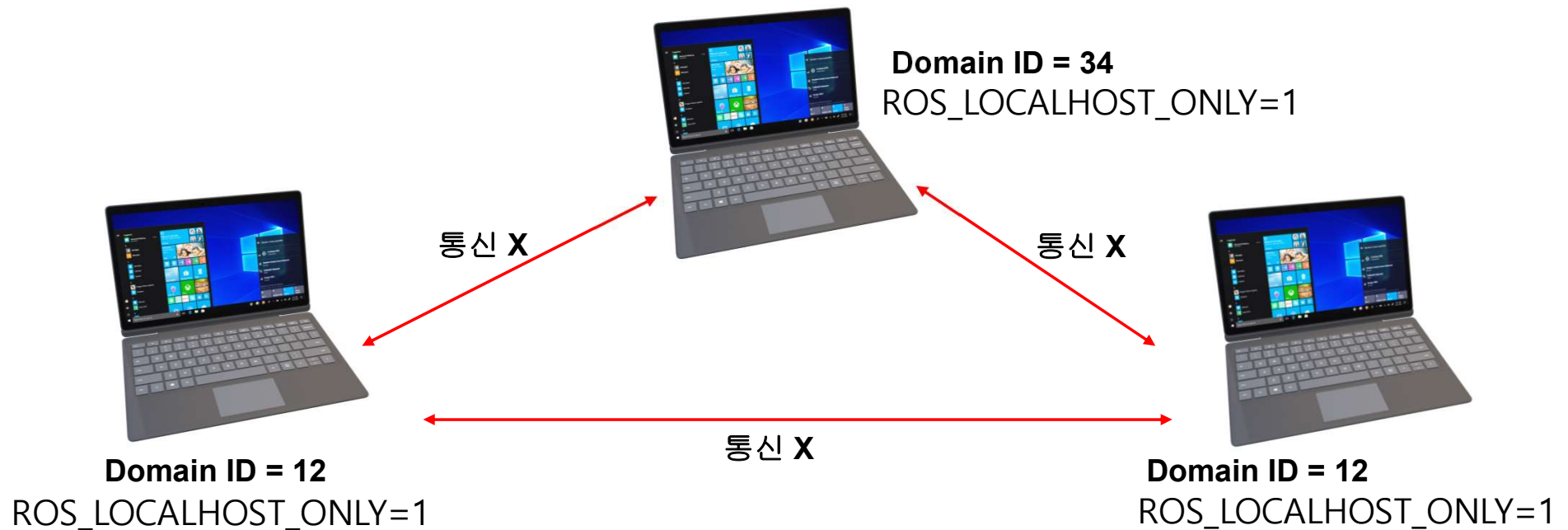
- 동일한 Domain ID를 가진 디바이스간 메시지 교환



ROS2 실습 - 외부통신 차단

■ ROS_LOCALHOST_ONLY

- 다음 명령어 실행시 외부 기기와는 통신하지 않음.
 - export ROS_LOCALHOST_ONLY=1



ROS2 실습 - 외부통신 차단

■ ROS_LOCALHOST_ONLY

- 다음 명령어 실행시 외부 기기와는 통신하지 않음.
 - `export ROS_LOCALHOST_ONLY=1`
- 다음 명령어 실행시 매번 위의 명령어를 실행하지 않아도 됨.
 - `echo "export ROS_LOCALHOST_ONLY=1" >> ~/.bashrc`
 - 입력 후 재실행

ROS2 실습 – 디바이스 이름

■ 아래의 명령어로 아두이노, 카메라, 라이다 장치 이름을 확인

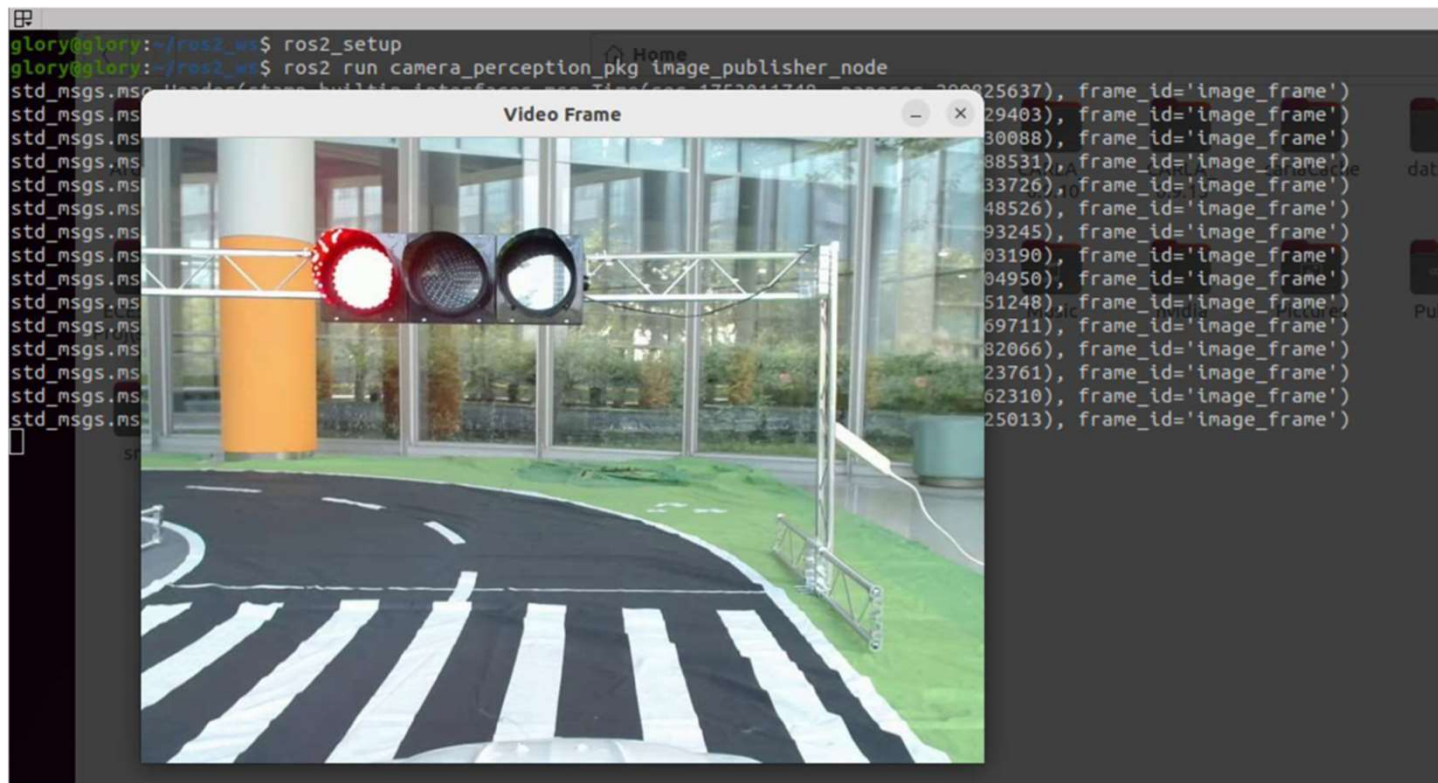
- `ls /dev/ttyACM*`
- `ls /dev/video*`
- `ls /dev/ttyUSB*`

■ 장치 이름이 바뀌는 경우?

- 라이다 장치가 분명히 `ttyUSB0` 이었는데 프로그램이 실행이 안돼서 확인해보니 `ttyUSB1`로 변경되어 있다면?
- 원인은 프로그램(`lidar_publisher_node`)이 동작하는 중에 라이다 장치를 물리적으로 제거한 뒤 재연결 하였기 때문일 가능성이 높음
- 카메라, 아두이노도 마찬가지로 프로그램이 동작하는 중에 커넥터를 뺀 뒤 재연결하면 동일한 문제 발생
- 따라서 긴급상황을 제외하고는 디바이스를 노트북에서 제거하기 전에 해당 디바이스와 관련된 모든 프로그램을 종료할 것을 권장

ROS2 실습 – ros2 run 명령어

- Ctrl + Alt + T (터미널 실행)
- `ros2 run camera_perception_pkg image_publisher_node`
 - 이미지가 출력되는지 확인



- The image is a screenshot of a computer screen displaying a ROS2 environment. On the left, a video window titled "Video Frame" shows a first-person view from a car driving on a track. The track has a green grassy area on the right and a dark asphalt road with white dashed lines on the left. In the background, there are large orange pillars and a glass-walled building. On the right, a terminal window shows the command-line interface. The prompt is "glory@glory:~/ros2_ws". The user has entered "ros2 setup" and "ros2 topic echo /image_raw". The output shows the message header and data for the "image_raw" topic, including the timestamp, frame ID, and the image data itself. The image data is a large, dark, abstract shape, likely a placeholder or a corrupted image.

ROS2 실습 – launch_pkg (main.launch.py)

- `ros2 launch launch_pkg main.launch.py`

```
main.launch.py X
C: > Users > Glory > Desktop > H-Mobility-Autonomous-Advanced-Course > src > launch_pkg > launch > main.launch.py
1  from launch import LaunchDescription
2  from launch_ros.actions import Node
3
4  def generate_launch_description():
5      return LaunchDescription([
6          Node(
7              package='camera_perception_pkg',
8              executable='image_publisher_node',
9              name='image_publisher_node',
10             output='screen'
11         ),
12         Node(
13             package='camera_perception_pkg',
14             executable='yolov8_node',
15             name='yolov8_node',
16             output='screen'
17         ),
18         Node(
19             package='camera_perception_pkg',
20             executable='lane_info_extractor_node',
21             name='lane_info_extractor_node',
22             output='screen'
23         ),
```